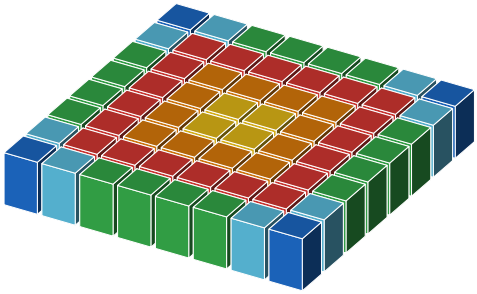
The 16GB Ceiling

Test-Time Training for ARC-AGI-2 on One Consumer GPU

Running the 2025 ARC Prize winning recipe, per-task test-time training
with augment-infer-reverse-vote selection, on one 16 GB card



16.28%

on a single 16 GB GPU

up from 12.79%, 13× faster

ARC Prize 2026 · Paper Track

ARC Prize 2026 Paper Track entry

Open source under MIT-0

Abstract

We run the 2025 ARC Prize winning recipe, a small grid model with per-task test-time training (TTT) and augment-infer-reverse-vote (AIRV) selection, on a single 16 GB consumer GPU, and ask what actually binds at that scale. The binding constraint is generation throughput, not model quality and not memory. Moving decode off the Hugging Face eager path onto llama.cpp buys a 13× speedup, turns an overnight failure-prone evaluation into a sub-90-minute one, and is what makes per-task TTT tractable at all, by hot-swapping each task's adapter on the fast engine. Building on the NVARC Qwen3-4B grid model, the system scores **12.79%** without TTT on the 120-task public evaluation set. The lift from TTT depends on regularization. A naive adapter overfits and ties the baseline, while a regularized, scale-tuned configuration reaches **17.6%** on the 84 trainable tasks and lifts the full evaluation to a blended **16.28%**. On the hidden main-track leaderboard the same recipe scores **13.61%** with TTT and 6.53% without, and the public-to-hidden transfer is itself a finding. The no-TTT baseline halves, and across the follow-on submissions every candidate-side lever that lifted or tied the public evaluation regressed the hidden score, so public-eval gains must be re-validated in the exact deployed condition before they are trusted. The climb then came from the training side rather than the vote. Deepening the regularized adapter lifted the hidden score to **18.19%**, a 4.58-point gain and the first configuration to beat the shipped operating point. The number is not a wall but climbs under the 16 GB ceiling as the per-task training deepens, while the methods that reach the field's level, heavy rank-256 test-time training at an eight-thousand-token context, run far slower than the contest budget allows on one card. That configuration turns out to fit the card's memory once the attention kernel is chosen deliberately, which corrects an earlier conclusion of our own, so the binding resource is time rather than capacity. We then measure the gap rather than assert it. Reproducing one such public pipeline on the contest's own four-L4 node, and repairing two defects the community port had introduced into it, scores **29.90%** as the mean of twelve runs of one immutable kernel version. That result is a reproduction of named public work and is not a single-card result, and we report it as the measured price of the constraint, roughly twelve points against the best configuration we could serve inside the single-card budget.

We quote a mean rather than a best because of what those twelve runs turned out to be. A single Kaggle kernel version, resubmitted twelve times across twenty-five days with nothing changed and verified byte-identical by script version rather than by description, scored **30.56, 32.64, 28.47, 31.39, 30.56, 30.56, 28.89, 28.89, 30.56, 28.47, 28.89** and **28.89**: a spread of **4.17 points**, ten task credits, and a standard deviation of **1.34** with a 95% confidence interval of $[0.95, 2.27]$. Measured against it, *every* build this project called a regression, and both public notebooks we ran as controls, fall inside two standard deviations. So does the 32.64 we spent weeks treating as our best result: it is the maximum of twelve draws from a distribution centred on 29.90, an order statistic rather than a score, and we now report the pipeline as 29.9 ± 1.3. That is the paper's most transferable result. At this operating point a single hidden-set score does not resolve the method differences the field routinely reports, our own included; separating two configurations at 80% power would need roughly **four points of separation and three runs per arm**, six submission days for one comparison. We only found this because we spent twelve submissions re-running one unchanged artifact.

Past that point the work becomes an account of what it takes to trust a number at all in this setting, and four results carry more than the accuracies do. A public notebook *displaying* 33.89 scored **28.06** when we forked it unmodified and submitted it ourselves, which is below our own build; we had spent a week producing three quantified explanations of a gap that did not exist, and we report them in the order we believed them. The selection ceiling turns out to be closeable by proof rather than by sweep: on a dumped candidate pool, the correct grid is dominated in every recorded statistic for most of the outputs that leak, so no monotone rule over those statistics can recover them and the family of

sixteen reweightings we were about to test was refuted before it ran. A scheduling hypothesis that the reclaimed time budget could fund restarts measured a 4.4-point gain and was **nothing**: rerunning the same seed disagrees with itself as often as different seeds disagree with each other, so the apparent effect was the decoder's own arithmetic, and only a same-seed replicate arm separated the two. And the residual is capability rather than budget, since 62% of tasks never place the correct grid in the candidate pool under any of sixteen views while consuming 69% of the compute. We give an honest accounting of what the recipe costs and where the effort should go for anyone working with one consumer GPU rather than the winner's hardware. The contest itself provides four L4 GPUs, 96 GB together, where the offline frontier now leads near 66% on the strength of large-scale synthetic-data pretraining, so we read the 16 GB result as the efficient floor of a larger climb, and the ceiling in our title is the 16 GB card itself, the hardware wall that shapes every choice here and not a limit on the task. All code, the conversion recipe, and a development log of every measurement are open source under MIT-0.

Contents

1 Introduction **4**

1.1 Efficient abstraction is the point 4

1.2 Contributions 5

2 Related Work **6**

3 System and Method **7**

3.1 Model and prompting 7

3.2 The engine is the bottleneck 7

3.3 AIRV: augment, infer, reverse, vote 8

3.4 Per-task test-time training 9

4 Results **10**

4.1 Two regimes, and the leaderboard progression 10

4.1.1 A number that did not reproduce, and a week spent explaining it 19

4.2 The engine speedup 22

4.3 Test-time training 23

4.4 Ablations on scale, colour, candidates, filtering, and the big-grid wall 24

4.5 The search-throughput frontier, and a correct decoder that scores worse 25

4.6 Is the base limited by synthetic rule diversity? 26

4.7 A generator corpus for ARC-AGI-2, and a trap worth reporting 28

5 Discussion **31**

5.1 Five lessons that generalize . . . 31
5.2 Accuracy per GPU-hour . . . 32
5.3 Limitations and ongoing work . . . 32
5.4 Scaling to the full contest hardware . . . 35
5.5 A routed dual-arm system . . . 36
5.6 A stronger public decoder, and its coupling to training . . . 37
5.7 The selection leak, measured on the candidate pool . . . 39
5.8 Revisiting the big-grid wall with the host's memory . . . 42
5.9 What the budget is actually spent on, and a policy the pipeline cannot express . . . 43
5.10 Perceptual pre-fitting on the test input . . . 49
5.11 The decode batch width is a memory cliff, and the deployed setting is over it . . . 51
5.12 The complete ledger: what is closed, what is open, and why . . . 52

6 Conclusion **59**

A Reproducibility **60**

B Extended results **61**

C Engineering log of walls and dead ends **63**

D Prompt and serialization format **64**

E Data and model provenance **64**

F The mathematics of the measurement **65**

F.1 The score and its quantum . . . 65
F.2 Resolving power of a paired comparison . . . 65
F.3 A family of rules is a multiplicity . . . 66
F.4 Why the pool cannot rank its own truth . . . 67
F.5 Time as the decision variable . . . 74
F.6 Composition and coverage . . . 77

Reader's guide. Section 1 motivates efficient ARC and states the contributions. Section 2 places the work against the 2025 winning systems. Section 3 is the method, covering the model and prompt format (3.1), the engine bottleneck and its fix (3.2), the AIRV selection rule (3.3), and the per-task TTT bridge (3.4). Section 4 reports the numbers. Section 5 draws the lessons. Readers who want to reproduce the system should go straight to Appendix A. The per-scale tables and significance detail are in Appendix B,

the honest list of dead ends in Appendix C, the exact prompt and 16-token vocabulary in Appendix D, and data licensing and BibTeX in Appendix E.

1 Introduction

1.1 Efficient abstraction is the point

ARC-AGI-2 [1, 2] is a benchmark of abstraction and reasoning. Each task gives a few input/output grid pairs that demonstrate a transformation, and the solver must apply that transformation to a held-out input. The tasks are designed to be easy for humans and hard for machines, and the competition is run *offline*, which rules out frontier API models and forces every entrant to ship a self-contained system. The 2025 ARC Prize was won by NVARC at 24.0% on ARC-AGI-2 using a 4B grid model with per-task TTT [3, 4]. The same recipe reproduced across the top teams. They fine-tune a small model on augmented grid data, train a fresh low-rank adapter on each test task, then sample many augmented candidates and vote.

The recipe is effective but front-loaded with compute. The base models are pre-trained on clusters of H100 or GH200 GPUs, though the winning offline submission itself runs on four L4 GPUs. Most researchers do not have that hardware, and the ARC Prize's stated mission is *efficient* skill acquisition, not skill bought with compute. We therefore characterize the recipe under a constraint we think is widely shared, a single RTX 5070 Ti with 16 GB, on the Blackwell architecture, running native Windows. The aim on one card is not to top the leaderboard but to build a working, reproducible system, measure honest numbers, and map where the small model stops being enough. That map is the efficient floor of a larger climb. The ceiling in the title is the hardware, 16 GB of VRAM, not a score, and the number climbs beneath it as the method deepens, from 13.61 to 18.19 on the hidden set (Section 4). We also put a number on the ceiling itself. Reproducing a public pipeline that cannot fit one card, on the hardware the contest provides, averages 29.90% over twelve runs of one immutable artifact on the same hidden set, so the constraint is worth roughly twelve points rather than an unknown amount. We report the mean and not the 32.64 peak, because those twelve runs differ from each other by 4.17 points with no code change at all, which makes the peak an order statistic. The same contest hands every entrant four L4 GPUs, 96 GB together, and the offline frontier on that hardware has already passed 50% with refinement loops and program synthesis [3]. We read the 16 GB walls not as the limit of the task but as the exact places scaling up has to break, and Section 5.4 lays out that path. Figure 1 shows a representative task our system solves end to end.

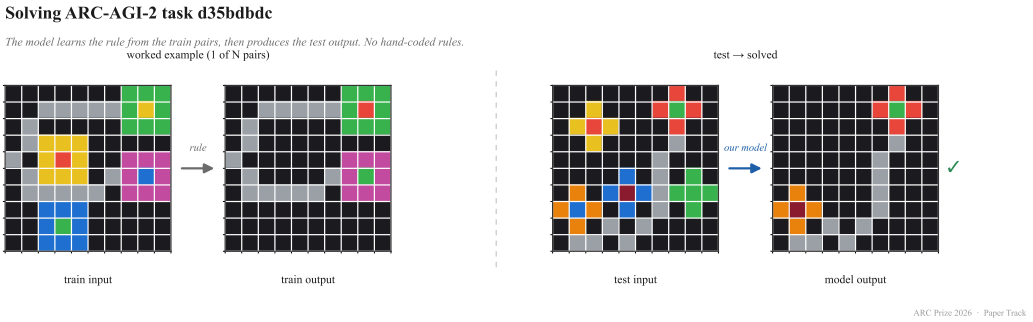


Figure 1: A real ARC-AGI-2 evaluation task solved by the no-TTT pipeline, shown as colour grids. The few-shot train pairs establish the rule, and the model produces the held-out test output exactly. Grids are rendered with the model's own 16-colour serialization (Appendix D).

1.2 Contributions

1. A reproducible offline ARC-AGI-2 system for the custom-vocabulary NVARC model on a single 16 GB card, with a clean **12.79%** no-TTT baseline (Section 4, Appendix A).
2. The finding that, on this hardware class, the inference *engine* dominates every other knob, since moving decode off the Hugging Face eager path onto llama.cpp is worth a 13× speedup (Section 3.2).
3. A practical bridge that makes per-task TTT runnable on one card by hot-swapping GGUF adapters on the fast engine, and a measured, regularization-dependent lift to a **16.28%** blend (Section 3.4, 4.3).
4. An "accuracy per GPU-hour" accounting that names the three walls a compute-poor practitioner hits first, the decode ceiling, the VRAM limit on large-grid TTT, and adapter overfitting (Section 5).
5. The measured finding that public-eval gains do not transfer to the hidden set. The no-TTT baseline halves from 12.79% to 6.53%, and across seven leaderboard submissions every candidate-side lever that lifted or tied the public evaluation regressed the hidden score when deployed (Section 4, 5.1).
6. **A measured error bar for the benchmark itself.** Twelve submissions of one immutable Kaggle kernel version span 4.17 points on the hidden set with a standard deviation of **1.34**, so a single score cannot resolve differences below roughly four points without three runs per arm. We believe this is the first such measurement published for ARC-AGI-2, and it retires a class of claim rather than a single claim: every regression we recorded, both public-notebook controls, and our own best configuration all fall inside 1.5 standard deviations (Section 4).
7. **A ledger of forty measured levers, none of which moved the score**, organised by where each sits relative to the candidate pool, together with the observation that eleven of them were closed with no GPU at all by pricing the idea before building it (Section 5.12).
8. A reading of why those walls are a property of a small model on one card rather than of the task, and a concrete path that scales the same verifier, TTT, and selection onto the full 96 GB the contest provides, where the offline frontier now leads near 66% (Section 5.4).

9. A complete open-source release, the conversion patches, the pipeline, the TTT harness, and a development log of every measurement and dead end behind these numbers (Appendix C).

2 Related Work

ARC and the measure of intelligence. ARC was introduced to measure skill-acquisition efficiency rather than skill itself [1]. ARC-AGI-2 [2] raises compositional difficulty while keeping tasks human-solvable, widening the human/machine gap that frontier models had begun to close on ARC-AGI-1.

Test-time training. The dominant 2025 approach fits a small per-task adapter at inference, trading test-time compute for generalization. The ARC Prize 2025 technical report [3] identifies this, together with heavy synthetic pre-training and disciplined engineering, as the core of the winning NVARC solution [4], whose released model we build on directly. We hold this recipe fixed and ask what survives a single consumer GPU.

Two different things are called test-time training. The name collides, and the collision matters for reading anything in this paper against the wider literature. In the ARC line, TTT means *supervised* fine-tuning on a task's demonstration pairs, which are labelled data the task supplies; this is the sense used throughout here and the sense the ARC Prize 2024 report fixes [13]. In the older line beginning with Sun et al. [16], TTT means *self-supervised* adaptation on the unlabeled test instance itself, with descendants using entropy minimisation or masked autoencoding [17]. Akyurek et al. [15] inherited the ARC-local sense and flagged the difference themselves. The two procedures share a name and almost nothing else.

A third sense has since arrived and is worth separating explicitly, because it is the one a reader is most likely to encounter now. NVIDIA's TTT-E2E [20] compresses a long context *into the weights* by continued next-token prediction at inference, on top of a meta-learned initialization, achieving inference latency constant in context length where attention is quadratic. It is an architecture rather than a procedure: the meta-learned pre-training is what makes the test-time descent work, so it cannot be retrofitted onto an existing transformer's weights. It is also lossy by design, keeping what is predictive and discarding detail. Neither property fits the setting here. Our prompts are about 2,000 tokens against an 8,192 cap that we measure to be non-binding, so the complexity it optimises is slack for us; and ARC is scored on exact grid equality, where 88% of a median output is a verbatim copy of its input, so discarding detail discards the answer. We name it because the acronym now covers three distinct things — per-task supervised fine-tuning, self-supervised instance adaptation, and context compression into weights — and papers in this area are increasingly ambiguous about which they mean. The one idea that does transfer is the meta-learned initialization, which is a different object from the cross-task warm start we close in Section 5.12: that transfers a neighbour's adapter, whereas meta-learning optimises an initialization *through* the adaptation step.

The distinction has a consequence we inherit rather than choose. Our pipeline descends from the ARChitects [14], who state that they compute gradients only on output grids, so that "the model is never tasked with predicting an input grid". The test input, the single largest piece of task-specific information available, therefore reaches our adapter only as a decode-time prompt and never as a loss target. Whether that is a missed opportunity or a correct design choice is an empirical question, and Section 5.10 answers

it for our system.

Augmentation and voting. Predicting under the eight dihedral symmetries of the grid, inverting each prediction back to the original frame, and voting is a standard self-consistency device for ARC. It is cheap, training-free, and provides a usable confidence signal (Section 3.3).

Synthetic data. Procedural generators such as RE-ARC [6] and LLM-mutated corpora supply the augmented grid data these models are trained on. We use RE-ARC only in a base-model probe (Section 5.3), not in the results below.

Efficient and on-device inference. Our engineering contribution, getting a custom-vocabulary grid transformer to decode near the memory-bandwidth roofline on a consumer card and then serving per-task adapters on it, sits in the practical lineage of quantized GGUF inference. The novelty is not the engine but the demonstration that the full TTT recipe, adapter serving included, fits one 16 GB card, and a measurement of exactly where it stops.

3 System and Method

3.1 Model and prompting

We use the NVARC `qwen3_4b_grids15_sft139` model, a Qwen3-4B [5] with a 16-token vocabulary, the ten grid colours as digits, a newline, two role markers, and three specials (Appendix D). A grid is serialized one token per cell, and a colour value equals its token id. Because the role and special tokens cannot be produced by tokenizing text, we assemble every prompt at the token-id level as a ChatML few-shot sequence. Getting this byte-exact is the single most important correctness condition, since a one-token drift between training and inference silently destroys accuracy.

3.2 The engine is the bottleneck

Our most consequential finding concerns throughput. Decoding a 4B model is memory-bound. Each token streams the full weight matrix from VRAM, so for a weight footprint M on a card of bandwidth B the decode rate is bounded by a first-order roofline

$$r_{\max} = \frac{B}{M}. \tag{1}$$

With $B = 896$ GB/s, Eq. (1) gives 132 tok/s in bf16 ($M = 6.77$ GB), 249 in Q8_0 ($M = 3.6$ GB), and 444 in Q4_K_M ($M = 2.02$ GB). The standard Hugging Face eager path prefills healthily ($\approx$ 3,400 tok/s) but *decodes* at only $\approx$ 13 tok/s, about a tenth of the bf16 roof, because of per-token framework overhead with no CUDA graphs. The usual remedy, compiling the decode step, is unavailable on this platform. `torch.compile` needs Triton, and Triton has no native-Windows backend (Appendix C).

The fix is to change engines. We convert the model to GGUF and serve it with llama.cpp. The custom vocabulary needs two small, documented converter patches (Appendix A): the unknown tokenizer hash is mapped to the `qwen2` pre-tokenizer it actually uses, and two throwaway byte-pair merges are added to

satisfy a loader that requires a non-empty merge list (they never fire on grid text). The prebuilt CUDA 12.4 binary runs on the Blackwell card through PTX just-in-time compilation. Measured decode rises to 102 tok/s in bf16, 170 in Q8_0, and 244 in Q4_K_M (Figure 2), a 13× speedup over eager at Q8_0, which we adopt. It is near-lossless (14 of 15 training tasks reproduced exactly under greedy decoding) and has a 3.6 GB footprint. The server's default prompt cache needs one note. It reserves ≈ 680 MiB per unique prompt, and since every ARC prompt is unique it yields no reuse and destabilizes sustained generation, so we disable it and run a single supervised stream.

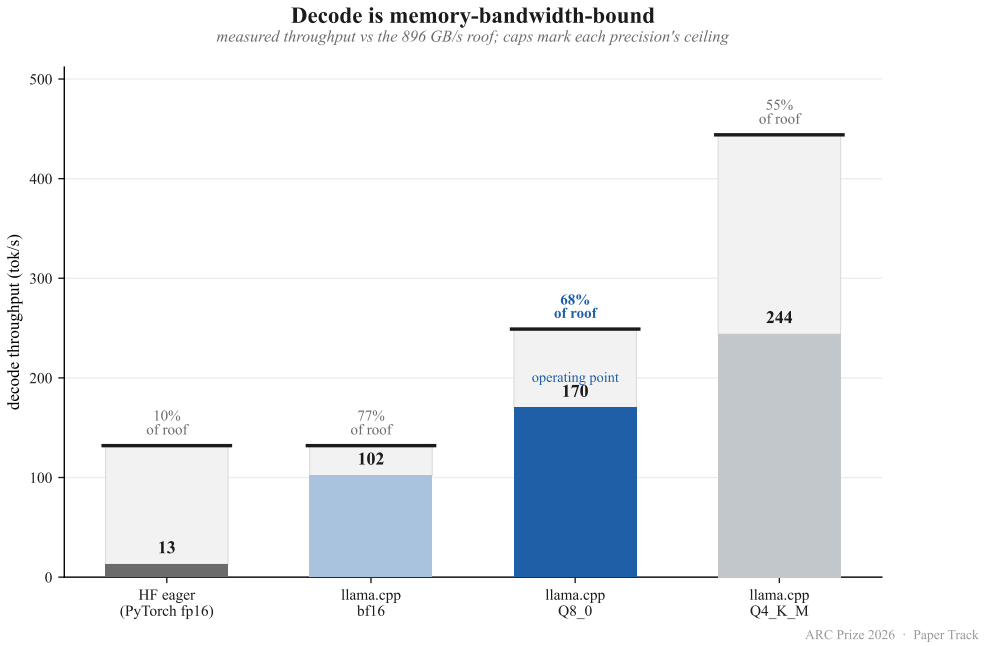


Figure 2: Decode throughput versus the memory-bandwidth roofline of Eq. (1) (the ceiling is a first-order model, bytes/token ≈ weight bytes). Bars are *measured* tok/s; the shaded gap to each cap is framework overhead, not the card. The eager path fills only ≈ 10% of its roof, and moving decode to llama.cpp at Q8_0 recovers the 13×.

3.3 AIRV: augment, infer, reverse, vote

Let f be the served model and $\mathcal{G}$ the eight dihedral (D4) symmetries of the grid. For a test input x we generate one prediction per symmetry, invert it back to the original frame, and vote. Writing n_c for a candidate's vote count and $\bar{\ell}_c$ for its mean sequence log-prob,

$$c_g = g^{-1}\big(f(g(x))\big), \qquad \hat{Y} = \operatorname*{select}_{c}\,\big(n_c,\,\bar{\ell}_c\big), \qquad n_c = \big|\{g \in \mathcal{G} : c_g = c\}\big|, \quad \bar{\ell}_c = \frac{1}{n_c}\sum_{g:\,c_g=c} \log p(c_g), \tag{2}$$

where select returns the two distinct candidates with the highest vote count, ties broken by mean log-prob, and emits them as the two attempts the metric allows. The vote count is itself a usable confidence signal. Agreement across symmetries predicts correctness (Figure 8). The whole offline system (the trainer, the converter, and the server, which never share VRAM) is sketched in Figure 3.

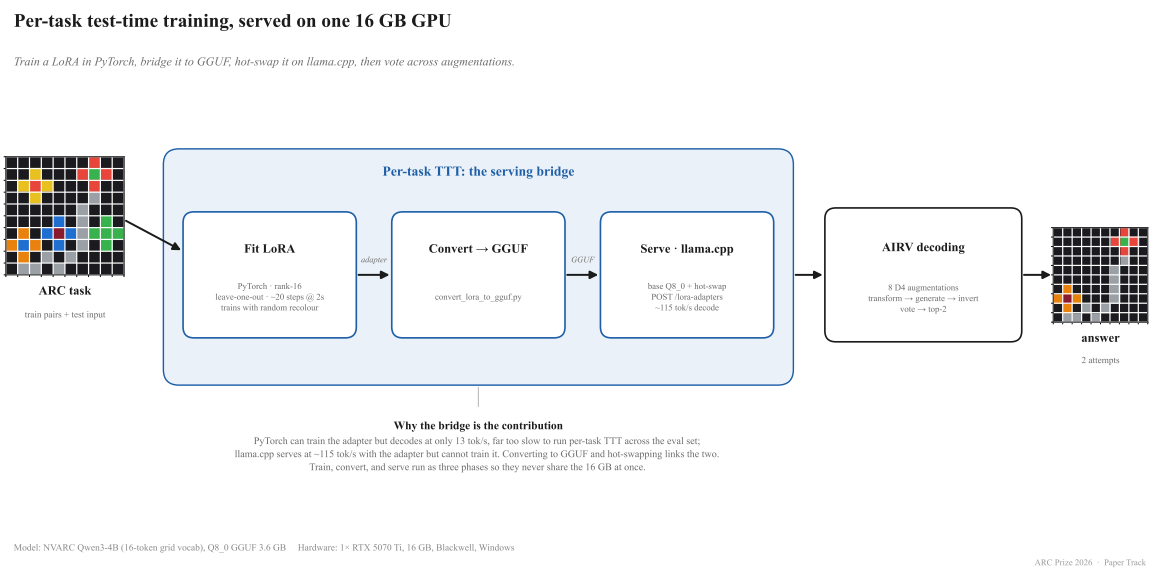


Figure 3: The offline system. The PyTorch trainer and the llama.cpp server never share VRAM; per-task adapters cross the bridge as GGUF and are hot-swapped on the fast engine per request.

3.4 Per-task test-time training

For TTT we fit a per-task LoRA adapter $\Delta W = sBA$ of rank r and scale s on the task's own train pairs T, with a leave-one-out target and the loss masked to the answer tokens:

$$\mathscr{L}(\theta) \;=\; -\sum_{(u,v)\in T}\;\sum_{t}\log p_\theta\big(v_t \mid v_{<t},\, \mathrm{ctx}_{T\setminus\{(u,v)\}},\, u\big), \tag{3}$$

drawing a fresh random augmentation each step. Capping the few-shot context to $\approx 1{,}600$ tokens drops a training step from ≈ 13 s to ≈ 2 s, so training is no longer the expensive part.

The harder problem is *serving* the trained adapter. TTT must run in PyTorch, where decoding is the slow 13 tok/s and intermittently becomes quadratic after a step. The fast engine does not train. We bridge them (Figure 3): train in PyTorch, save each adapter, convert it to a GGUF adapter, and hot-swap it on llama.cpp through its `/lora-adapters` endpoint, where decode holds ≈ 115 tok/s with no stalls. We run three phases (train all, convert all from a clean process, then serve and generate all) so the two frameworks never share VRAM (Figure 4). Large-grid tasks remain too VRAM-bound to train and are excluded by their largest-train-pair token length (Figure 4).

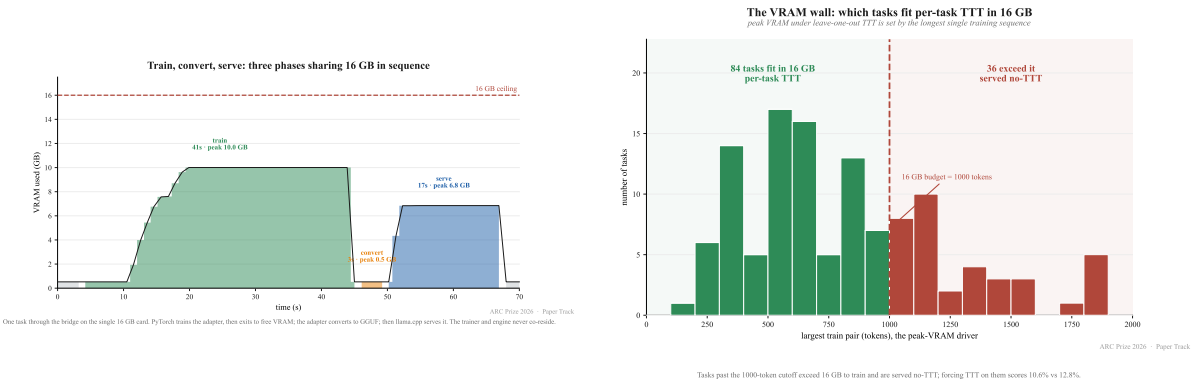


Figure 4: The left panel shows the three-phase schedule keeping PyTorch and llama.cpp from sharing the 16 GB at once, with the VRAM trace sampled live and the peaks measured. The right panel shows the VRAM wall, with each task placed by grid size and context length, and tasks above the operational cutoff excluded from TTT. In the right panel only the per-task points and the cutoff are measured. The translucent VRAM surface is a calibrated model, not a measurement, drawn only to make the wall legible, and the cutoff metric is the largest train pair.

4 Results

4.1 Two regimes, and the leaderboard progression

Two hardware regimes appear in this paper, and every number belongs to exactly one of them. Every public-eval number, the 12.79% baseline, the 17.6% trainable-task peak, and the 16.28% blend, is measured on the single RTX 5070 Ti against the 120-task public evaluation set, and all method development ran there. Every leaderboard number is measured by Kaggle's offline scoring rerun on the contest's four-L4 node, against the hidden 240-task set (120 semi-private plus 120 private). The submissions through v6.1 use the four L4s only to shard tasks for throughput, since the bf16 per-task training does not fit the 12-hour budget on one card. No single task there needs more than one 16 GB GPU, so the method under test is the single-card recipe. The v7 entries are the one exception and this paper marks them as such wherever they appear. They reproduce a public pipeline whose per-task training runs at rank 256 over an eight-thousand-token context, which fits 16 GB only under an attention kernel we identified after those runs and would still need roughly forty hours on one card against the twelve the contest allows, so those numbers measure competition hardware rather than the card this work is about. They are kept in the same table deliberately, because the distance between the best single-card entry and the best four-L4 entry is the most direct measurement we have of what the constraint costs.

That exclusion is measured rather than assumed. Sweeping LoRA rank against sequence length on the single card, with training configured exactly as our shipped path, every rank from 32 to 256 fits comfortably at a two-thousand-token sequence and every one of them fails at four thousand. The blocking term is neither the adapter nor the optimizer, which together account for under a gigabyte at rank 32 and three at rank 256. It is activation memory, which grows from roughly 2.6 GB at two thousand tokens to roughly 8.5 GB at four thousand, a factor of 3.3 for a doubling of length. That superlinear growth is attention materializing its score matrix. Leaner adapter dtypes buy a full rank doubling at equal memory,

enough to run rank 256 at two thousand tokens in 14.5 GB, but they leave the activation term untouched. On that evidence we recorded the context as a hard wall and the eight-thousand-token configuration as beyond the card.

That conclusion was wrong, and the sweep above contains the reason. It never chose an attention kernel. Repeating the measurement with the backend forced rather than left to the framework's dispatch (Table 1) moves the wall entirely. Under cuDNN attention the rank-256 configuration trains at an eight-thousand-token sequence in **14.86 GB** at 5.04 seconds per step, on the same card that had failed at four thousand. The framework's automatic selection is numerically identical to the forced math backend at both lengths, because the flash and memory-efficient kernels report no available implementation for this GPU in this build, so the dispatcher had been falling back to the quadratic path throughout. The result is corroborated from outside the measurement, since the four-L4 runs fit the same training configuration into 13.0 GB under a flash-attention kernel, and two different memory-efficient implementations on two different cards landing within two gigabytes of each other is the agreement we would expect.

Three limits on that result, stated because they bound what follows from it. It measures training memory for a single sequence at batch size one, where the deployed pipeline may pack differently. It measures memory and step time rather than numerical equivalence, which would need a paired run against the math backend on a real task. And it shows only that the training configuration fits, not that the pipeline runs end to end on one card, which would additionally require porting that pipeline off a training library with no working install on this platform.

What it does settle is which resource binds. At 5.04 seconds per step the 128-step schedule is about eleven minutes for one adapter, so the 240-task contest set would need roughly forty hours against a twelve-hour budget. The card holds the winning configuration and cannot serve it in time. That is a throughput limit rather than a capacity limit, and it is the second time in this work that a ceiling we recorded turned out to be a default rather than a property of the hardware, after the six-hundred-token training truncation of Section 5.8. Both were found by re-examining a measurement we had already filed as settled.

Table 1: Attention backend against sequence length, at rank 256 with bf16 adapters on the 16 GB card, peak reserved memory in GB. Cells marked thrash spilled into host memory rather than raising, running at 33 to 55 seconds per step against roughly 2 seconds when resident. Blank cells were not attempted once the shorter sequence had failed.

attention backend	2048	4096	8192
eager	13.44	19.33 (thrash)	
sdpa, framework default	13.94	18.62 (thrash)	
math, forced	13.94	18.62 (thrash)	
memory-efficient, forced	no kernel available for this GPU		
flash, forced	no kernel available for this GPU		
cuDNN, forced	**12.31**	**12.80**	**14.86**

The AIRV pipeline without TTT scores **12.79%** (22 of 172 test outputs) on the public evaluation, a clean single-stream run completing in $\approx$ 78 minutes, and **6.53%** on the hidden set, a halving we return to in

Sections 5.1 and 5.3. Adding per-task TTT roughly doubles the hidden score. Table 2 gives the full progression.

Table 2: Leaderboard submissions, scored on the hidden 240-task set on the four-L4 node. Entries through v6.1 are the single-card recipe sharded for throughput. The v7 entries reproduce a public pipeline whose per-task training cannot be served in budget on one 16 GB card and are not single-card results.

version	configuration	hidd
v1	no-TTT AIRV, eight greedy D4 views	
v2	+ per-task TTT (160 tasks) + colour-perm	
v2.1	scaled TTT (232 tasks), single scale, no colour-perm	
v3	+ three-scale vote	
v3.1	+ two-scale vote + colour-perm	
v4	+ more-candidates sampling (four per view)	
v5	+ white-box plausibility filter	
v6	deep-regularized TTT (60 steps, rank 32), disk-exhausted	
v6.1	deep-regularized TTT, all 232 tasks trained	
v7	sorokin reproduction as ported (Section 5.6)	
v7.1	same pipeline with both port defects repaired	
v7.2	identical build, an accidental repeat run	
v7.3	+ loss-gated skip and greedy fill of empty slots	
v7.4	cap wired to 1200, intended as a control, confounded by 24 unbudgeted fills	
v7.5a	+ symbolic object-count prune, the theorem's program-execution channel (Appendix F)	
controls and replicates		
ctrl	the notebook we first blamed for 33.89, forked unmodified	
ctrl2	the notebook actually titled for 33.89, forked unmodified with its author's own attachments	
v7.2r	v7.2 re-run unchanged, the same-configuration replicate	
v7.2r′	*the same kernel version as v7.2r, resubmitted*	
v7.2r″	*the same kernel version again, a third execution*	
v7.2r‴	*and a fourth*	
v7.2r⁗	*and a fifth*	
cap	the replicate with one live literal, 1800 to 1200	
noAIF	the replicate minus computation its own output discards; 30.00, 29.03, 30.56	2
v7.2r×12	the same kernel version, twelve executions in all (Table 3)	2
v7.6	the replicate plus the greedy fallback alone; 30.56, 29.72, 29.44	2
v7.7	the replicate with the DFS cache cropped to the branch; 29.72, 27.36, 29.03	2
v7.7b	the replicate with each task's remaining time divided evenly over its view batches; 29.44, 27.78 so far	2
rebuild	a fresh build of the unmodified v7.2 configuration, a control on the build script	

The scaled single-scale configuration (v2.1) reached **13.61%**, ranking 468 of about 4000, and four candidate-side submissions failed to beat it. One errored in the scored rerun, and each of the three that scored carried exactly one lever that had lifted or tied the public evaluation, multi-scale voting with colour

permutation, more-candidates sampling, and the white-box plausibility filter, and each regressed the hidden score (Section 4.4 gives the levers and the mechanism). The eight greedy D4 views at a single adapter scale are a local optimum that the candidate and voting side does not improve, and the repeated public-to-hidden reversal is itself a finding (Section 5.1). The break came from the other side of the pipeline. A deep-regularized adapter that trains longer at a higher rank (60 steps, rank 32, Table 15), moving the per-task training rather than the vote, lifted the hidden score to **18.19%** in v6.1, a 4.58-point gain and the first configuration to beat v2.1. Its intermediate form v6 scored 12.36% as the same recipe crippled by a disk-exhaustion bug that left 73 of 232 tasks with no adapter, and fixing the scratch path recovered the full lift. This is the paper's thesis made concrete, the candidate and voting side is saturated and the per-task training side is where the hidden number moves. For reference the 2025 winner reached 24.0% with TTT [3] on the same Qwen3-4B base lineage with a more developed recipe.

The two v7 entries then measure the constraint itself. Reproducing the public pipeline of Section 5.6 on the four-L4 node scored 16.81% as the community port shipped it, and **30.56%** once two defects introduced by that port were repaired, and **30.97%** when the same repaired pipeline ran at the upstream per-task cap. The last of those is the highest this project has posted and it is not a single-card result, since its per-task training cannot be served on one card inside the contest's twelve hours. Read against v6.1, the best configuration we ran within that budget, the pair gives the cleanest figure in this work. The same author, the same base model, the same hidden set and the same twelve-hour budget yield **18.19%** under 16 GB and **30.97%** on the contest's 96 GB, so roughly twelve points separate what one consumer card reaches from what the provided hardware reaches on a method built for it. That gap is the honest size of the ceiling in the title, measured rather than argued, and it is the paper's central quantitative claim.

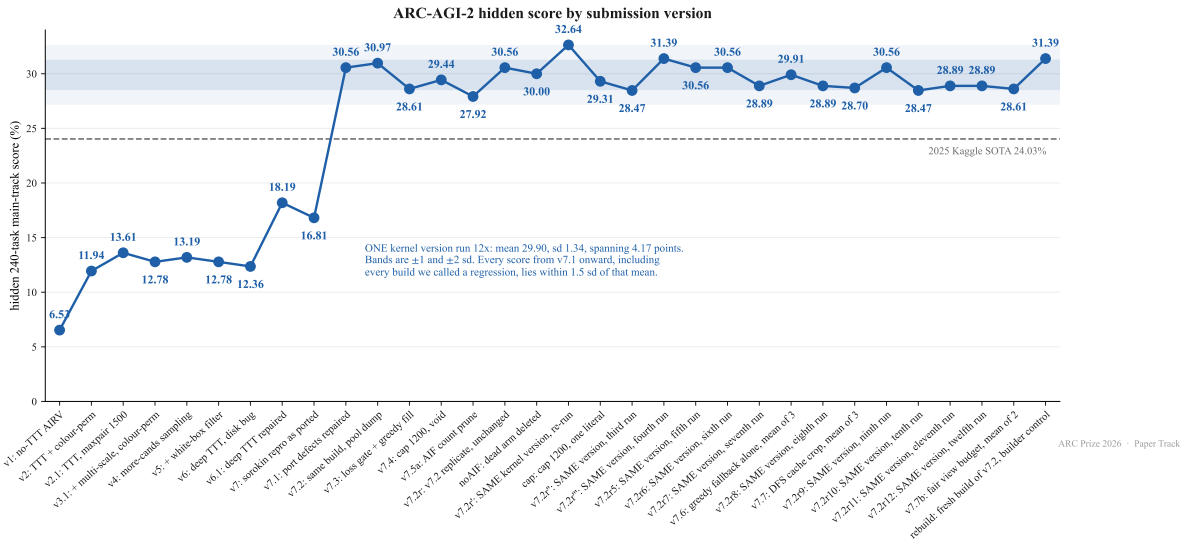


Figure 5: Hidden score by submission version, with the measured same-configuration band. The band comes from twelve runs of *one immutable kernel version*, mean 29.90 and standard deviation 1.34; the inner shading is ±1 sd and the outer ±2. Every score from v7.1 onward, including every build we called a regression, lies within 1.5 standard deviations of that mean. An earlier version of this caption read that the spread was "about one task credit" and that every point below the band was "a real regression rather than noise"; that was estimated from three runs and both halves of it were wrong. The two public-notebook controls are plotted separately in Figure 6.

The replicate, and the error bar every comparison had been assuming. Every comparison above silently assumed the hidden score of a fixed configuration is stable, and nothing in the project had ever measured it. After the fifth consecutive build scored below v7.2, with the scores split into two clusters and nothing between them, the cheapest explanation left was that 30.97 had been a favourable draw and the regressions were noise, so we spent a submission on the only experiment that answers it, v7.2 resubmitted byte-identical except for the size of its own commit run. It scored **30.56**. On three runs that looked like a range of 0.41, one task credit, and a standard deviation of 0.24, and we wrote that a lever must move roughly 0.4 points to be distinguishable from a re-run. Two further runs destroyed that estimate, and the second destroyed it completely.

The fourth run deleted the solver's dead symbolic-arm computation, work whose output the scoring v7.2 provably discards on the next line, a change with no path to the computed answers except timing. It scored **30.00**. The fifth was the decisive one. We resubmitted the replicate kernel, and resubmitting a Kaggle kernel version re-runs an immutable artifact: not a rebuild, not a re-export, the same committed version identifier as the run that had scored 30.56. It scored **32.64**. Identical code, two executions, **2.08 points apart**, which is five task credits.

We then kept submitting that same kernel version, as often as the quota allowed, into late August. Twelve executions of one immutable artifact scored **30.56, 32.64, 28.47, 31.39, 30.56, 30.56, 28.89, 28.89, 30.56, 28.47, 28.89** and **28.89**: a range of **4.17 points**, ten task credits, on code that cannot have changed. This is the central measurement of the paper and we arrived at it by widening the same estimate five times, each time in the same direction.

Mean **29.90**, standard deviation **1.34**, with a 95% interval on that standard deviation of $[0.95, 2.27]$ and on the mean of $[29.05, 30.75]$.

Two methodological points about that series, both of which cost us something to learn. First, it is twelve runs of one *script version*, verified by the `scriptVersionId` recorded against each submission rather than by what its description said. One member carried the note "unchanged except commit size", which could easily have meant a rebuild and a different artifact; the audit confirmed it did not. Second, an earlier draft of this section pooled these runs with three one-factor variants into an eight-member "same-method family" at mean 30.64 and sd 1.18. That family is the wrong instrument, because its spread confounds the code change with the day. Only the single-version series measures the environment alone, and it is the wider of the two.

The obvious objection, and a second artifact that answers it. Twelve runs of one kernel version measure that kernel version. A reader is entitled to ask whether we happened to hammer an unusually erratic artifact, and no amount of resubmitting the same one can answer that. So we spent three submissions on a *different* build of the same configuration, `scriptVersionId` 337769272, which had scored 30.97 on its single previous run. It scored **30.56** and then **28.47**. A third artifact came at the end of August and for a different reason. The build script had by then been edited repeatedly since the replicate kernel was pushed, and two new arms had landed below the mean, so before reading them we submitted a fresh build of the unmodified configuration as a control on the builder itself. It scored **31.39**, the second-highest score of the season, which rules out the build script as the explanation for anything below the mean.

Table 3: The same configuration, three independently built artifacts, sixteen runs. The two spreads that can be compared are indistinguishable, which is what licenses treating the standard deviation as a property of the pipeline rather than of one kernel.

script version	runs	mean	sd	range
339772104, the replicate	12	29.90	1.34	4.17
337769272, the v7.2 build	3	30.00	1.34	2.50
346281718, a fresh build of the same configuration	1	31.39	–	–
pooled	16	30.01	**1.30**	4.17

The variance ratio between the two multi-run artifacts is $F = 1.01$, which at these sample sizes is as close to identical as the data can express. Two separately built copies of one configuration reproduce the same run-to-run spread to two significant figures. The 95% interval on the pooled standard deviation is $[0.96, 2.01]$.

That also disposes of an intermediate reading we briefly held. After two runs the second artifact had scored 30.97 and 30.56, a gap of 0.41, and it was tempting to conclude the second build was quieter. It was not: two draws from a standard deviation of 1.34 differ by more than 0.41 about 83% of the time, and the third draw came in at 28.47. A pair of runs cannot measure a spread, which is the same lesson the three-run estimate taught earlier in this section, arriving a second time by a different route.

The consequence is not a correction to one claim, it is the retirement of a whole class of claim. Measured against this series, *every* deviation this project has recorded falls inside *two* standard deviations, and after the twelfth draw none of them reaches even 1.5:

Table 4: Every build we called a regression, priced against twelve runs of one immutable kernel version (mean 29.90, sd 1.34). The rightmost column is what we previously reported, using a standard deviation estimated from three runs, then five, then six, then eight pooled members. *Nothing* now reaches 1.5 standard deviations. The last row is not a regression at all: it is the configuration we called our best, and it sits four fifths of a standard deviation above the mean of the code it is supposed to beat.

build	score	from mean	in sd	previously reported as
v7.5a	27.92	-1.98	1.48	real at 11, then 3.0, 1.93, 2.14 sd
ctrl (aimo)	28.06	-1.84	1.38	below our band
LB33 control	28.47	-1.43	1.07	real, 8.8 then 2.4 sd
v7.3	28.61	-1.29	0.96	real, 8.2 then 2.3 sd
cap 1200	29.31	-0.59	0.44	prediction unsupported
v7.4	29.44	-0.46	0.34	real, 4.7 then 1.5 sd
no-dead-AIF	30.00	$+0.10$	0.08	already retracted
v7.2, “our best”	30.97	$+1.07$	0.80	the configuration everything was compared to

What the noise costs a research programme. The standard deviation is not only a caveat on past claims, it is a budget constraint on future ones. Separating two configurations at 80% power with a two-sample comparison at this standard deviation needs about **4.1 points of separation with three runs in each arm**, by an exact two-sample t computation; the normal approximation an earlier draft used gave 3.3 and understated it, because with two degrees of freedom per arm the standard deviation is itself poorly known. The contest allows one submission a day, so a single properly powered comparison costs six submission days. Nothing in our ledger of forty measured levers is remotely four points; the largest coverage change any of them produced on a held-out slice is one output in fourteen. This reframes the ledger. It reads as a wall of null results, and the honest reading is not that nothing helps but that *the leaderboard cannot see anything smaller than a third of our own score*, so a programme of single-submission A/B tests, which is what the field and this project both ran, was never capable of the discrimination it claimed.

What the variance is made of, from the one per-run quantity Kaggle exposes. The submission record carries the byte size of each run's `submission.json`, and that turns out to be a usable instrument. Across the fifteen same-configuration draws of the two multi-run artifacts, size and score correlate at $r = +0.55$, and the relationship holds independently inside each artifact ($+0.53$ on twelve runs, $+0.80$ on three). The size spread is 25.5 kB on a 778 kB file, which at roughly 2.8 kB per output is about **nine outputs' worth of content** — and nine outputs is 3.75 points against an observed range of 4.17.

The mechanism that fits is not exotic. A task whose pool comes back empty is written out as the `[[0]]` fallback, a 1×1 grid, which is both tiny and certainly wrong. So a slower session produces more empty pools, a smaller file, and a lower score, all at once. The variance is substantially *throughput*, and the entire observed range is accounted for by fewer than a dozen tasks failing to produce any answer at all.

This refines rather than contradicts the earlier refutation of tail-dropping. That simulation asked whether the global twelve-hour wall truncates the task queue and found it finishes with 14% slack, which remains true. The binding deadlines are the per-task 1800 seconds and the 540 seconds inside a single DFS call, and those fire task by task on a slow session without the global queue ever running out of room.

It also revises a hypothesis we held earlier in this section. The observed standard deviation is consistent with about 41 outputs behaving as coin flips, which matches the independently measured 12.5% same-seed pool disagreement almost exactly, and that remains a real effect. But the size correlation says a substantial share of the variance is the coarser thing: not *which* answer a task produced, but *whether it produced one*. The two are not exclusive and we cannot yet apportion them, since Kaggle exposes no per-submission log through its API.

The practical consequence points the opposite way from the obvious one. Making the search *cheaper* rather than more thorough should raise the mean and cut the variance together, by converting empty pools into answered tasks on slow sessions. Our own offline floor sweep found no gain from a tighter threshold at a fixed decode budget, but that measured a rig at 600 seconds per task and not a contest worker at 1800 with a variable machine underneath it.

The uncomfortable direction of that argument. It is easy to read the preceding as a defence: our regressions were not real, so we did not break anything. The symmetric reading is the one that costs us

something. If a single submission cannot resolve a two-point effect, then a lever that genuinely *worked* by one or two points would have produced a score indistinguishable from a bad draw, and we would have killed it. We must therefore allow that some of what this project discarded was working, and we cannot say which. Five builds were rejected on a hidden score alone — v7.3, v7.4, v7.5a, the 1200-second cap, and the no-dead-AIF variant — and every one of them is a candidate for that error. We are not claiming any of them worked; we are recording that our procedure could not have told us.

Two qualifications keep this from becoming a general licence to doubt everything. First, the measurement is taken at the 30-point operating point, and the per-task binomial component of the variance is smaller at 13 points, so the early v2-to-v5 comparisons are not automatically covered by this number and would need their own. Second, and more importantly, **most of the ledger was never judged on a hidden score at all.** Thirty-two of the forty levers were closed by paired offline measurement, where both arms see the same tasks under the same budget on the same afternoon, task difficulty cancels in the pairing, and the twelve-hour contest wall clock — the mechanism that produces this variance — is not in the loop. Those closures have a different weakness, which is small samples and sign tests at $p = 0.25$ to 0.50, and it is not this one. Conflating the two would let a real result about submission noise excuse a separate and equally real problem with statistical power offline.

The forward-looking cost is arithmetic. Re-testing one discarded lever properly, at three runs per arm, is six submission days. A season has of order ninety. So the entire remaining budget of a compute-poor entry, spent on nothing else, adjudicates about fifteen hypotheses — and only those with effects above four points, which is larger than anything this project has produced.

Two of those lines are worth stating plainly rather than leaving in a table. **The LB33 control scored 28.47, and so did our own unmodified pipeline.** The phantom-notebook section (Section 4.1.1) concluded that two public artifacts "both land near 28 when executed, below our own unchanged pipeline's band". Our own unchanged pipeline reaches exactly that number. The conclusion that survives is the one that never depended on the gap, that a displayed score does not bind to the published file; the comparison we drew underneath it does not survive. And **v7.5a, the symbolic prune we treated as the theorem's escape route measured and refuted, has now been reported at 11, 3.0, 1.93, 2.14 and 2.30 standard deviations as the sample grew from three runs to eight, without its own score ever changing.** Its verdict crossed the two-sigma line in both directions on single additional draws, which is a sharper argument against single-comparison verdicts than anything we could write. The interval on the standard deviation itself is $[0.78, 2.41]$, so v7.5a's distance from the mean is somewhere between about one and three sigma and the threshold is not a stable object. We keep the conclusion about oracle accuracy because the offline invariant-mining experiment reaches it independently and does not rely on the hidden score, and we decline to reinstate the hidden-score argument on a tenth of a sigma.

This also settles the mechanism question the earlier draft went back and forth on. We wrote that the wall-clock account failed because three baseline runs did not spread. They spread as soon as the sample grew, and they spread in the comparison where nothing but session speed could differ. The decode is deadline-bounded in three places, so how far the machine gets decides where the search truncates and therefore which candidates ever exist. The throughput account is reinstated and the alternative explanations of the cluster structure are unnecessary.

We report all of this rather than quietly restating the family because it is the error this paper spends its discussion section warning other people about, and we committed it three times. The winners' own writeup remarks in passing that they "saw 1-2 points variability in the scores across different re-runs of the same notebook" [4], and their batched decoder is nondeterministic by construction, logits changing with batch composition. The variability was known inside the strongest team and, as far as we can find, never measured or accounted for by anyone. Every ablation on this leaderboard, including theirs, is read against noise nobody had quantified.

What the pair does not do is attribute that gap. The two runs differ in their hardware and in their method at the same time, so twelve points is the size of the distance rather than an account of where it comes from. Isolating the method term would mean running the reduced configuration on the four-L4 node rather than on the single card, holding the platform fixed and moving only the training context, and we flag that as the experiment this measurement wants next. Running it on our own hardware instead would confound the answer with the card, the driver and the attention kernel all at once, which is the fault the sweep in Section 4 corrected in itself.

The more useful statement is about *where* the constraint acts, and by the end of this work we can make it precisely rather than as a point count. It does not act at inference. The offline track fixes that for everyone at four L4 GPUs and twelve hours, our submissions run inside that budget, and the cap is in fact why the field runs a 4B model at all. It acts at pretraining, which is unconstrained, and the resource is nameable from the winning entry's own published configuration: a full-parameter supervised fine-tune under tensor-parallel training across eight GPUs, over a corpus whose synthetic component is of order 10^6 puzzles produced by an LLM pipeline that summarizes and recombines task concepts, writes transformation programs and executes them. We estimate that generation alone at six to eight months of continuous single-card inference, before any training step. So the ceiling in our title is a training ceiling wearing an inference costume, and the twelve points measure the distance between a base one card can fine-tune and a base one card cannot build.

Three consequences follow, and each is a measurement rather than an inference. The published artifacts do not close the gap: our base *is* that model, and our own earlier continued fine-tune regressed because two of the datasets we added were already inside its training corpus (Section 5.3). Nor can the gap be closed by generating replacement data, because the tasks this model misses are not separable from the ones it reaches on any structural axis we can measure, which is the only kind of property a generator can be instructed to produce (Section 5.9). And it cannot be closed from the inference side, since selection is bounded by a dominance argument on the candidate pool (Section 5.7) and the scheduling lever turned out to be numerical noise once a same-seed control was run. Four independent levers, four independent closures, all pointing at the same unpurchasable resource. That is a sharper claim than twelve points, and unlike twelve points it is falsifiable: a stronger base model reached on comparable hardware would refute it.

A third submission of the reproduction, identical to the second except that it leaves the per-task search cap at the upstream default of 1800 seconds rather than the 1200 we had chosen for margin, scored **30.97%** and placed 264th. Two facts about that number matter more than its size.

Roughly sixty entries on the leaderboard carry it exactly. A tie that wide is not coincidence, it is the public

notebook being forked and submitted unchanged, which identifies 30.97 as that notebook's number rather than as anyone's result. Our restoration therefore converges on the public baseline exactly rather than approximately, and that is stronger evidence that the two port defects we isolated account for the whole difference than the near-agreement we claimed before. It is also, plainly, a reproduction rather than a differentiating result, and we report it as one.

We first read the 0.41 points between our two runs of it as the per-task cap, since the two builds differed by nothing else we had declared. That reading was wrong, and its correction is the most useful thing in this section. An audit of the generated notebooks found the cap knob had never reached the solver at all. The build script rewrote a guard it expected to find as a local variable comparison, the source writes the same test inline, the pattern never matched, and so every submission from the first reproduction onward ran the same hardcoded 1800-second cap while its configuration banner printed whichever value we had set. The two runs were therefore the identical build at the identical cap, and their 0.41-point spread is run-to-run variance. We took that 0.41 for the noise floor, and it was only the smallest gap the sample happened to contain. Resubmitting one immutable kernel version later produced 30.56, 32.64, 28.47 and 31.39 on the same artifact, so the floor is about ten task credits and not one, and every comparison in this work should be read against the wider figure derived above rather than this one. Appendix F derives the resolution limit such a number instantiates. One further observation follows for anyone reading the leaderboard rather than the method. A single task credit separates an entire sixty-entry cluster, so on this board the cheapest available gain is not a better model but a better use of the twelve hours.

The measurement run's own log then priced that claim. Of its 24821 task-seconds, 71 percent went to tasks that yielded nothing, no solve and no correct grid anywhere in the candidate pool, and the two longest tasks each burned to within sight of the 1800-second cap for nothing. One task in the forty trained normally and then decoded no output at all, a forfeited slot that scales to about four on the public 120 and plausibly eight on the hidden 240. Final training loss predicts these dead tasks weakly but usably at the extreme, cutting every task whose adapter finished above a loss of 0.05 reclaims 2330 seconds while touching no task the selector harvests, since the one such task with a correct grid in its pool was being lost at selection anyway. Three scheduler changes follow, none touching the method. A task whose search returns nothing falls back to a plain greedy decode with its already-trained adapter, which can only fill otherwise-empty slots. A task whose training loss ends above the cut serves the cheap greedy answer and skips the expensive search. And the time both changes reclaim becomes margin for the per-task cap, the one knob already measured to buy a task credit. The next submission carries these as a scheduling variant of the unchanged pipeline.

4.1.1 A number that did not reproduce, and a week spent explaining it

The public board then handed us what looked like a controlled measurement and was not one. We report the episode in full, including three wrong explanations we published internally along the way, because the correction cost a week and the general lesson is worth more than the specific result.

A public notebook restoring this same pipeline displayed **33.89** against our restoration's 30.56. Taking that at face value, we spent the following week attributing the 3.33-point deficit. First to the dependency tree, since its kernel metadata mounts the original author's wheel set and model upload where ours mounts a

rebuild whose pins predate the current execution image, and our logs had warned throughout that compiled extensions were being skipped for a version mismatch. That reading priced the software environment at 3.33 points and reframed the sixty-entry cluster at 30.97 as a population unknowingly running a degraded one. Rebasing onto that environment scored **28.61**. Second to the per-task cap, after an audit found the cap knob had never reached the solver, on the argument that a looser cap lets slow tasks eat the wall until the queue tail is dropped unstarted. That rested on the belief that the public build ran a tighter 1200-second cap; it forks a source that hardcodes 1800, as did every submission of ours through the wiring fault, so the two sides had agreed on the cap all along. The build sent to test it scored **29.44** and moved a second factor besides. Third to two changes of our own in the solver, deleting the port's symbolic arm and regrouping its two-view search batches into fours, which a diff read module by module leaves as the only differences that survive into deployment.

Then we forked the public notebook unmodified and submitted it from our own account. It scored **28.06**, 2.91 points below the restoration it was supposed to be beating. We took that as showing the deficit had never existed, and reported it that way. That reading was itself wrong, and the way it was wrong is the most useful thing in this section.

The notebook we forked was not the notebook the score belonged to. Enumerating the competition's public notebooks, which are published precisely to be read, turns up one titled for the score in question and carrying it in its own name, along with a byte-identical fork of it. That notebook runs the same base model and the same dependency tree as ours, and differs from our best submission in exactly two respects, both of which govern how much search fits inside the budget. It computes the decoder's per-node score on the GPU over the twelve grid tokens rather than as a full-vocabulary softmax moved to the CPU, which is algebraically the same quantity and so cannot change what an unbounded search finds, but the search is bounded and the cheaper form covers more of the tree per second. And it caps each task at 1200 seconds rather than 1800. Every other difference we had agonized over is absent: it removes the symbolic arm and uses the four-view batching, both of which we had arrived at independently.

So the correct account is neither the one we started with nor the one we replaced it with. The number was real. A public artifact producing it was available for the whole period. We measured a different artifact, concluded the effect was illusory, and wrote that up with more confidence than testing one file supports. The 28.06 measurement stands and is worth keeping, since that notebook demonstrably does not produce the number attached to it in our reading of the board; what does not stand is the inference from one such test to the claim that nothing produces it.

The exact size stays uncertain and we decline to sharpen it beyond what the design supports. The control carries the public author's own attachments, so against our 30.97 it moves both the code and the environment and reads $+2.91$; against our 28.61, which shares its environment and cap, it reads $+0.55$ but that build carried two scheduler additions of its own. Both comparisons favour the restoration and neither isolates it. One reading does invert cleanly: our own environment now looks better than the one we had priced at $+3.33$, since every build we ran on the public author's mounts scored below every build we ran on ours. We state that as a direction, not a number, having just been taught what happens when a single uncontrolled comparison is promoted to a price.

The identified artifact was then run as its own control, forked byte-identical with only the ownership

metadata changed and mounting its author's original wheel tree and model upload exactly as its metadata specifies. It scored **28.47**. That is the second public notebook displaying a 33.89-class number to land near 28 when actually executed from a clean account, and it converts the episode from an anecdote about one notebook into a pattern about the board.

One sentence of what follows has since been withdrawn, and we leave the reasoning visible rather than editing it out. We wrote that both controls landed "below our own unchanged pipeline's band", and used that gap to refute the environment hypothesis outright. Our own unchanged pipeline subsequently scored **28.47** on a later execution of an immutable kernel version (Section 4), which is the control's number exactly. So the gap does not exist and no comparison of single runs at this separation can be made. What survives is the part that never depended on it: a displayed score binds to a submission and not to the published file, the two are mutable independently, and forking the artifact unmodified is the only way to learn what it does. The environment hypothesis is therefore *unsupported* rather than refuted, which is where it stood before we over-read the control, and the per-task cap's motivation was independently removed by the clean one-factor test of Appendix F. A rig benchmark of the remaining mechanism, the GPU per-node arithmetic, measured 1.33× the candidates of the CPU form at a fixed budget with an identical truth-in-pool count of 1 of 7, so the stated search-throughput mechanism is at best weakly present on our hardware and bought nothing that scores.

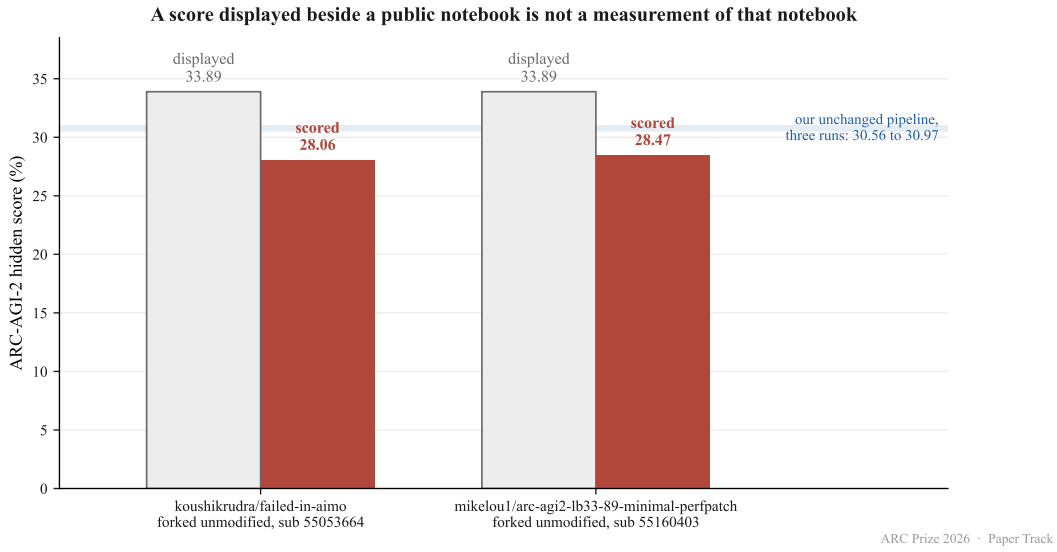


Figure 6: Displayed against measured. Two public notebooks carrying 33.89-class numbers score 28.06 and 28.47 when forked unmodified and submitted from our account. The band drawn for our own unchanged pipeline is from the three runs available when this figure was made; twelve runs of that one kernel version now span 28.47 to 32.64 and contain both controls, so the figure supports the claim that a displayed score does not bind to the published file, and no longer supports any comparison between the controls and our own.

The durable lesson survives both errors and is sharper for them. A score displayed beside a public notebook is not a measurement of the code in front of you, and it does not even identify which code it came from. A submission binds to a notebook version and its attachments at submit time, both mutable

afterwards and neither auditable, and a leaderboard entry names a team rather than a file. We treated a number as though it pointed at an artifact, first believing it and then disbelieving it, when what it actually required was finding the artifact. The remedy is not scepticism, which produced our second error, but enumeration: the competition publishes its notebooks, and reading them is both permitted and the only way to know what a number refers to. We did that a week late, and it took an afternoon.

The costs are worth stating plainly, since they are the argument for the discipline rather than decoration on it. Believing the number bought three quantified explanations of a gap, one of them elevated to a claim that the software environment outweighed every method lever we had measured. Disbelieving it on one control bought a confident published assertion that the effect was illusory. Enumerating the notebooks bought a two-line mechanism, testable offline, that neither belief would ever have produced. Roughly sixty leaderboard entries share our 30.97 exactly, which is what forking public notebooks produces; the same practice means the artifact behind almost any number is sitting in public, and the reason to look is that it is cheaper than any amount of reasoning about the number.

The methodological cost is measurable in this document. Three published explanations, two of them quantified to two decimal places, one of them elevated to a central claim about software environments being worth more than method, all of them explaining a gap that a single control retired in one submission. The control was cheap and available on day one. We ran it seventh. Each explanation was individually reasonable and collectively they formed a research programme aimed at a phantom, which is the failure mode this section exists to record: we never tested whether the thing we were explaining was real. Section 5.1 takes it up as the project's sharpest instance of a recurring habit.

Three builds are staged to price this, each moving one factor at the 1800-second cap both sides ran. One restores the object-count signal alone and keeps our batching, one reverts the batching alone, and one runs both factors upstream and so tests our build process rather than the method. Section 5.3 records that none of the three has returned at the time of writing, and the prediction registered before they run is that the symbolic signal carries most of the gap. Appendix F gives the allocation model that the withdrawn cap argument rested on, which remains correct as mathematics and is now simply attached to no observed effect.

4.2 The engine speedup

The $13\times$ decode improvement from changing engines (Section 3.2, Figure 2) is the single most important result for this hardware class. It turned an overnight, frequently-failing process into a sub-90-minute evaluation. On one consumer GPU, the inference engine dominated every other knob we touched (Figure 7).

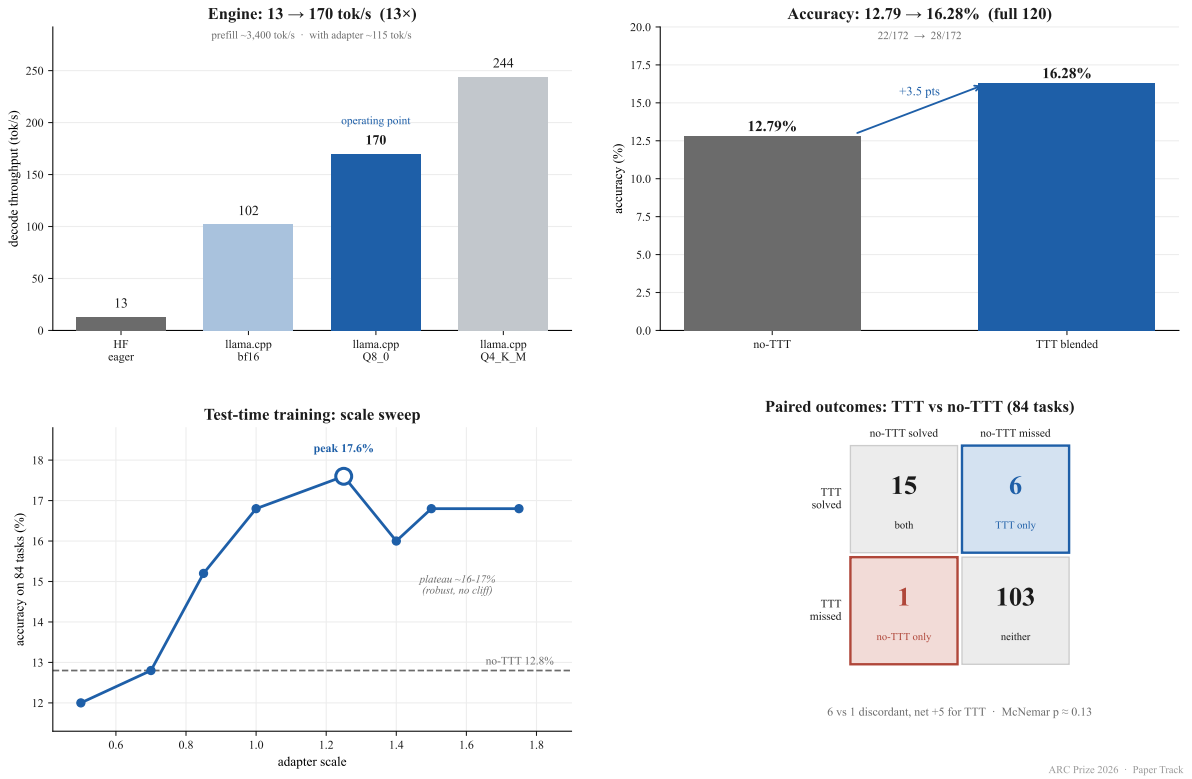


Figure 7: Engine choice, accuracy, and the adapter-scale curve. Accuracy peaks at scale 1.25 on the 84 trainable tasks, then declines, a real optimum rather than a monotone gain. All values are measured.

4.3 Test-time training

With generation fixed, we first swept the adapter configuration on 15 small-context tasks, scoring each against no-TTT on the same tasks (Table 5). The standard configuration overfits (its training loss reaches 0.000) and ties the baseline. The regularized configuration recovers a one-task lift. An over-aggressive reduction undertrains and ties again. That is coherent with an overfit diagnosis, though one task on 15 is only a hint.

Table 5: Adapter-configuration sweep on 15 small-context tasks (23 test outputs).

configuration	TTT	no-TTT
40 steps, rank 32, no dropout	3/23	3/23
20 steps, rank 16, dropout 0.1	**4/23**	3/23
8 steps, rank 8, dropout 0.05	3/23	3/23

We then validated the regularized configuration at scale, on the 84 evaluation tasks small enough to train on a 16 GB card, against no-TTT on the same tasks (Table 6). The lift holds and strengthens, adding $+5$ test outputs for a four-point gain ($\approx 31\%$ relative). The paired breakdown is clean. Of the tasks the

two methods disagree on, TTT is right on six that no-TTT misses and wrong on only one. With seven disagreements this is not individually significant (exact McNemar $p \approx 0.13$, Appendix B), but three measurements now agree in direction (sweep $+1$, scale $+5$, six-to-one discordant), and no-TTT scores 12.8% here against its 12.79% full baseline, so the subset is representative.

Table 6: Regularized TTT vs. no-TTT on the 84 trainable tasks (125 test outputs), scale 1.0.

method	correct	accuracy
regularized TTT	**21/125**	**16.8%**
no-TTT	16/125	12.8%

4.4 Ablations on scale, colour, candidates, filtering, and the big-grid wall

Five further levers, each measured on the 84 trainable tasks or the full evaluation (full per-scale numbers in Appendix B). The adapter *scale* matters. Accuracy peaks at 17.6% at scale 1.25, then declines (Figures 7, 8). Colour permutation lifts the no-TTT baseline to 14.4% but does *not* stack with the already colour-robust adapter. Forcing TTT onto the big-grid tasks does not help. We recorded the VRAM exclusion as a genuine ceiling rather than a missed opportunity, a conclusion Section 5.8 re-examines, since that early run trained under a 600-token context cap that admits a single demonstration on these tasks. The fourth lever is how many candidates the vote sees, and it cuts both ways. Sampling each of the eight D4 views four times raises the no-TTT score from 12.79% to 13.95% on the full evaluation, but like colour permutation it does not stack with the per-task adapter. Carried into a TTT submission those extra sampled candidates regressed the hidden score from 13.61% to 13.19%, and stacking multi-scale voting and colour permutation regressed it further to 12.78%. The adapter already concentrates the model on its single best answer, so more candidates help only the unsure no-TTT base and dilute the confident TTT one.

The fifth lever is a white-box plausibility filter adapted from a small-model recipe [7], which drops candidates that violate the task's colour palette or the train pairs' size pattern before the vote. On the public evaluation it is exactly neutral. The eight greedy D4 candidates are already plausible, so filtering changes no attempt on any of the 120 tasks, and where its inferred size rule misfires, on tasks whose train pairs coincidentally share one output size while the true rule varies, it empties the candidate list and the pipeline falls back to the unfiltered vote. Submitted as v5, it regressed the hidden score from 13.61% to 12.78%, a roughly two-task loss consistent with that size-rule tail landing on hidden tasks where the filter kept a plausible wrong candidate instead of emptying, though run-to-run TTT variance cannot be excluded from outside. The asymmetry is the lesson. A lever that measures neutral on the public set has no upside and only tail risk on the hidden one.

The eight greedy D4 views at a single scale are a local optimum that neither more votes nor vote filtering beats. The best blend, TTT at scale 1.25 where trainable and no-TTT elsewhere, reaches **16.28%** on the full 120-task evaluation, up from 12.79%. The honest summary is that naive per-task TTT *ties* a strong no-TTT AIRV baseline, while a regularized, scale-tuned TTT *beats* it by a consistent if modest margin.

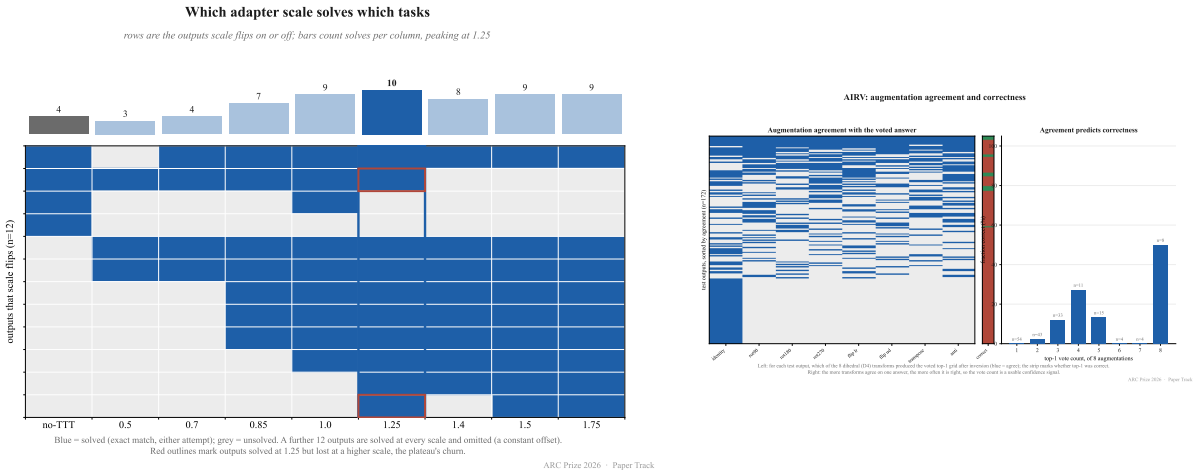


Figure 8: The left panel shows which tasks TTT solves at each adapter scale, with most gains concentrated at scale 1.25. The right panel shows AIRV vote agreement (Eq. (2)) predicting correctness, where full eight-way consensus is right far more often than split votes.

4.5 The search-throughput frontier, and a correct decoder that scores worse

Three experiments run in consecutive weeks move the decoder's budget in different directions: more search, less search, and the same search divided differently among the views. None helps, and the set is more informative than any one of them, because together they locate the pipeline on a frontier rather than at an arbitrary point.

The budget that is actually binding. Instrumenting per-task time on the rig gives 259 seconds of test-time training against 600 seconds of decode, so decode is 69% of the cost. It also never converges: across every banked row it stopped at 600, 600, 601 seconds, guillotined by its own wall. The whole pipeline wants about 859 seconds a task, and 240 tasks inside a twelve-hour budget allow far less, so tasks that run long starve the ones behind them. That matters because the score variance is largely throughput: submission size and score correlate at $r = +0.55$ across fifteen same-configuration draws, and the entire four-point observed range is accounted for by about a dozen tasks producing no answer at all.

Direction one: search more. It scored worse. The deployed `turbo_dfs` threads one KV cache through its recursion and reuses it for every sibling in the loop, position pinned to the branch, with no truncation anywhere. On the transformers version the competition image runs, `DynamicCache` appends rather than overwriting, so each sibling attends over every branch explored before it. Our own local harness clones the cache at each branch and its docstring says exactly why; the shipped decoder and the validated one had differed all along.

Cropping the cache back to the branch position is correct, is bit-identical to a full deepcopy on every task we tested, and recovers **27% more candidates** (51 against 65, ahead on 8 of 10 tasks). It also takes about 7% longer per task. Submitted three times it scored **29.72, 27.36, 29.03, mean 28.70**, against the twelve-draw same-version series at 29.90, sd $= 1.34$. That is $t = -1.49$, $p \approx 0.22$, with all three draws below the control mean. Not significant, and we do not claim it as a regression; what we claim is that a

decoder we can prove more correct did not score better, and the direction was consistently wrong.

Direction two: search less. It is an exact wash. If decode always exhausts its wall, the natural question is what the wall costs. Holding the adapter fixed and decoding the same ten tasks at three budgets:

budget	truth in pool	selected	candidates	s/task
600 s	6	5	71	859
300 s	4	3	34	559
150 s	1	1	11	409

Decode time is not being wasted; it is finding truth right up to the moment it is cut off. The throughput this buys back cancels almost exactly: at twelve hours, 600 s allows 50.3 tasks at a 0.60 truth rate and 300 s allows 77.3 at 0.40, which is **30.2 expected truths against 30.9**, a difference of six events. Cutting to 150 s is clearly worse.

Direction three: divide the wall fairly. Two draws, both below the mean. The sixteen evaluation views decode in four batches, the per-task cap is checked only between batches, and each batch may run its full 540 seconds, so on the scoring node the first batch can consume a task's whole allowance and the remaining views contribute nothing. Dividing the time still left evenly over the batches still to run guarantees every view decodes something, at the price of cutting the first batch short. Pre-registered for three draws, two of which are in as we write, it scored **29.44 and 27.78, mean 28.61**, against 29.90: $t = -1.41$, unresolved at this sample and pointing the same way as the crop. Both changes take time from the views that decode first, and the views that decode later did not give it back.

What the three mean. Moving the decoder in either direction is neutral to negative, and so is redistributing its time, so the 540-second budget the pipeline inherited is not an arbitrary constant but sits near a local optimum of a real trade. It also revises an earlier reading of ours: the floor sweep found that searching less was neutral, and we had taken that as evidence that decode time was slack. It is not. Raising the floor prunes low-probability branches, which are cheap to lose; cutting the budget stops the search wherever it happens to be, including on branches about to pay. Those are different operations and only one of them is free.

The uncomfortable observation is the first one. Under a binding wall clock, correctness and score are not the same objective, and a change that provably explores more of the intended search tree can lose on the leaderboard because it explores it more slowly. We report the crop as a defect found and fixed in the code, and separately as a change we did not ship.

4.6 Is the base limited by synthetic rule diversity?

Every result above changes the pipeline around a fixed base model, and every one of them returns to the same place. That pattern points at the base rather than the pipeline, and the base was trained on of order 10^6 LLM-generated puzzles drawn from a finite set of rules. The natural hypothesis is that its ceiling is set by how many distinct transformation rules it has seen. We hold 802: the 400 RE-ARC verifiers and

402 pairwise compositions of them built for this work. Buying more is possible and costs money, so we pre-registered a gate before spending any.

The gate: rule count at fixed example count. Two fine-tunes of the same base, each on 8,000 examples under an identical recipe, differing only in how many distinct rules those examples were drawn from: 100 rules against all 402. If diversity does nothing at 4×, it will do nothing at 8×, and the direction closes for the price of two training runs.

It did nothing. Both arms are *identical* on both evaluations: better on 1 task, worse on 0, tied on 7 of 8 composed holdouts, and better on 0, worse on 0, **tied on 12 of 12** ARC-AGI-2 tasks. The only thing that moved is candidate count, which fell 33% in the narrow arm and 25% in the wide one without changing a single solve. Four times the rule diversity is not visible in the output.

The honest reading is weaker than it looks. Twelve of twelve tied means the gate failed to detect an effect in either direction, which is not the same as establishing there is none. At this sample size only a multi-task swing would register, so the result bounds the effect rather than excluding it, and we record it as an inconclusive null rather than a refutation. A stop condition that fires on an underpowered tie was under-specified when we wrote it. The defensible claim is narrow: within composed RE-ARC rules, at 8,000 examples, quadrupling rule count buys nothing we can measure.

Rule KIND, holding count and budget fixed. The gate above varies how many rules a fixed example budget is drawn from. It does not vary what kind of rule, and every rule we hold is ARC-1: a RE-ARC verifier, or a pair of them composed. The generator corpus of Section 4.7 is ARC-2-shaped by construction, since each program was written against an ARC-AGI-2 training task and verified against that task's own demonstration pairs. Matching 100 ARC-2 rules against 100 composed ARC-1 rules at 7,840 examples each gives **0 better, 1 worse, 11 of 12 tied** on ARC-AGI-2.

Matching them properly cost an extra training run and was worth it. Phase 0's narrow arm was already trained and free to reuse, but it was built at 160 pairs per pool while the ARC-2 samplers yield 12.8, so reusing it would have confounded rule kind with pool depth by a factor of twelve and a loss for the ARC-2 arm could not have been attributed to anything. Rebuilding the ARC-1 arm at matched depth also produced a finding of its own: cutting pool depth twelvefold changed the candidate counts substantially and changed *no solves at all*.

Training DURATION, the one training-side variable never varied. Both gates above stop at 8,000 steps, and so does every other fine-tune in this paper. Since the largest single gain reported by the 200M LongT5 system is attributed to grokking (+7.78), extended training past convergence is the obvious axis, and unlike more data it costs only card time. Training the same corpus for 96,000 steps and gating at four points:

steps	validation loss	ARC-AGI-2	candidates
8,000	0.0534	tied 12/12	36 → 24
16,000	**0.0491**	tied 12/12	36 → 38
48,000	0.0670	worse 2, tied 10/12	36 → 40
96,000	0.0790	tied 12/12	36 → 55

Training loss fell to 0.0034, full memorisation of the corpus, while validation loss reached its minimum at 16,000 steps and then rose for sixty thousand more. There is no grokking signature: grokking is a plateau followed by a sharp drop, and this is ordinary overfitting. Not one task improved at any checkpoint. The 48,000-step dip is noise rather than trend, since 96,000 is deeper into the overfit regime and loses nothing.

Two things follow. Candidate count grew 53% while solves stayed flat, which is this paper's recurring observation that widening a pool the selector cannot rank buys nothing. And more usefully, **the validation optimum is near 16,000 steps while every other arm in this paper stopped at 8,000**. We had flagged that as a threat to the earlier nulls: perhaps rule count and rule kind measured nothing because both were measured undertrained. They were not. Gating at the optimum gives the same answer, which makes the earlier results stronger rather than suspect.

Three axes, one answer. Rule count, rule kind and training duration were each varied while the other two were held fixed, each was pre-registered with a stated falsification, and each is null at its own optimum. We resist calling this three independent confirmations, because all three share the same twelve-task gate, which has now tied 12 of 12 three times and plainly cannot resolve a single-task effect. The correlation runs through the instrument, not the interventions. What the three do establish jointly is narrower and still useful: within the data and compute this budget allows, no training-side variable we can reach moves coverage, and the axis that the published results credit most, pretraining at the scale of 10^6 puzzles, is the one a single 16 GB card structurally cannot pay for.

4.7 A generator corpus for ARC-AGI-2, and a trap worth reporting

The gate above tests *composed ARC-1* diversity, which is not the same intervention as ARC-2-shaped data, so we built the second thing separately. RE-ARC implements ARC-1 rules and ships generators that mint unlimited fresh instances; no equivalent exists for ARC-2. The 1,000 public ARC-AGI-2 training tasks are published for training, which is what makes this legitimate where the evaluation split would not be, and each one is a rule specification with worked examples attached.

We ask a model to write, for each task, a `solve` implementing the rule and a `gen_input` sampling fresh inputs it applies to. **Verification needs no model and admits no judgement**: a program is correct if and only if `solve` reproduces every demonstration pair of that task under execution, checked in a process-isolated worker with a hard kill timeout. That is a proof, it is free, and it is strictly stronger than asking any model whether the grids look right.

The trap. A reasoning model can spend its entire output budget without emitting an answer, and the symptom is indistinguishable from incapacity. Our first runs returned nothing at all, every extracted program empty, which reads exactly like a model that cannot do the task. It was not. The backend is a reasoning model and it needs far more budget on ARC than a generation task normally warrants. Raising the cap traces a clean dose-response, and the shape of it is the finding:

output cap	tasks	converged	verified
4,000	4	0	0
32,000	12	0	0
64,000	5	3	**3**

At 4,000 and 32,000 tokens it produced no code whatsoever, spending a mean of 31,999 of 32,000 on reasoning. At 64,000 three of five converged, at 43,488, 55,800 and 59,671 tokens, and **all three that converged verified**. The two that did not emitted *zero* answer tokens and stopped exactly at the cap. Reasoning accounts for 97 to 98% of output; the program itself is only 735 to 1,555 tokens. So the failure mode is not a wrong program, it is an unfinished thought, and a cap set anywhere below roughly 45,000 tokens converts a capable model into an apparently useless one while still charging for every token.

This reframes our own headline number, and the effort dial is not monotone. Disabling reasoning returns a complete program in about 1,600 tokens at an 11% verification rate, which is what the 110-generator corpus was built on, and at $0.0016 per task that was the only option a $3 budget allowed. Holding the tasks fixed and moving only the effort setting, on five tasks that the non-reasoning pass had already failed:

effort	verified	mean output	CNY/call	CNY per program	seconds
off (repair)	1 of 5	1,600	0.015	0.60	12
low	**4 of 5**	22,375	0.101	**0.126**	188
max	3 of 5	57,391	0.259	0.433	534

Low effort strictly dominates maximum effort here: higher yield, 2.6 times cheaper and 3 times faster. The mechanism is legible in the token counts. Two of the five maximum-effort calls stopped exactly at the 64,000 cap having emitted zero answer tokens, and one of them, `009d5c81`, is a task low effort solved in 20,238. On the single task both failed, low reached 3 of 4 demonstration pairs where max reached none. The additional reasoning is not buying accuracy; past some point the model fails to terminate and is billed for every token of it.

We take two things from this. First, **the 11% is a floor imposed by our budget rather than a ceiling of the model**, and the honest statement of this contribution is that we measured what three dollars buys, not what the method is worth. Second, **the cheapest marginal program is not the cheapest call**. Once the non-reasoning pass has swept all 1,000 tasks its 11% is exhausted, and further programs must come from somewhere more expensive: a blind retry converts the remainder at 5% for 0.21 CNY per program, low effort at 80% for 0.126, and max effort at 60% for 0.433. The setting with the lowest per-call price is the most expensive per unit of corpus, and the setting with the highest is worse than the middle on every axis at once. These are five paired tasks and the intervals are correspondingly wide; the ordering is what we claim, not the rates. the most promising unexplored direction here rather than claiming a result we did not buy.

Results. Of the 1,000 training tasks, **110 (11.0%) yielded a verified generator** and 109 of those also emitted a sampler, at a total cost of $1.57 in off-peak API charges. The low-effort reasoning setting

described above, run afterwards over 25 of the failures, verified 17 of them (68%) for about 2.5 CNY, and a blind-retry control added one more, so the corpus stands at **128 verified generators, 127 with samplers**. The corpus is usable rather than nominal: of the first 95 verified programs, **93 (98%) mint fresh valid instances**, 89 of them reliably, meaning at least 8 of 10 random seeds produce a novel grid the transform handles, averaging 9.5 usable draws per 10 seeds.

Execution against demonstrations is a 93% precise proxy for capturing the rule. The concern with a demonstrations-only proof is that a program can fit three worked examples while implementing a simpler rule than the task intends, and then mint unlimited data for the wrong transformation. The ARC-AGI-2 training split publishes held-out test pairs, so this is directly measurable: running each verified program on the test input it never saw, **93 of the first 100 reproduce it exactly**, putting the true overfit rate at 7%. This is the result that licenses the method. The free verification is a close substitute for the expensive property, and no model was needed to judge correctness at any point. It is measurable only because we built on the training split; on the evaluation split the same corpus would carry an error rate nobody could check.

The local alternative, and the ceiling reappearing where we did not expect it. An API budget is a poor foundation for a corpus, so we tested whether generation could be moved onto the same 16 GB card the rest of this paper is about. Qwen3.8-27B (released 14 August 2026, Apache 2.0, so its outputs are training data without a licensing question) is the natural candidate. The largest quantisation that leaves room for a KV cache is Q3_K_M at 13.6 GiB, which loads entirely on the GPU and decodes at **47.7 tokens per second**, or about 4.4 minutes per task. On the same five paired tasks it verified **2 of 5**, against 4 of 5 for the hosted model at low effort. We cannot separate the model from the quantisation with this evidence, and program synthesis punishes quantisation asymmetrically, since one wrong index fails verification outright with no partial credit.

Two results from that exercise are worth more than the comparison. First, **the non-termination failure reproduces in a second model family**. Left unconstrained, the local model spent its entire 20,000-token allowance without emitting a program, exactly as the hosted one did at 32,000 and 64,000. Imposing a 10,000-token thinking budget made it verify the same task it had just failed. The pathology is therefore not a quirk of one vendor's effort dial, and an independent report of the same model *"defaulting to wildly overthinking things"* appeared within days of its release [21].

Second, and more pointedly for this paper, **the standard remedy for slow decoding is unavailable at 16 GB**. The distributed weights include multi-token-prediction layers, which llama.cpp can use for self-speculative decoding, and which the server otherwise loads and discards. Enabling them took throughput from 47.7 to **6.96 tokens per second**, a 6.3-fold *loss*, because with 13.6 GiB of weights resident there is only 258 MiB left and the draft context has nowhere to live. Speculative decoding trades memory for bandwidth, and at this capacity there is no memory left to trade. The constraint this paper is named for does not only cap how large a model can be; it removes the optimisation that would have compensated.

Bimodality appears again, in a different model at a different task. Section 5.1 records that the 4B solver is bimodal, solving or missing with little partial credit, and that this is why refinement methods need an external partial-credit signal. The generating model is bimodal in the same way, though less sharply than a 30-task pilot first suggested. That pilot showed 1 of 27 failures matching even one demonstration

pair; at full scale the figure is **63 of 483**, or 13%. The direction survived and the magnitude did not, which is the same small-sample lesson this paper records elsewhere, arrived at once more by our own hand. A further 407 failures, 46% of the total, are programs that raise rather than programs that answer wrongly, so the near-miss tail available for repair is thinner still than the clean-failure rate implies.

5 Discussion

5.1 Five lessons that generalize

First, on a single GPU the inference engine is the dominant design choice. Quantization and sampling mattered far less than the 13× we recovered by moving decode off the eager path. Second, the real obstacle for TTT is the *serving bridge*. Training needs a gradient framework, fast generation needs an optimized engine, and per-task TTT produces a fresh adapter the engine must serve. We show this bridge is buildable on one card, and that once it exists the open question is not plumbing but *regularization*, because the small per-task training set overfits easily. Third, a 4B model is bimodal on these tasks. It solves or it misses with little partial credit, so the refinement family that defines the current frontier needs an external partial-credit signal to work at this scale. A method that only re-ranks the model's own outputs stalls (Section 5.3).

Fourth, public-eval gains must be re-validated in the exact deployed condition, because they do not transfer on their own. The level drops, the no-TTT system halves from 12.79% public to 6.53% hidden. More telling, the deltas reverse. The three candidate-side levers that lifted or tied the public evaluation, colour permutation with multi-scale voting, more-candidates sampling, and the plausibility filter, each regressed the hidden TTT score when submitted (Table 2). Two mechanisms compound. A lever validated on the no-TTT base meets a different system once the adapter has sharpened the vote, and a lever that measures neutral on one task distribution still carries tail risk on another. The operational rule this bought is that a lever counts as validated only in the exact submission condition, and a lever that measures neutral is never shipped.

That rule has one clean counterexample and it sharpens the rule rather than weakening it. Repairing the community port's two defects was worth eleven points on the public slice and carried to the hidden set in the same direction (Section 4.1.1). What transferred was a repair, the removal of a component that was actively discarding tasks and a batching change inside the search. What failed to transfer, every time, was a lever that re-weighted selection among candidates the system already had. The distinction is the same one Section 5.7 reaches from the pool statistics, and it is a more useful predictor than public magnitude: ask whether the change fixes something broken or tunes something working.

Fifth, and most expensive here, confirm that the effect you are explaining exists before you explain it. A public notebook displaying a score three points above ours produced, over one week, three quantified explanations of the gap, one of which we elevated to a central claim that the software environment outweighed every method lever we had measured. Forking that notebook unmodified and submitting it scored 2.91 points *below* our own build (Section 4.1.1). There was no gap. The control was the cheapest experiment available and we ran it seventh, after the explanations, because each explanation felt like

progress in a way that re-measuring the baseline did not. Two properties of a research question should now trigger the control first: the effect comes from an artefact whose provenance we cannot audit, and the effect is the premise for a programme of work rather than one comparison. Both held from day one. The general form is that a number is evidence about the thing that produced it, and a leaderboard score beside a public notebook is produced by a submission whose code and inputs are not the ones on display.

5.2 Accuracy per GPU-hour

We argue the right framing for compute-poor ARC work is **accuracy per GPU-hour** (Figure 9), and the map of walls reported here (the decode ceiling, the VRAM limit on large-grid TTT, and the overfitting of naive adapters, all in Figure 9) is a first step toward it. The findings tell a practitioner with similar hardware exactly which problems to solve first. Fix the engine before touching anything else, budget VRAM by largest-train-pair length, and regularize the per-task adapter rather than training it harder.

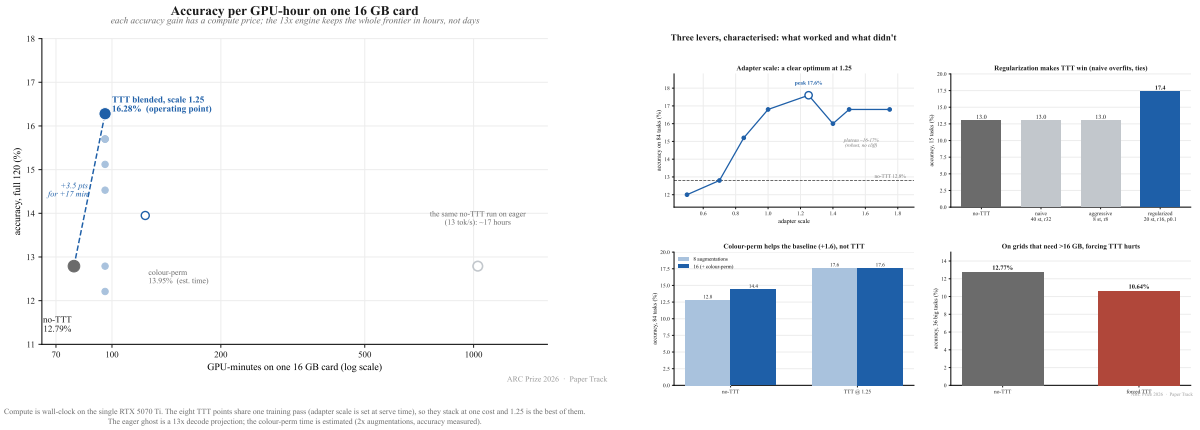


Figure 9: The left panel shows accuracy per GPU-hour on one card, where the TTT blend at 16.28% sits on the Pareto front (compute is measured wall-clock, and any eager "ghost" points are 13× projections, labelled in the source). The right panel shows what worked and what did not. The engine swap and regularized TTT helped, while naive TTT, colour-perm stacking, and big-grid TTT did not.

5.3 Limitations and ongoing work

Our absolute accuracy is below NVARC's 24.0% winning result, by design. That score was set offline on four L4 GPUs, not a cluster, by a more developed recipe on the same Qwen3-4B base lineage, so the gap reflects recipe sophistication (synthetic data, ensembling, TTT scale) more than hardware. We study the one-card inference-plus-TTT recipe rather than maximize a system. The public-to-hidden gap is a finding, not a caveat (Section 5.1). Levels drop, the no-TTT system carries 12.79% public to 6.53% hidden while the TTT blend carries 16.28% to 13.61%, and deltas reverse, with every public-validated candidate-side lever regressing when submitted (Table 2). The public ARC-AGI-2 eval is the set every open recipe is tuned against, ours included, so the leaner the recipe the more of its public accuracy is public-specific. Part of the v1 drop may also be per-task failures in the scored rerun, which the later submissions instrument

with a per-task log, and the TTT system's smaller gap is consistent with both readings. The 2025 analysis names verifier-guided selection and a refinement loop (check each candidate against the train pairs, then re-prompt on the discrepancy) as the central advance, so we built and tested it. It does not lift here. A passive form (weighting the augmentation vote by leave-one-out train fidelity) and an active form (verification-gated re-sampling on the tasks the base is unsure of) each match the plain no-TTT vote exactly on the eval slices we ran. The reasons are structural. The verifier is silent exactly where it is needed, since on a task the base misses it reproduces none of the held-out train pairs, so there is nothing to select between or correct toward. Our 16-token model also cannot read a natural-language critique, so the canonical re-prompt-with-the-discrepancy step is not even expressible. We also probed whether a one-time continued LoRA fine-tune of the base on official ARC-AGI-2 training data and RE-ARC [6] generations yields a better static model. It did not. The fine-tuned base scores 11.63% no-TTT against the stock base's 12.79%, so we ship the stock base. We recorded that at the time as evidence that more public data is not a free lever once a base is task-tuned, which was the right conclusion for the wrong reason, and the correct reason is worth more.

Reading the base model's own published training configuration settles it. Our base is the product of a full supervised fine-tune of a reasoning-tuned Qwen3-4B over a corpus whose named components include both the official ARC-AGI-2 training split and RE-ARC generations, alongside a synthetic set of order 10^6 puzzles built by an LLM pipeline that summarizes puzzle concepts, mixes them into hybrids, generates transformation code and executes it. We had therefore fine-tuned the model on two datasets it had already consumed. The 1.16-point regression is duplicate-corpus overfitting rather than a statement about data scale, and the general lesson is narrower and more useful than the one we drew: before adding data to a task-tuned base, read what the base was trained on, because the failure mode of training on a subset of your own pretraining corpus looks exactly like the failure mode of the data being useless.

The same document bounds what a single card can do about this from above. That fine-tune ran as a full update under tensor-parallel training across eight GPUs, not as an adapter, so reproducing or extending the base is not a matter of patience on one 16 GB card. The only base-side lever that remains open to us is data the published corpus does not already contain, which means running that concept-mixing pipeline ourselves, and we state it as an option we have scoped rather than one we have tested.

Building on that null result, we tested an expert-iteration self-training loop. We solve the official training tasks with the current model, distil the base on a curriculum of its own verified-correct solutions, and iterate. Exact-match grids impose a sharp constraint. A verified-correct solution on a training task simply is its ground-truth output, so a naive self-training corpus coincides with the data the continued fine-tune already used. We hoped to escape this through the curriculum, distilling only the near-miss tasks the base reaches but does not solve robustly, but a 200-task harvest shows that set is nearly empty. The base self-solves 194 of 200 training tasks under some augmentation (97%) and 191 by the robust vote (95.5%), leaving only three near-miss tasks. The base is saturated on its own training distribution, so self-training has no signal to add. The training tasks split into already-solved (nothing to learn) and unsolved (no correct trace), with almost nothing between. This is a second, sharper negative result on weight refinement, and it explains the fine-tune failure directly. On one GPU, with a base already tuned to the grid format, neither continued fine-tuning nor self-training moves ARC-AGI-2, because the public training data carries no signal the base has not already absorbed.

These three null results, the fine-tune, self-training, and the refinement loop, share one cause. The base is bimodal on ARC. It solves a task robustly or misses it entirely, with almost no partial-understanding middle (of the training tasks it harvests, 95.5% are solved robustly and only 1.5% are near-misses). Every method that refines, re-ranks, or distils the model's own understanding therefore has nothing to grip on the tasks it misses, and per-task TTT, the one lever that helped, is the exception that proves the rule. It does not refine partial understanding. Instead it manufactures new capability for the task. The implication for compute-poor ARC is concrete. The next lever must supply an external, continuously-gradable signal that is valid even where the base is lost, a program checked against the train pairs being the cleanest, rather than another pass over the model's own grid outputs.

We built that arm and measured it. A 1.5B model trained to emit programs in an executable DSL, sampled and kept only when a program reproduces every train pair, solved zero of the 101 ARC-2 evaluation tasks our transductive system misses. It generalized weakly on the easier ARC-1, about one held-out task in nine, but on ARC-2 the DSL cannot express the new compositional concepts, and the tasks the base misses are also the largest grids where the small proposer exhausts the card before it can search. The external-signal lever is therefore real but bounded by model scale on one GPU, the same wall the refinement family hits from the other side. The lever exists but needs a larger model to grip, which is exactly the move the full contest hardware allows (Section 5.4).

One experiment on this lever is built and unrun, and its motivation changed underneath it in a way worth recording. The community port carries a symbolic arm that we deleted wholesale (Section 4.1.1), and it returns two independent things. One is a solved flag that preempts the learned search entirely, and that half is measured harmful, taking 12 of 39 slice tasks with nothing correct to show. The other is an object count read off the demonstration pairs on the CPU while the adapter trains, which never preempts anything and only re-ranks beams the search has already produced. Our stub killed the second silently along with the first, and it is a weak but free instance of exactly what this section argues for: an external signal, valid where the base is lost, obtained by reading the demonstrations rather than the model's own beliefs, and therefore outside the ceiling of Appendix F that binds every selector built from model likelihoods. It costs no GPU, which is the constraint that defeated the 1.5B program synthesiser above.

We had staged that experiment to close a three-point deficit to a public notebook. The control in Section 4.1.1 then showed the deficit was never there, which removes the reason we were running it and leaves the reason we should have been running it all along. It is now a single-factor test on top of our best configuration rather than a catch-up move, and it is the only one of the three staged builds still worth a submission slot: the other two revert our changes toward a configuration the control has already measured at 28.06. The prediction registered before it runs is a gain of no more than two or three task credits, since the count is a coarse statistic that is silent on most tasks, and we note in advance that a coarse external signal producing a small real gain and a coarse external signal producing nothing are separated by less than the width of our error bar on a single submission.

Finally, the significance of the TTT lift is directional rather than conclusive (Appendix B). A full-eval paired run is the obvious next measurement.

5.4 Scaling to the full contest hardware

Everything above holds a single 16 GB card fixed, but the offline contest gives four L4 GPUs, 96 GB together, and that hardware is where the real climb happens. As of late July 2026 the live public leaderboard is led by one entry near 65.8%, with second place near 38.6% and a chasing pack in the mid thirties, and every one of those runs uses the same offline no-internet sandbox our system does. On the available evidence the leader is the same NVIDIA Kaggle Grandmasters lineage that won the 2025 offline track and whose grid model our system reproduces, and their stated recipe is to move all complex reasoning offline into a large synthetic-data pipeline and train a small model that runs fast at evaluation [3]. That framing matters for where the twenty-seven-point gap to second place actually lives. The contest's own 2025 technical report attributes the distance between the offline field and the higher application-track numbers to knowledge coverage and verifier-guided optimization rather than to any fundamental scientific breakthrough [3], which is to say the remaining lever is training-data scale and a working refinement or verification loop, not a decoding or selection trick of the kind we exhausted here. The higher numbers often quoted for ARC-AGI-2, a 54% Gemini refinement harness and frontier scores above 70%, come from a separate public track that allows internet and external compute, so they are not the bar we measure against [3]. The offline bar is the leaderboard above. The field runs test-time adaptation, heavy synthetic-data pretraining, and program search on the full 96 GB, and our best single-card score of 18.19% sits below it. Our faithful reproduction of the winning lineage's own pipeline, run on the four-L4 node it was built for, reached 30.97% and still sits far below the leader, which locates the gap in the base model and its training data rather than in the inference-time machinery we can run. None of our negative results argues against that climb, but none of them tests it either, so we state the scaling case as a falsifiable hypothesis for future work rather than as a claim. Every wall we mapped belongs to a 4B model on a single card, and the sharpest is bimodality. The model solves a task robustly or misses it entirely, which is the exact reason refinement and verifier-guided selection found nothing to grip. The hypothesis is that bimodality is a property of this model's scale and training rather than of the task. The falsifiable prediction is that a more capable base run in the 96 GB produces partially-correct candidates on tasks it does not solve outright, a measurable non-empty near-miss band where ours is 1.5%, and that this in turn reactivates the refinement and program-search levers that stalled here. Three of our measurements bound the hypothesis from below. A second small transductive model buys almost nothing, since our fine-tuned variant solves only two tasks the stock 4B misses. A union of our own two pipelines buys as little. The deep-TTT greedy vote and the restored DFS decoder share the same 4B and produce nested solved sets, the greedy arm adding exactly one task the decoder missed over the tasks both attempt, and that task already sat in the decoder's own candidate pool and was merely lost at selection, so the apparent diversity is a selection artifact rather than new capability. The Tiny Recursive Model family, the strongest small grid models available, scores far below our pipeline while needing more compute than one card gives it. That gap has not closed with time. The original TRM reports 8% on ARC-AGI-2, its 2026 denoising-recursion successor reaches only about 9.6% [3], and the winning team's own ensemble measured a TRM arm as adding nothing to a strong 4B, so any diversity that pays must come from capability rather than from a second small vote. One published result warns against reading the hypothesis as a bare parameter-count claim. A $\sim$200M encoder-decoder system with test-time training and white-box filtering reports 27% on the ARC-AGI-2 semi-private set, ahead of the 4B-based

2025 winner [7], so method quality can dominate size, and our own four failed elaborations of the 4B's candidate machinery (Table 2) sharpen the same point from below. We read the 16 GB result as the place where this small-model recipe runs out. The prediction above is the test of what replaces it, and we leave it as stated, a hypothesis with a measurement that would falsify it, rather than as a promise.

5.5 A routed dual-arm system

Section 5.3 argued that the next lever must supply an external, continuously-gradable signal that is valid where the base is lost, and Section 5.4 offered one escape, a larger non-bimodal base, as a falsifiable hypothesis. We are building a second escape that stays on the single-card efficiency budget rather than betting on scale, and it follows directly from two measurements already in this paper. The AIRV vote agreement predicts correctness (Figure 8), so the grid model reports for free which tasks it has solved and which it has no grip on. And the base is bimodal, so on a missed task there is nothing to refine, only something to replace.

The system routes on the first fact and fills the gap with a method that does not share the base's blind spot. High-consensus tasks are banked with the grid model's own answer, which is most of the current score at a fraction of the compute. Low-consensus tasks, where the transductive model scores near zero, are routed to an induction arm, a code model that writes plain-Python grid transforms which we execute against the train pairs. A program that reproduces every train pair is verified on observable data in a way no vote can be, so its output becomes the first attempt and the grid model's guess falls back to the second. The two arms are complementary by construction rather than by hope, induction expresses the precise compositional rules transduction misses.

The design is deliberately do-no-harm. A banked task is byte-identical to the transductive submission, and a program is trusted only when it reproduces every train pair, so the arm can only add coverage on tasks the base was already missing. This is the property the five candidate-side levers lacked. Each of them rewrote the vote on every task including the ones already solved, and each regressed (Table 2). The one loss channel, a program that fits every train pair yet is wrong on the test and displaces a correct second attempt, is measured on the local eval before any submission. The arm also inverts the engine economy of Section 3.2. Verifying a program by running it is cheap, and where several programs survive we rank their outputs under the grid model's per-view likelihood, which is prefill-priced and about thirty times cheaper than the decode that produced them, so the system generates less and checks more. The whole arm runs inside the idle two-thirds of the offline budget the transductive submission leaves unspent.

This is the same idea whose cheap form failed in Section 5.3, and it is built to answer why. That probe used a 1.5B model emitting a fixed ARC-1 domain-specific language and solved none of the 101 missed tasks. The arm here uses a 14B code model emitting unrestricted Python, which addresses both named causes at once, model capacity and an expressive target language.

Status. The first probe is a disjoint-coverage run on the single card. A 14B code model (Qwen2.5-Coder) samples sixteen programs per task at temperature 0.8, on exactly the 101 evaluation tasks the transductive system misses, and a program is kept only if it reproduces every train pair. It solved none of them. More telling than the zero, across roughly 1600 programs only one task saw a program reproduce even a single

train pair, so the model is not close anywhere. The programs are not the failure mode the arc-dsl proposer was. They are fluent and sensible. On a task whose output size changes with every example the model wrote a clean routine that extracts and sorts rows, which crashed because its fixed-shape hypothesis cannot produce a variable-size output, and on a task with a narrower output it wrote correct code for the wrong rule and returned the input width. The 1.5B DSL proposer failed because it could not express the concepts. This larger model expresses them freely and still cannot infer the rule from a few examples. The bottleneck is inference, not expression, and a general coder does not close it zero-shot on one card. Priming the coder with three verified worked-example solvers, one of them a variable-size crop aimed squarely at that failure, did not change the outcome. It solved none of the 101 and lifted the partial signal only from one task to two, with nothing reaching a near-miss. Two forms of a strong single-card coder, cold and primed, both come back empty, which is itself the result. The bottleneck is rule inference on the hardest tasks, and neither sampling more programs nor showing worked examples closes it at 14B on one card. A same-size reasoning model, which works through the transformation at length before it writes code, does not change the verdict either. On a bounded run over twelve of the missed tasks it solved none and reproduced no train pair, at roughly four minutes of reasoning per sample. Three model classes, a small restricted proposer, a strong instruct coder cold and primed, and a reasoning model, all fail to induct these hardest tasks on a single card, and the near-miss band stays empty throughout. We read this as a decisive single-card result rather than a tuning gap. The router is sound, vote agreement does report which tasks the grid model has solved, but the arm it would route to cannot grip the tasks the grid model misses without more compute than one card provides. That is the same wall Section 5.4 predicts, now measured from the induction side, and it is where the second arm's value waits on the full contest hardware.

5.6 A stronger public decoder, and its coupling to training

The clearest evidence that the gap in Section 4 is method rather than model arrived from outside the project. A public notebook ports the 2024 ARC-Prize-winning pipeline, the ARChitects', onto our exact base and reports a score in the low thirties on ARC-AGI-2, more than double our operating point on the same weights. The port's lineage deserves naming. The pipeline is published by Ivan Sorokin, the author of the NVARC base model itself, as the notebook `arc2_qwen3_unsloth_flash_lora_batch4_queue`, and the variant we study is its community re-target to the 2026 contest. Everything we report about this system is reproduction and measurement of that public work, credited to its authors, and our contribution is confined to the analysis around it. It bundles two changes against our recipe. The per-task training is an order of magnitude heavier, a rank-256 adapter that also trains the embeddings and the output head at an eight-thousand-token context, where ours is rank sixteen at two thousand. And the decoder is not our greedy vote. In its place it runs a depth-first search over every completion whose running probability stays above a fixed floor, then selects among the survivors by the product of their likelihoods across fresh augmentations rather than by a majority.

We had named this decoder as the untried lever ourselves and never built it, so the first question is whether it is a separable part we could capture on one 16 GB card without the heavy training it ships with. We test it directly. Holding the adapter fixed at our rig-feasible configuration, we decode each task twice on the identical trained model, once with our greedy vote and once with the search-and-select, and score both against the solution. Only the decoder moves.

On our light adapter the search loses, and it loses for a reason that names the coupling. Over eighteen of the trainable tasks the greedy vote scored the equivalent of 5.17 solved tasks to the search's 3.33, 28.7 percent against 18.5. The mechanism is visible in the candidate counts. On task after task the search returns nothing at all, because no whole-grid completion clears the probability floor. A grid runs to about nine hundred tokens, and a little uncertainty at each cell compounds across all of them, so an adapter that is not sharply peaked pushes the probability of even its best completion under the floor. That floor is calibrated for a model trained hard enough to stay almost certain across the whole grid. Ours is not, so the search starves and forfeits tasks the greedy path still lands. The decoder is not a model-agnostic component we can lift. It is tuned to the sharp adapter the heavy training produces, and the two are one system rather than two levers. This is also why we could never have reached the field's level by swapping decoders on the pipeline we already had.

Status. Resolved, and the coupling did not save the decoder. Over thirty trainable tasks the rank-128 adapter, with the embeddings and output head trained as far toward their configuration as a 16 GB card holds, gives the greedy vote **24.4%** against the search's **13.3%**. The search fell further behind than it was at rank 32, not closer, because the sharper adapter still starves the probability floor on most tasks while the heavier training only helped the vote. On every configuration a 16 GB card can run, rank 32 and rank 128, the search loses to the vote. We read this as the decoder being inseparable from the full rank-256, eight-thousand-token training it ships with, which one card can hold but cannot serve inside the contest budget (Section 4), so the reach to the field's low thirties stays a claim to be settled on the contest hardware rather than a lever we could capture here. The lift we did capture, from 13.61 to 18.19, came from the training depth alone. A first hidden-set submission of the reproduction itself, run faithfully with the community port's bolted-on symbolic arm still active, scored 16.81%. Two facts inside that number matter more than its size. The public-to-hidden transfer is flat, 16.2% on the evaluation slice against 16.81% hidden, where every candidate-side lever we built ourselves lost points crossing that gap. And the symbolic arm is measured poison, on the evaluation slice it preempted twelve of thirty-nine tasks with its own answer and got none of them right, so the faithful reproduction understates the pipeline. The arm is stripped and the corrected variant is measured next. That measurement is in, and it also surfaced a second inherited defect, the community port had split the decoder's four-view batches into pairs, halving search throughput. With the symbolic arm removed and the original batching restored, the same evaluation slice rises from 16.2% to **27.2%**, no tasks preempted, no timeouts, and half again more correct answers present in the search beams. Eleven points of the gap between the reproduction and its source's claim were the port's own mutations. Submitted to the hidden set, the restored variant scored **30.56%**, against 16.81% for the same pipeline as ported and 18.19% for our own best single-card configuration. Three things follow. The transfer property held a third time, 27.2% on the evaluation slice against 30.56% hidden, so this pipeline carries its public number across the gap that every candidate-side lever of ours fell through. The original author's claimed tier is credible, since a faithful restoration lands within striking distance of it while the ported form did not. And the result is a reproduction, not a contribution of ours. What this work adds around it is the measurement, the isolation of the two port defects, the paired evaluation that priced each one, and the placement of the restored number against a single-card baseline so the hardware gap becomes visible.

5.7 The selection leak, measured on the candidate pool

A search-and-select decoder is only as good as its selector, and the restored pipeline of Section 5.6 throws away correct answers it has already found. To measure how many, we ran a build identical to the submitted one except for a gated dump of the entire candidate pool. It records, for every test output, each distinct candidate grid together with the number of augmented views that produced it, the beam likelihood of each view, and the per-augmentation likelihoods the selector consumes. Over forty evaluation tasks the run produced 60 test outputs across 39 tasks, one of the forty yielding no decoded output at all, holding 432 distinct candidate grids drawn from 847 decoded subkeys, of which 152 reproduce the ground truth exactly. Scoring follows the harness, where each task contributes one credit divided among its test outputs.

The dump is a replica of the deployed decision rather than a model of it. Reimplementing the shipped selector against the file returns 9.833 credits of 39, equal to the run's own scored output, so a candidate rule tested offline here is tested against the real thing at no GPU cost.

Table 7: Selection against the pool ceiling on the 40-task measurement run (60 test outputs, 39 tasks).

selection rule	outputs solved	credits
published vote, deployed	15	9.833
confidence-weighted vote	15	9.833
augmentation likelihood alone	16	10.333
correct grid present in the pool	**19**	**12.333**

The leak is large. Nineteen of the 60 outputs hold a correct grid somewhere in their pool and the deployed rule converts fifteen, 9.833 credits against a ceiling of 12.333, so 2.5 credits or 6.4 points of this slice sit inside candidates the system generated and then ranked away. The published rule and the confidence-weighted alternative select identically on all 60 outputs, neither gaining nor losing one, which is a third independent reading that the two are near-equivalent once deployed.

We then asked whether any reweighting of the statistics already in the pool closes that gap. Sixteen rules were scored against the dump, spanning the vote count alone, the beam likelihood alone, the augmentation likelihood alone, the deployed count-minus-penalty form at five weightings, a best-view variant taking the minimum augmentation penalty rather than the mean, and a length-normalised family motivated by summed log-likelihoods penalising physically larger grids. Exactly one rule gains. Ranking by augmentation likelihood alone, discarding the vote count entirely, recovers one output and loses none, for sixteen solved and 10.333 credits. The length normalisation is inert, since only 11 of the 60 outputs contain candidates that differ in size, and the normalised and unnormalised forms return identical solved sets.

Table 8: The four outputs whose pool holds a correct grid the deployed selector ranks outside its two attempts. Views is the number of augmented decodes producing that grid, penalty its mean augmentation negative log-likelihood, where lower is better.

test output	pool size	correct grid (views / penalty)	selected grid (views / penalty)
`142ca369_0`	10	3 / 18.64	4 / 10.57
`3dc255db_0`	13	2 / 13.57	1 / 7.36
`3a25b0d8_0`	7	1 / 5.96	7 / 4.69
`4c7dc4dd_1`	11	1 / 12.90	3 / 8.79

That gain does not survive inspection, and the reason is in the four outputs the deployed rule misses (Table 8). In every one the correct grid carries a heavier augmentation penalty than the grid the selector chose, so the signal does not merely fail to surface these answers, it actively orders them below the wrong one. On one output the correct grid was decoded more often than the grid that beat it and still lost, on the penalty term alone. The single recovered output is a correct grid moving from third place to second, which slides it into the second attempt slot while still losing outright. Two of the four are reachable by some rule in the family, each time at the price of losing outputs the deployed rule already holds, and the other two are reached by nothing we tried. On three of the nineteen recoverable outputs the correct grid was decoded exactly once in the whole pool, so no vote-based rule can reach them in principle.

The exercise also carries a warning about the slice it ran on. An earlier reading of these same forty tasks found three discordant outputs favouring the confidence-weighted rule over the published one, with a clean mechanism behind it, so we shipped the switch, and a 120-task run then reversed it. This sweep points the opposite way on the same tasks, that discarding the vote count beats keeping it, on the evidence of one output in sixty and as the best of sixteen rules tried. Two contradictory prescriptions of equal statistical weight, from one evaluation slice. We therefore record the count-free rule as measured and leave the deployed selector alone.

What we carry forward is the negative, and it is sharper than the headroom number by itself. The selection leak is worth roughly six points on this slice, and it is not recoverable by rescoring what the pipeline already computes, because every statistic in the pool ranks the discarded correct grid below the chosen one. Closing it needs a signal the pool does not contain, consistency against the task's own train pairs, which is the verifier direction Section 5.3 finds silent exactly where it is needed. The contribution of this measurement is to eliminate the cheaper alternative. Reweighting is exhausted, and the instrument for any future attempt is the pool dump itself, which makes selection rules testable offline without further contest compute.

The signal that paragraph asked for has since been built, and it fails too. It is worth reporting because the construction avoids the objection that sank our earlier verifier, which scored an adapter rather than a candidate and so could not choose between two candidates the same adapter produced. Here a candidate is scored as evidence. An output proposed for the test input is either a true instance of the task's rule or it is not, so appending the pair of test input and candidate to the task's demonstrations either hands the model a fourth true example or hands it a contradiction. We measure the negative log-likelihood of a held-out real demonstration's output twice, once from the remaining demonstrations alone and once with

the candidate injected, and average the difference across held-out choices. A negative delta means the candidate behaved like a true example of the rule. Scoring a difference rather than a likelihood cancels per-task and per-fold difficulty, and the whole thing costs forward passes rather than decodes, so it is affordable over every candidate inside the contest budget.

A cheap gate ran first, over all 172 evaluation test outputs, scoring the true grid against four distractors, the test input unchanged, the true grid with two colours swapped, the true grid transposed, and another task's answer. The true grid ranked first on 133 of them against a chance rate of one in five, which read alone looks like a working signal. The margins say otherwise. The median winning margin is 0.0043 nats and the median losing margin is 0.0031, so the two distributions sit on top of each other, and 58 percent of the wins are by less than 0.005. The shape of the failures matters more than the rate. Another task's answer loses 171 times out of 172, so the score reliably rejects a grid unrelated to the rule, while transpose beats the truth 17 times, colour swap 11 and the unchanged input 10. Those three preserve the structure of the answer and change its content, which is precisely what a decoder's near-misses are.

Table 9: Round-trip candidate scoring against the deployed selector, on the same 40-task pool as Table 7. Credits are out of 39.

selection rule	outputs solved	credits
published vote, deployed	15	9.833
round trip alone	7	5.500
published vote, round trip as tiebreak	15	9.833
correct grid present in the pool	**19**	**12.333**

Against the real pool it is worse than the rule it was meant to improve (Table 9). Ranking by the round trip alone solves 7 of the 60 outputs where the deployed rule solves 15, losing eight and gaining none, and applied as a tiebreak it changes nothing at all, because the deployed rule produces no exact ties to break. On the four outputs the leak consists of, the correct grid ranks seventh of ten, seventh of seven, tenth of thirteen and fourth of eleven. A merely noisy score would scatter those near the middle. Landing in the bottom half on three of four says it is measuring something real and something other than correctness.

That something is grid complexity. A simple, repetitive, low-entropy candidate is cheap to append and perturbs the model very little, so its delta sits near zero, while a complex correct answer perturbs more. The score therefore rewards candidates that are easy to encode over candidates that are right, which is also why the unchanged test input, usually the structurally cheapest grid on offer, beat the truth ten times in the gate.

One qualification is owed. The scoring ran against the unadapted base model, because the per-task adapters that produced this pool were discarded with the contest container, and a sharper adapted model would react more strongly to a contradictory demonstration. We report the number as a floor for that reason. It is not a floor we expect to lift far, because the failure is systematic rather than weak, and a complexity bias does not dissolve under better adaptation.

Two further attacks followed, on the attempt slots and on the grids themselves, and both close the same

way. The first observed that our two attempts come off a single ranking, so when the top two candidates are near-duplicates the second slot restates the first, and tested spending attempt two on the highest-ranked candidate at least k cells away from attempt one. At every k tried the rule loses one output and gains none, because the geometry runs opposite to its premise. On `4c3d4a41_1` the correct grid is rank two, one cell from rank one, so any diversity requirement skips it, and on `3dc255db_0` the correct grid ranks sixth while sitting two cells from the chosen wrong answer. When this pipeline misses, its miss is a near-duplicate of the truth, and the distant candidates a diversity rule promotes are noise.

That anatomy suggested the opposite mechanism, fusing the near-duplicates instead of avoiding them. A per-cell weighted vote across same-shape candidates can emit a grid that appears in no candidate, which grid-level voting cannot do by construction. It emits nineteen novel grids on this pool and not one of them is correct. Count-weighted fusion in the second attempt slot is exactly neutral, and every other weighting regresses. The failure case is the informative part. On `3dc255db_0` thirteen same-shape candidates fuse to a grid two cells from the truth, and they are the same two cells the selected answer gets wrong. The candidates' errors are correlated, so no vote among them can cancel what they all agree on.

We stop here, and the stopping is the finding. The leak is the largest measured gap in this system, 6.4 points on this slice and roughly fifteen task credits at contest scale, and it has now turned back four attacks with unrelated mechanisms, sixteen reweightings of the pool's own statistics, an out-of-pool consistency score that ranks the answers backwards, a diversity-forced second attempt, and cell-level fusion. The correlated errors explain all four at once. The model does not hold a noisy version of the right rule that clever aggregation could denoise, it holds the wrong rule, and every candidate it produces expresses that same wrong rule. This is the bimodality of Section 5.4 surfacing at cell level, and it relocates the leak from a selection problem to a training problem. The information needed to pick the correct grid is absent from the pool and from the model that produced it, so any repair has to change what the model believes, not how its outputs are ranked. Appendix F states this as a dominance proposition over the pool's statistics and verifies its premise on all four missed outputs.

5.8 Revisiting the big-grid wall with the host's memory

The journey of one cap tells this subsection's story. Thirty-six of the 120 evaluation tasks have a train pair larger than a thousand tokens, and their per-task training runs out of memory on 16 GB, so from the first TTT submission onward they were excluded and served without adaptation. We tested that exclusion once, early, by forcing TTT onto them with the training context capped at 600 tokens, the most that then fit. It scored 10.6% against 12.8% without, and we recorded the loss as "the VRAM exclusion is a genuine ceiling, not a missed opportunity." That sentence survived every draft since. Rereading the measurement now, it does not support the sentence. A single train pair on these tasks is at least 1010 tokens, so a 600-token cap admits exactly one demonstration into the training context, and the adapter learned each step from a single example where the small tasks see several. What we measured was truncation, not the ceiling.

The second look came from turning away from the card. The rig carries 64 GB of system memory that sits idle while the 16 GB of VRAM starves, and during TTT the base model is frozen, only the small adapter trains, so the 7.5 GB of frozen weights never need to live on the GPU at all. Streaming them from host

memory through a device map, with a 6 GiB resident-weight budget on the card, frees the VRAM for what actually grows with grid size, the activations, and those stay small here because the 16-token vocabulary keeps the output layer tiny even at long sequence lengths. Under that arrangement the training context rises from the old 600-token cap to 8000 tokens, above the 7471-token worst case across all thirty-six tasks, which is to say literally full context, the same leave-one-out signal the small tasks have always had. The cost is streaming time, the frozen weights cross the PCIe bus every step, which is unaffordable inside a contest budget and nearly free on the rig.

Status. The staged probe has run its first two gates and the ceiling sentence is already refuted in its strong form. The smoke stage trained the single largest excluded task end to end, 14.6 GB peak on the 16 GB card with the base streaming from host memory, every mechanical check passing. The six-task stratified panel then produced the first solve this excluded set has ever yielded, task 8e5c0c38 moves from zero of two test outputs without adaptation to one of two with its full-context adapter, and no panel task got worse. So at least one of the thirty-six is winnable, and what was recorded as a genuine ceiling was, at least in part, the truncation artifact Section 4.4 now flags. Training losses tell the mechanism, adapters on these tasks now converge to near zero where the 600-token cap never could. One operational wrinkle surfaced, on the longest-context tasks Windows silently overcommits GPU memory into host RAM rather than failing, which crawls training, and the affected adapters were retracted and retrained at a lower resident-weight budget. The full thirty-six run is complete and the count is decisive. Across the excluded set's 47 test outputs, the unadapted system solves 4 in this paired harness and the full-context adapters solve **8**, five tasks flipping to solved against one flipping away, that one a displacement the two-attempt format removes in deployment by carrying both arms. Four of the largest tasks remain honestly flagged undertrained where even the reduced-budget training exceeded its time cap, so the 8 is a floor with headroom. The doubled count settles the question this subsection opened. What we recorded as a genuine ceiling was in substantial part a truncation artifact, and the excluded third of the evaluation is partially winnable on this card with the host's memory carrying the frozen weights.

One postscript, because it may make the streaming unnecessary. The activation growth that forced the frozen weights off the card in the first place was measured under an attention kernel the framework had chosen by default, and forcing the cuDNN backend instead trains rank 256 at eight thousand tokens fully resident in 14.86 GB (Section 4). Our big-grid configuration is rank 32, well below that, so these tasks would plausibly train resident and pay no streaming cost at all, which would move the arrangement from unaffordable inside a contest budget to ordinary. We have not measured that cell, so we state it as the obvious next probe rather than as a result, and we leave the streaming numbers above as they were measured.

5.9 What the budget is actually spent on, and a policy the pipeline cannot express

Every lever in Section 5.7 operates on candidates the system already produced, and that section closes them by proof. The measurement run's own timing log points somewhere else. Instrumenting each of the forty tasks with its wall time, final training loss and outcome gives Table 10, and three readings follow that no selector can act on.

Table 10: Where the 24821 task-seconds of the measurement run went. Outcomes are exclusive: *solved* means the deployed selector returned the truth, *in pool* means the truth was decoded but not selected, *miss* means it was never decoded, *empty* means the search returned nothing at all.

outcome	tasks	share of budget	mean seconds	mean final loss
miss	25	69.3%	688	0.0203
solved	11	23.4%	528	0.0076
in pool	3	5.8%	481	0.0228
empty	1	1.5%	367	0.0172

First, the search is not time-starved, it is capability-starved. Sixty-five percent of tasks never produce the correct grid under any of the sixteen views, and they consume seventy-one percent of the budget doing it. Second, final training loss does not separate the outcomes. Solved tasks span 0.00063 to 0.02531 and unsolved tasks span 0.00269 to 0.09104, overlapping across almost their whole range, so the natural progress signal is not informative enough to gate on, which is also the retrospective explanation for why the loss-cut scheduler rule of Section 4.1.1 did not help. Third, and most usable, solves land early. The eleven solved tasks finished at 316, 316, 346, 363, 456, 491, 528, 547, 581, 742 and 1118 seconds. The slowest solve of the entire run arrived at 1118 seconds, while the four tasks that ran past 1200 seconds consumed 26.3% of the budget and returned nothing at all.

A later experiment supplies the signal the second reading says is missing, and it arrived from an experiment aimed at something else. Asking whether a task's demonstrations underdetermine its rule, we trained leave-one-demonstration-out adapters per task and compared their candidate pools. As a repair the answer is a flat null: over 20 outputs the union of every subset adapter equals the full adapter exactly, so removing a demonstration does not move the adapter to a different reachable hypothesis and the 65% stays a property of the base model. But the *disagreement* between those subset pools separates the outcomes that training loss cannot. Mean pairwise pool Jaccard is **0.373** on outputs where the full adapter found the truth and **0.015** where it did not, against a same-configuration replicate baseline of 0.543. Small samples, seven and four outputs, so we report it as a lead rather than a policy. The direction is what matters: when a task's rule is determined, adapters trained on different subsets of its demonstrations agree about what to propose, and when it is not, they share almost nothing. That is a per-task confidence measure computed before the answer is known, which is precisely what a budget allocator needs and what the loss-cut rule lacked. Building the policy on it is future work, and its cost, one extra adapter per task, is the same reclaimed budget the cap frees.

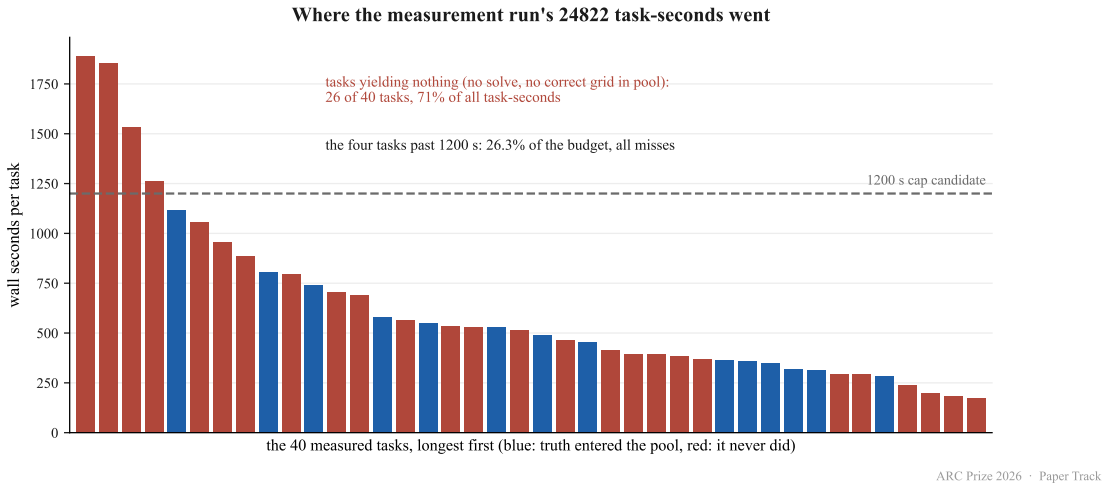


Figure 10: Where the measurement run's 24822 task-seconds went, one bar per task, longest first. Red bars are tasks whose pool never contained the correct grid; they take 26 of the 40 tasks and 71% of all task-seconds. The four tasks beyond the 1200-second line are all misses and consume 26.3% of the budget between them, which is the empirical basis for the cap analysis of this appendix.

The visiting-order question received the same treatment before any engineering was spent on it. Best-first enumeration, the anytime alternative to threshold-DFS that the general decoding literature offers and that no published ARC system uses, makes a testable claim: under truncation, expanding nodes in cumulative-likelihood order should reach the first correct grid in far fewer expansions than the depth-first order actually taken. We instrumented the deployed decoder to log every node expansion and every emission over sixteen stratified tasks, 30,480 batched forward passes and 217 completed candidates, twelve of the sixteen decodes budget-truncated, and replayed each recorded tree under both orders at equal information. Ten tasks put a correct grid in the tree and are the ones that can discriminate. Across them the median saving to the first truth is **2.9 percent** and no task is worse under best-first, but the distribution is the finding rather than the median: three tasks save nothing at all, four save between one and three percent, and the entire effect sits in three tasks saving 13, 26 and 60 percent.

The mechanism is visible in the logs and explains the shape. Nearly all negative log-likelihood accrues at a handful of branch points, the forced runs between them are order-invariant, and threshold-DFS sorts each node's children before descending, so it already traverses in nearly likelihood order. Best-first can only recover the cases where the tree's first subtree is both deep and wrong, which is a minority of tasks. A lever that pays on three tasks in sixteen, all of them already truncating, does not justify replacing the decoder.

We record this as a refutation of our own prediction rather than as a revision. An earlier draft of this section argued that anytime enumeration was the strongest unexplored lever available and that the published absence of best-first search for ARC was an opportunity. The measurement cost one logging run and says otherwise.

A cap at 1200 seconds therefore costs zero measured solves and frees about 7800 seconds per worker against a 42480-second wall, roughly eighteen percent. Decide-early, heavy-tailed, and with no usable

progress signal is precisely the regime in which restart theory says a hard cutoff plus a fresh attempt dominates waiting, and Appendix F already derives the fixed-cutoff objective for exactly this queue. We derived the mathematics and never applied the policy, which is the same failure this paper records elsewhere in a different costume.

We then applied it, and the timing model did not survive contact with the hidden set. The cap had never once been tested cleanly: v7.3 declared 1200 and ran 1800 because the build script's replacement targeted a form of the guard that exists only inside a commented-out cell, and v7.4 really ran 1200 but also fired 24 unbudgeted greedy fills, so two factors moved and its 29.44 is void rather than adverse. The clean test changes one live literal in the replicate notebook, the artifact whose code cells hash identical to the runs that scored 30.56 and 32.64, with the config banner's default moved to match so it cannot again print a cap the code does not run. It scored **29.31**.

That is 1.33 points below the same-configuration mean, 1.13 standard deviations, and below all but one member of the eight-run family. By this paper's own rule a difference of under two points is not a result, and we hold to the rule: the cap is *not* shown to be harmful. What it is not is supported. The prediction, recorded before the run and derived from our own timing log, was neutral to positive on the reasoning that the four tasks past 1200 seconds were all misses consuming 26.3% of the budget while the slowest solve arrived at 1118 seconds. The observation is the lowest score any variant of this configuration has produced. A draw this low or lower has probability 0.05 under the measured family, so it is weak evidence against the prediction and no evidence for it.

The instructive part is why a well-grounded timing argument can fail. Our timing log was collected on one card at a per-task granularity, and it says nothing about how a 240-task queue interacts with four workers and a wall. Capping does not only remove doomed tasks, it changes which tasks land on which worker and where each worker's tail falls, and the same session-speed sensitivity that gives this pipeline a one-point standard deviation acts on that rearrangement too. A single submission cannot separate those. Settling it needs the paired design this section spent its length arguing for. At one submission per day and the measured $\sigma = 1.34$, detecting an effect of the size this proposal could plausibly have needs roughly six runs and therefore six days for a single comparison, and we record the cost rather than paying it. That arithmetic is the reason this appendix ends in a cost rather than a number, and it generalises: at this noise level the contest's own submission quota is the binding constraint on how many hypotheses a season can test, which is perhaps eight.

The proposal is to cap hard and spend the reclaimed time on a second independent adapter for tasks that missed, unioning the candidate pools. Seven thousand eight hundred seconds per worker buys roughly fifteen restarts each, sixty across the node, against about one hundred and fifty misses on a 240-task set. Whether that is worth anything depends entirely on one quantity we do not have: whether a second independent run fails the same tasks.

Here the pipeline turns out to forbid its own experiment. Every stochastic choice in the deployed solver is seeded by a constant: the adapter initialiser at 42, the trainer at 42, the training augmentation draw at 1, and the evaluation augmentation draw at 2. Rerunning a task therefore reproduces the same adapter bit for bit, and a restart in the naive sense returns the identical pool. The lever exists only if the training augmentation seed is varied, which changes which sixteen of the dihedral-and-colour permutation space

the adapter ever sees, and that is a genuinely different training set for the same task. A determinism that reads as good hygiene is, for this purpose, the thing standing in the way.

We built the probe that answers it, and three details of its design are load-bearing. It trains the deployed configuration on the same task under different augmentation seeds and records, per test output, whether the truth is present in the decoded pool, so the comparison is union against single. Tasks are sampled stratified by their Kaggle outcome rather than taken in order, because of the first twenty tasks only four ever had the truth in the pool and fifteen are outright misses that cannot discriminate between seeds while still inflating the denominator; rates are reweighted back to the slice. The scope of any answer is narrower than the question sounds: the augmentation always applies all eight dihedral views and the seed selects only the colour permutation and example order, and we measured the resulting training sets to be almost disjoint as text, so a null licenses "colour permutation restarts do not help" and not "restarts do not help".

The third detail is a negative control, and it is not optional. The search admits a continuation on a hard threshold over half-precision logits, so a rounding difference at a borderline token prunes or admits an entire subtree and two runs of the *same* seed need not agree. We measured that noise directly: against a cache-free reference the decoder's scores agree only to about 2×10^{-3}, which is the same order as the 0.0043-nat margin scale Section 5.7 measures for the selector itself. Without subtracting a same-seed replicate, "union exceeds single" would look positive whether or not seeds decorrelate, so the reported quantity is cross-seed disagreement minus replicate disagreement.

The answer is no, and the negative control is what makes that statement worth anything. Over 65 cells covering 19 tasks and 30 test outputs, the three seeds place the truth in the pool for 11, 17 and 15 outputs, and their union is 17. The union equals the best single seed exactly: seed two alone finds everything all three find together, and this held at every sample size we inspected as the run proceeded. Reweighted to the slice that is a 4.4-point apparent gain over the mean single seed, and reporting that number would have been the error the control exists to prevent. Rerunning the *same* seed disagrees with itself on 12.5% of outputs; different seeds disagree on 13.3%. The net is $+0.008$, which is indistinguishable from zero, so what the naive contrast measures is the decoder's own numerical nondeterminism and not any property of the training draw.

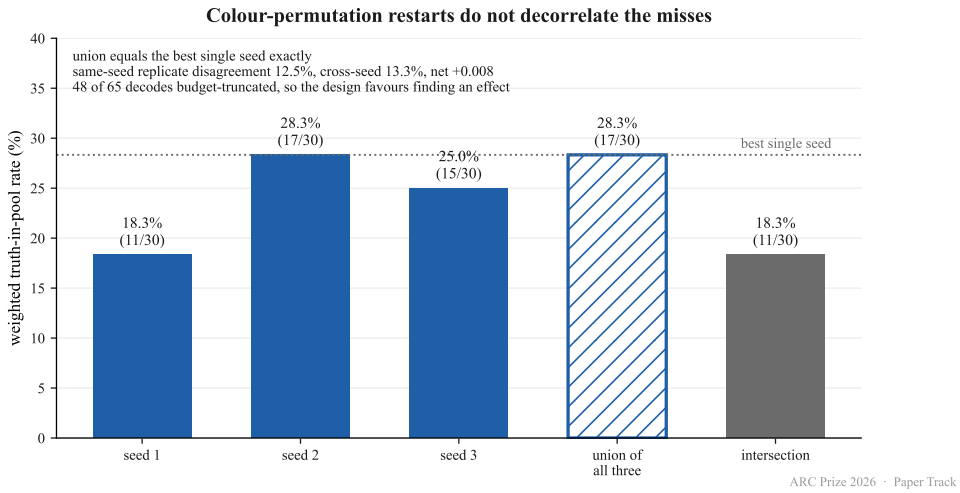


Figure 11: The restart probe's answer. Per-seed weighted truth-in-pool rates over 65 cells, 19 tasks and 30 test outputs, with the union of all three seeds equal to the best single seed exactly. The apparent 4.4-point union-over-mean gain is fully accounted for by the decoder's own run-to-run nondeterminism, measured by the same-seed replicate control at 12.5% disagreement against 13.3% cross-seed.

Two things make the null more credible than a null usually is. The bias in this design runs toward the hypothesis rather than away: 48 of 65 decodes were cut at their budget, so the probe is uniformly weaker than deployment, and decorrelation generically rises as success probability falls toward the noise floor. We failed to find an effect under conditions that would exaggerate one. And the sample was stratified precisely so the cells would land where seeds could differ, yet every hit lies in the reachable stratum, 17 of 21 outputs, with nothing at all from the nine outputs drawn from tasks the deployed run missed. The tasks a restart would need to rescue are not close to being rescued.

What this licenses is narrower than it sounds and we hold to the narrower claim. The augmentation always applies all eight dihedral views, so the seed re-rolls the colour permutation and the example order, and the finding is that re-rolling those does not change which tasks the adapter can reach. It does not establish that no restart of any kind helps. What it does establish is that the cheap version, the one the reclaimed cap could actually have paid for, does not, so the scheduling lever of this section is closed and the 65 percent of tasks that never produce the truth remain a property of the model rather than of the budget. Fidelity supports reading it that way: across the 19 shared tasks the probe's mean training loss tracks the deployed run's at a median ratio of 0.93, and the two agree on outcome for 15 of 19.

Building the probe also produced a result about our own instrument that we report because it nearly cost us the experiment. On the reproduction stack the model's tokenizer cannot represent the prompt format the model was trained on. It is a word-level vocabulary of sixteen tokens with no merges, and the entries for the two turn markers are unreachable: encoding the word for either returns nothing, and the user and assistant preambles encode identically. The completion-only loss mask keys on exactly those two ids to find the assistant spans, so every label was masked, the cross-entropy ran over zero targets and returned NaN, and the gradient norm was zero at every rank and optimizer we tried. Nothing raised. Left undetected it would have trained every adapter to NaN, decoded an empty pool for every task, and

reported that seeds do not decorrelate, which is both the answer that ends the investigation and the answer we had no independent reason to doubt. The deployed runs do not have this defect, since the measurement run's recorded losses are finite and converged, which is what pins the intended tokenization and let us reconstruct it exactly.

Porting the deployed solver to a current software stack surfaced a portability result worth more than the probe, and we are careful below to say exactly how far it is established. The depth-first search recurses with a single key-value cache object and calls the model repeatedly inside a loop, passing that same cache each time. That is correct only if a forward pass leaves its input cache untouched, which was true of the legacy tuple interface the pipeline was written against, where the returned cache was a new object. Current library versions replaced it with cache objects that append in place. On those versions the second branch of every search node would continue from an already-extended cache, so positions advance past what the cache holds and the search explores from the wrong prefix. Restoring the original behaviour needs only a truncation back to the branch point before each call, after which the algorithm is unchanged, and that is what our port does.

The repair is now verified, though not in the way the first draft of this section claimed. We tested it by enumerating the search tree with the pruning threshold set so loose that nothing is ever discarded, under which the cached decoder and a cache-free reference that recomputes from scratch must agree exactly. They do: identical candidate sets, with scores matching to the numerical tolerance quoted above. The truncation also agrees with an independent implementation that slices the cache tensors directly and never calls the library's own truncation method. What we have *not* observed is the unpatched decoder failing on the deployed stack, since that stack pins the older interface, so the consequence of the dependency remains an inference from the interface change rather than a reproduction of the fault. We are explicit about that split because the alternative is the mistake of Section 4.1.1, where a plausible mechanism was fitted to an effect nobody had confirmed. What is solid is the dependency itself: the pipeline every entry in the thirty-point cluster is running relies on a deprecated interface contract, that reliance is invisible in the notebook, it produces no warning, and it is held in place only by the competition image's pinned versions. If that image moves, the failure is not a crash in the general case but a search proceeding from a corrupted prefix, which would surface as a quiet drop in solve rate that no log would explain. A dependency tree really can be worth points, which Section 4.1.1 claimed on bad evidence and withdrew. This is what a defensible version looks like, and it is offered as a specific mechanism with a stated failure mode, still awaiting its own control.

5.10 Perceptual pre-fitting on the test input

Every result above spends its adapter budget on the demonstration pairs. The test input grid is never a loss target, only a decode-time prompt, which means the largest single piece of task-specific information we hold is discarded before the first gradient step. Section 2 records that this is inherited from the ARChitects rather than chosen by us. Three published results suggest it might be expensive. CompressARC [18] reaches roughly 20% on ARC-AGI-1 evaluation by gradient-fitting a 76K-parameter network per task to compress that task's own grids, the test input included and only the answer withheld, with no pretraining at all, which is a floor on how much signal per-task grid structure carries by itself. The LongT5 system [7]

gains 2.08 points from UL2 denoising of grid tokens, though offline and on demonstration grids. And the self-supervised test-time-training line [16, 17] is built entirely on adapting to the unlabeled test instance.

So we ran the composition: a masked-cell denoising objective on this task's test input, expressed inside the deployed prompt format so the existing collator supervises it unmodified, for a fixed number of steps before the ordinary demonstration objective. Corrupted grid in the user turn, clean grid in the assistant turn, cells dropped at 0.25 to a colour drawn from the task's own palette complement so the model is never asked to invent a colour the task does not use.

The third arm is the reason the experiment is worth running at all. A two-arm design cannot separate instance-specific grid fluency from the brute fact that the treated arm took more gradient steps than the control, and an unattributable positive is how three of this project's retracted explanations began. So the control arm denoises *a different task's* test input for the same number of steps on the same volume of data. If pre-fitting gains and the control does not, the effect follows the grid. If both gain, the honest finding is that more steps help and the interesting claim is dead.

We record the prior-art position before the outcome, because it does not depend on the outcome. The precise composition appears to be unclaimed, but the fair characterisation is adjacent-negative rather than unexplored. Akyurek et al. [15] defined the neighbouring objective, attributed it explicitly to the Sun et al. lineage, ablated it, and dropped it when it lost by 1.9 points; their loss targets were demonstration inputs of synthetic leave-one-out tasks rather than the real test input, and the ablation was on BIG-Bench Hard rather than ARC, but a reviewer will reach for it first and should. CompressARC establishes that fitting to the test input at test time works, as an entire from-scratch solver rather than as a warm-up. The two adjacent ARC-side measurements point opposite ways, which is the only reason the question is still live.

We stated a prediction in advance so that it could be scored: a null, because the demonstration inputs already carry the palette, the object vocabulary and the symmetry group of the test input by construction, so the marginal information in $x_\star$ is small, and because the perceptual account of ARC failure that motivates the intervention is weakest exactly in our regime, the 80% perception figure resting on vision models reading rendered images rather than on text-grid models with literal cell access.

The prediction was correct, and the null is unusually clean. Forty pre-fit steps per cell over eight stratified tasks, 900 seconds of decode each, 24 cells: the truth is in the pool for **7** of 14 outputs under the deployed configuration, **7** under pre-fitting on the task's own test input, and **6** under the control that denoises a different task's grid for the same number of steps on the same volume of data. Not one output changed status. Zero gained, zero lost, so the intervention did not even trade coverage, which is the signature of a change the downstream objective absorbs entirely rather than one that perturbs it.

The denoising itself worked. Each pre-fit stage supervised 870 label positions per sample across sixteen corrupted views and ran in about 33 seconds, on grids from 10×20 to 30×30. The adapter learned to reconstruct the instance's grid and that competence transferred nothing to the transformation objective. Read against the prior art, this places an ARC-side companion beside Akyurek et al.'s BIG-Bench Hard ablation of the neighbouring loss: putting a generative objective on input grids inside test-time training was measured to lose there, and measured to do nothing here, on the one variant they did not run, the real unlabeled test input under a masked-cell rather than autoregressive form. CompressARC's per-task grid signal is therefore either already supplied by pretraining or not reachable by this route on a model of this

kind. We report it as a kill, and note that the control arm is what makes it one: without the third arm, "7 against 7" would have been an uninterpretable wash rather than a measurement.

5.11 The decode batch width is a memory cliff, and the deployed setting is over it

The deployed decoder splits the sixteen evaluation views into batches of four and runs one depth-first search per batch. Nothing in the pipeline documents why the width is four, and on this card it is the single most expensive unexamined constant in the system.

Single-token decoding is bounded by streaming the weights out of memory rather than by arithmetic, so doubling the batch should be close to free. It is, right up until it is catastrophic. Timing each width in its own process, on prefixes drawn from real evaluation tasks:

Table 11: Per-step decode cost against batch width and prompt length, one process per measurement, warmed up before timing. Peak is CUDA reserved memory on a 16 GB card. The cost is not width, it is crossing a memory threshold between 12.7 and 14.2 GB.

prefix tokens	width	per-step ms	peak reserved	
1398	2	129.0	9.73 GB	
1398	4	**127.6**	11.47 GB	free
1642	2	131.3	10.27 GB	
1642	4	**130.2**	12.68 GB	free
1994	2	129.4	11.05 GB	
1994	4	**3253.4**	14.23 GB	25×
2730	2	126.9	13.54 GB	
2730	4	**4104.5**	19.22 GB	32×

Two readings follow, and the second is the one that matters.

First, the mechanism is memory and not computation. At 1398 and 1642 tokens the fourth lane costs nothing measurable, which is the bandwidth account behaving exactly as predicted. At 1994 and 2730 the same width costs twenty-five to thirty-two times more per step. Nothing about the arithmetic changed between those rows; what changed is that reserved memory crossed somewhere between 12.7 and 14.2 GB on a 16 GB card, after which the platform pages to host memory rather than failing.

Second, **the majority of the evaluation set is on the wrong side of that threshold**. Over 172 test outputs the prompt length has median 2198 tokens, first quartile 1610 and third quartile 3018. **57% of outputs exceed the shortest prefix at which width four collapsed, and 74% exceed the longest at which it was still free.** So a single-card deployment of this configuration spends most of its decode budget running one to two orders of magnitude below the speed the same hardware delivers at half the width, and it does so on precisely the tasks with the most demonstration content.

This is the sharpest instance we have of the ceiling in the title, and it is not the one we expected to find. The wall is usually described as capacity, whether the model and its adapter fit. Here everything fits, the configuration runs to completion, and the cost appears entirely as throughput. Since the search is

deadline-bounded in three places, a decode running twenty-five times slower does not merely take longer, it truncates earlier and the candidates it would have found never exist. Throughput is coverage, and this constant costs most of it.

The remedy is a two-line policy rather than a method change: choose the batch width from the prompt length so reserved memory stays under the cliff, which for this model means width four below roughly 1650 tokens and width two above it. It preserves the same thirty-two views and changes no score by construction, since lanes are independent and only the grouping differs.

Three honest limits on the claim, and the third is the one that bounds it hardest.

The platform matters: this card runs Windows, where exceeding device memory pages silently to host rather than raising, so on Linux the same configuration would be expected to fail outright rather than degrade.

The contest node's L4s carry 24 GB each, and extrapolating our peak-memory curve puts width four's cliff near 3200 prompt tokens there rather than 1650, which is roughly the top fifth of our prompt distribution instead of most of it.

And the inference stack is not the same one. Our probes run plain extttransformers with extttpeft, because the deployed path uses Unsloth and Unsloth requires Triton, which has no native-Windows backend. The submitted notebook calls extttFastLanguageModel.for_inference, whose fused kernels and attention implementation are precisely what decides whether activation memory grows linearly or quadratically in prompt length. So the cliff we measure may be a property of our stack rather than of the pipeline, we cannot test the alternative on this hardware, and we make no claim about the deployed runs. What we do claim is narrower and still useful: the batch width is an undocumented constant, on a 16 GB Windows card with a stock stack it is the difference between 130 and 4100 milliseconds a step for most of the evaluation set, and anyone porting this pipeline to a single consumer card without Unsloth inherits that silently.

5.12 The complete ledger: what is closed, what is open, and why

This section exists because the scattered negatives above are more useful collected than distributed, and because the most valuable thing a compute-poor entry can hand the next one is a map of where not to spend. Everything here is measured on this pipeline; none of it is argued.

The headline count. Forty levers were measured. None moved the hidden score. That is the finding, not an apology for one: a system whose remaining knobs are all worth less than its own measurement noise is at a ceiling, and the useful thing to publish is which knobs and why. The noise makes that statement sharper than it looks. At sd 1.34 the leaderboard cannot resolve anything below roughly four points without three runs per arm, and the largest coverage change any of these forty produced on a held-out slice is one output in fourteen. So "none moved the hidden score" is partly a claim about the levers and partly a claim about the instrument, and separating the two honestly is the point of this section.

Two of the forty produced a genuine positive on their own terms without moving coverage — task retrieval by demonstration embedding, and warm-starting an adapter from a same-rule neighbour — and both

sit *upstream* of the candidate pool. Every lever that produced nothing at all sits downstream of it, on candidates the search has already generated or on how time is divided between them. We did not notice that pattern until the ledger was assembled, and it is the single thing we would most like a reader to take from this section: the theorem bounds the downstream half, the budget table says the upstream half is starved, and we spent most of a season optimising the bounded one.

A confounded lever, isolated and closed. One entry in that count deserves its own note, because for most of the season it was not really measured. The greedy fallback, which decodes tasks the solver left without an answer, was first submitted inside v7.3 alongside a loss cut and a 1200-second per-task cap, scoring 28.61. Attributing that to any one of the three would have been guesswork, and the kernel then became unreachable, its output no longer held by the host, so the obvious re-test was not available either. It was rebuilt as v7.2's pipeline with the fallback as the only change and drawn three times: **30.56, 29.72, 29.44, mean 29.91**. Against the twelve-draw same-version series (29.90, sd 1.34) that is $+0.01$, a hundredth of a task; the difference has standard error 0.88 and $t = 0.02$. **The mechanism does nothing on its own**, and the three draws straddle the control mean. The general point is about experimental hygiene rather than this knob: a lever bundled with two others and submitted once is not a measured lever, and the cost of finding that out was three days of a one-a-day submission quota spent re-establishing something a single clean run would have given us the first time.

A second pattern is about cost rather than direction. **Eleven of the closures required no GPU at all**, and six of those eleven came from a single afternoon spent deliberately pricing ideas instead of building them. The selection bound is a proof. Sub-grid cropping was closed by measuring that 5.8% of cells in ARC-AGI-2 grids are removable margin. Run-length serialization was closed by observing that a 16-token vocabulary has no spare token for a run marker, and its lossless variant by measuring that uniform row deletion truncates the true answer on 6.7% of tasks. The tail-dropping account of our own score variance was refuted by simulating the task queue against the wall. Later, nine proposed candidate filters were replayed against a dumped pool, the whole family of serialization-compression ideas was closed by measuring what its saved tokens could ever buy, and canonical palette pre-sorting was closed twice over, once by a two-line proof and once by a measurement (below). Each of those took minutes and pre-empted an overnight run. Pricing an idea before building it is the cheapest discipline available in a compute-poor project and we adopted it far too late.

One number organises the map. Sixty-five percent of tasks never produce the correct grid under *any* of the sixteen augmented views, and they consume seventy-one percent of the budget doing it. When the truth does reach the candidate pool, the deployed selector places it in the top two essentially always, 18 of 18 on freshly generated tasks. The system is therefore **generation-starved, not selection-starved or time-starved**, and every closed lever below is closed because it operates downstream of generation on a pool that either already contains the answer or never will.

Table 12: Every lever this project measured and closed, with the evidence that closed it. Hidden-set numbers are quoted against the twelve-run same-version series (mean 29.90, sd 1.34); no hidden-set delta we recorded clears *1.5* standard deviations, which is why the offline column carries the weight and why the offline evidence was collected in the first place.

lever	where	what closed it
Selection, downstream of a fixed pool		
selector reweighting	offline	leave-one-task-out refit scores below the deployed selector
cell-wise voting	offline	0 of 41 missing truths reconstructed; pool errors are correlated
second-model rescoring	offline	oracle gap 0 of 18 on fresh tasks; nothing to fuse
any belief-only selector	proof	bounded by top-two posterior mass (Appendix F)
nine task-mined candidate filters	offline	7 of 9 false-kill; the other 2 kill *nothing*, ever
Search, over a fixed adapter		
best-first enumeration	offline	median 2.9% saving over 16 logged trees, 30,480 passes
augmentation-seed restarts	offline	union of three seeds equals the best single seed exactly
invariant mining as a filter	offline	0 recovered, kills 11 of 19 truths
per-task cap at 1200s	hidden	29.31, inside noise; prediction unsupported
depth-decayed admission threshold	offline	backwards: truths emit deeper *and* cheaper than wrongs
raising the probability floor	offline	at fixed wall clock, 6 of 14 outputs falls to 5
greedy fallback for empty pools	hidden	isolated in v7.6: 30.56, 29.72, 29.44, mean 29.91 against 29.90
DFS cache crop	both	provably correct, 27% more candidates offline; hidden 29.72, 27.36, 29.03, mean 28.70
decode budget, 600 to 300 to 150 s	offline	truth in pool 6, 4, 1 of 10; the throughput bought back cancels it
truncation-fair view scheduling	hidden	29.44, 27.78 of a pre-registered three; below the mean, unresolved
Adaptation		
adapter rank and depth	offline	inverted U; the shipped configuration sits on its peak
pre-fitting on the test input	offline	7 / 7 / 6 across base, treated, control; nothing moved
leave-one-demo subset adapters	offline	+0 coverage over 20 outputs
continued fine-tuning of the base	offline	honest negative; bimodality unmoved
synthetic rule count, 100 against 402	offline	tied 12 of 12 at an equal example budget
synthetic rule kind, ARC-1 against ARC-2 generators	offline	0 better, 1 worse, 11 of 12 tied at matched pool depth
training duration, 8,000 to 96,000 steps	offline	validation optimum at 16,000; tied 12 of 12 at every checkpoint
refinement loops	offline	no partial-credit middle for a loop to grip
unchanged-cell loss masking	offline	works and loses: see below, the most informative negative here
Cross-task structure, a surface an earlier draft called open		
retrieval of similar tasks	offline	*works*: AUC 0.884 same-rule against 0.684 surface
warm start from a related adapter	offline	final loss 30% lower on 7 of 8; coverage 6 vs 6
The augmentation space, the other surface called open		
masked-output completion	offline	8 vs 8 on 12 matched tasks
sub-grid bounding-box crops	offline	3 vs 3 on 5 matched tasks; 5.8% of cells are margin
rule inversion, shape-matched pairs	offline	4 vs 4 on 7 matched tasks
background run-length serializa-	offline	unimplementable: 16-token vocabulary, no spare marker

Seven architectural constraints, and which of them we actually probed. The closures above are consequences of a small number of structural facts about the system, and separating the two makes clear which walls are load-bearing.

1. **The base model has a sixteen-token vocabulary.** It emits grid tokens and control ids and nothing else. Chain-of-thought, self-critique, natural-language hypotheses, verifier feedback in text and program emission are all structurally unavailable to it, which removes most of the contemporary reasoning literature from consideration in one stroke. This is the deepest constraint in the system. An earlier draft said we never attacked it; we have now attacked it exactly once, with the cheapest in-vocabulary syntax we could design, and the result sets the price. Prefixing each output grid with its own dimensions as four zero-padded digits — no new token, five extra tokens per grid — took coverage from 4 of 10 outputs to **0**, and 14 of the 19 grids the model produced under it contradicted the header it had just written. After per-task training the model cannot hold its own stated intent for the length of one grid. The direction is not closed, because nothing else has touched it, but any future proposal here has to explain how it avoids that failure.
2. **Gradients flow only on output grids.** Inherited from the ARChitects as an explicit design choice. Probed twice: pre-fitting on the test input returned an exact null, and reshaping the loss *within* the output grid, by masking the cells the rule leaves unchanged, made the transformed cells 4.6 times more accurate and the answer worse (Table 13).
3. **Two hundred and forty independent adaptations that share nothing.** A fresh adapter per task, discarded after use. Nothing learned on one task reaches another. Probed: retrieval of a same-rule neighbour works at AUC 0.884 and warm-starting from it lowers final loss 30% on 7 of 8 tasks, changing coverage by nothing.
4. **A narrow augmentation space.** Eight dihedral transforms crossed with colour permutation and example order. An earlier draft called this largely unprobed. It has since been probed to exhaustion: three auxiliary objectives, palette canonicalisation, column-major ordering, the compression family, and the translation family are all closed above, and the space turns out to be narrow for reasons rather than by oversight — most enrichments either duplicate a view the dihedral group already contains or inject false supervision on the majority of tasks.
5. **Every budget is wall-clock, not operations.** Three deadlines in seconds. This is why one immutable artifact scores 30.56, 32.64 and 28.47, and why throughput converts into coverage.
6. **The search admits on a hard threshold over half-precision logits.** Identical runs disagree on 12.5% of outputs and share only 0.543 of their pools.
7. **The base is bimodal per task.** It internalises a rule or concentrates on a wrong one, with no partial-credit middle. This is what kills refinement, verification-gated resampling and every method that assumes a gradient of partial correctness.

The two surfaces an earlier draft called open, and what happened to them. We named cross-task transfer and the augmentation space as the two places with a plausible mechanism, no measurement

against them, and no published work claiming them. Both have since been measured, and the way each closed is more informative than the closure.

Cross-task transfer works and does not pay. Task embeddings do recover rule identity: mean-pooled hidden states over a task's demonstrations separate same-rule from different-rule instances at AUC 0.884, against 0.684 for a deliberately weak surface descriptor of colour histogram, grid dimensions and density, with precision@1 of 0.65 against a chance rate of 0.035. Ground truth comes from RE-ARC, whose generators emit unlimited instances of one transformation, so slicing a pool gives tasks identical in rule and different in every surface respect. Retrieval is therefore not the obstacle. The obstacle is what retrieval would be for. Initialising a task's adapter from one trained on *a different instance of the same generator* lowers the final training loss by 30%, on 7 of 8 tasks, graded by relatedness (same-rule 0.00414, unrelated 0.00569, cold 0.00594) so it is genuine transfer and not a warm-start artefact — and it changes truth-in-pool not at all, 6 against 6. Same rule is the most favourable case any retrieval scheme could return. The adapter fits the demonstrations better and reaches no new hypothesis, which is what the budget table implies and this measures directly: training loss was already between 0.0006 and 0.09 on solved and unsolved tasks alike, so optimisation was never the binding constraint.

The augmentation space is not obviously enrichable either. Three auxiliary objectives, each sharing a fixed step budget with the demonstration objective so that a winner must beat the real training it displaced, all changed coverage by nothing: masked-output completion 8 against 8 on twelve matched tasks, bounding-box crops 3 against 3 on five, rule inversion 4 against 4 on seven. The two serialization-shrinking variants failed before reaching a GPU, one because the 16-token vocabulary has no spare token for a run marker, the other because lossless uniform deletion saves 1.0% of cells while truncating the true answer on 6.7% of tasks.

A later pass closed the rest of that surface without a GPU at all. *Canonical palette pre-sorting*, remapping colours by descending pixel count so the adapter does not have to absorb an arbitrary labelling, is a no-op by construction: the deployed augmentation already composes a permutation drawn uniformly from all 10! bijections, and a fixed bijection composed with a uniform one is uniform, so the training distribution does not move. Applied per demonstration, as originally proposed, it is worse than a no-op — the per-demonstration maps disagree about where a shared colour goes on 92% of tasks, on the half of tasks where the palette itself carries the rule. *Column-major output ordering* is the transpose view, which the dihedral group already contains, so every training set has always included it. *Toroidal wrap, zero-padded translation and blank row permutation* inject false supervision at scale: content touches a grid edge on 79% of grids and 93% of tasks, the cells a rule changes touch the border on 65% of shape-matched pairs, and the count of uniform rows or columns changes across 72% of them, so all three are editing the label on most of the set. And the entire compression family — run-length coding, coordinate interleaving, digit-position prefixes — rests on a motive that does not exist. Saved tokens can only change an answer through `cut_to_len`, which drops demonstrations from the front of an over-long prompt; at the deployed 8192-token context that costs **0 of 240 training views** and 6 of 344 evaluation views, 6 demonstrations out of 1046. The compression is real and there is nothing competing for what it frees.

The most informative negative in this paper: an idea that worked and lost anyway. Every proposal in the loss-shaping family shares one premise, that gradient spent reproducing cells the rule does not touch

is waste. The premise is quantitatively attractive. Among the 67.5% of ARC-AGI-2 evaluation tasks that preserve grid shape, **88% of a median output is a verbatim copy of its input**, so the collator's uniform cross-entropy spends most of its signal teaching the model to echo. Masking the loss on those positions is the sharpest version of the idea, and unlike the background-reweighting variants it assumes nothing about which colour is background — which matters, because colour 0 has a *median share of zero* in these grids and the modal colour occupies 66%, not the 80% those proposals assume.

We ran it with three arms at an identical step budget: the deployed objective, the masked objective, and a control masking an equal number of randomly chosen supervised positions. The third arm is what makes the result interpretable, since any masking reduces the token count and changes the effective learning rate.

Table 13: Unchanged-cell loss masking, eight tasks and fourteen outputs, identical step budget. The error columns are cell error rates for the top-ranked candidate, split by whether the true output copies that cell from the test input or changes it.

arm	truth in pool	candidates	error, copied cells	error, changed cells
deployed objective	**6** / 14	48	0.024	0.286
mask unchanged cells	3 / 14	39	**0.095**	**0.062**
mask an equal random set	**6** / 14	44	0.021	0.146

The intervention did exactly what it was designed to do. Redirecting the gradient off the copied cells cut the error on the cells the rule actually changes by a factor of **4.6**, from 0.286 to 0.062, which is the largest movement any lever in this paper produced on any quantity. And it lost, because it broke the other half: error on the cells that should have been copied rose fourfold, and when 88% of a grid is a copy, a model that transforms correctly and mis-copies one cell in ten submits a wrong grid. Coverage fell from 6 of 14 to 3, worse on three tasks and better on none.

The control is what turns this from a suggestive null into an explanation. Masking the same *number* of positions at random is indistinguishable from the deployed objective, tied on all eight tasks. So the loss is not caused by having less supervision and not by a smaller effective learning rate. It is caused specifically by removing *all* supervision on copying. A uniform twelve percent sample of every kind of cell is sufficient; zero copy-supervision is fatal. The premise underneath the whole family is therefore false in an interesting way: the gradient spent on unchanged cells is not waste, it is the load-bearing half, and the transformation the task is nominally about can be improved several-fold while the answer gets worse. We report the strength honestly — the coverage drop is three tasks of eight, which a sign test alone would not call, while the cell-level split is measured over thousands of cells and is not close.

One constructive result, and it began as a claim we made without measuring. Asked whether a compute-poor entrant could train a base model rather than inherit one, we answered that the blocker was data rather than GPUs: ARC-AGI-2's difficulty is compositional, its training set is about a thousand tasks, and procedural generators such as RE-ARC implement ARC-1 rules. Building generators for ARC-2-style composition, we said, was a larger project than a remaining season. That was an assertion, and this paper's own discipline says a question about how much structure exists is answered with a script.

It is answerable in an afternoon, and the answer is no. RE-ARC ships 400 *verifiers*, each a pure grid-to-grid function implementing one task's rule. Transformations compose, so $\text{verify}_B(\text{verify}_A(x))$ is a new deterministic rule and 400 verifiers give 159,600 ordered pairs. Consistency, the property that normally makes synthetic task generation hard, is free: both stages are functions, so every instance of the composition obeys the same rule by construction. What is not free is non-degeneracy, so we require that B runs on A's output, that B visibly acts, that A remains visible in the result, that the pair is not an identity in disguise, and that the output is within ARC's dimension and colour bounds.

A first pass applied each predicate as "holds on at least one instance" and kept 1,440 pairs. That number is wrong, and how we found out is the useful part: we rendered two surviving tasks as grids and looked at them. In one, the stage-by-stage trace showed the second stage was a *no-op on the test instance*, so the task was two-step on some examples and one-step on others. In the same task the shape relationship changed between demonstrations, because a combine-two-halves first stage was being fed an input that did not split evenly. Neither defect is visible in any aggregate we had computed, and both would have taught a model the wrong thing.

Tightening every predicate to hold on *every* instance, and adding a consistency requirement on the input-to-output shape relationship, leaves **402 of the 159,600 ordered pairs**, drawn from 30 distinct first stages and 28 second stages. Each can still be instantiated without limit. The count is therefore below ARC-AGI-2's training set rather than above it, and the compositional variety is narrow: thirty first operations crossed with twenty-eight second ones.

Two qualifications, in opposite directions. The shape-consistency filter is blunt, since real ARC rules do produce variable output shapes and "extract the largest object" is one, so 402 is a conservative floor and the true figure lies between it and 1,440. And the first-stage bottleneck is an engineering limit rather than a fundamental one: verifiers raise on each other's outputs because each was written for its own input distribution, so a normalisation pass between stages would likely recover a share of the 158,000 pairs that never ran at all.

We report this as a capability rather than a result, because we have not yet trained on it at scale and do not claim it helps. Two things would have to hold. The first, that compositions of two ARC-1 rules resemble ARC-2's compositional difficulty rather than merely stacking, is not established here. The second, that a corpus consistent by construction is also *learnable* — a composition is guaranteed well-defined but not guaranteed to admit a short description — we did measure, because a corpus that is consistent but arbitrary would teach a model noise rather than composition.

The test runs the deployed per-task pipeline on both, drawing single-rule and composed tasks from the same RE-ARC pools so that grid distribution is held fixed and only rule structure differs, with the reading fixed in advance: composed near single means the compositions add instances but not difficulty; composed near zero means they are arbitrary rather than hard, and training on them would be worse than not; composed clearly between is the case a corpus meant to teach composition should show. Over fourteen matched pairs, single-rule tasks put the truth in the pool **14 times out of 14** and composed tasks **5 out of 14**, with composed worse on nine pairs, tied on five and better on none, and mean final training loss of 0.0019 against 0.0176. That is the middle case: substantially harder than one rule and still solvable on better than a third of instances. The corpus is not noise.

What that costs is worth stating precisely, because it is the sharpest instance of this paper's own thesis. A pilot fine-tune of 8,000 of the 78,792 examples, rank 32 at a 2048-token context, runs at 11.7 seconds per optimizer step on the 16 GB card with 97.6% of its memory in use, which puts the full corpus at roughly **eleven days** of continuous training. Every lever in the ledger above changes the pipeline around a fixed model and is worth less than the measurement noise; the one lever that changes the model itself is precisely the one the memory constraint prices out. That is a cleaner statement of what 16 GB costs than any of the accuracy comparisons in this paper, and we arrived at it only after exhausting the alternatives.

What we can also say is that the reason we originally gave for not attempting any of this was wrong, and wrong in a way one script would have caught at any point in the preceding months.

What survives as advice is therefore narrower and, we think, more useful than a list of ideas. Every lever we have measured either operates downstream of generation, where the theorem bounds it, or improves the fit to the demonstrations, where there was no headroom to begin with. The demonstrations underdetermine the rule and the base model resolves that underdetermination wrongly on 65% of tasks. Nothing that reweights, re-searches, re-initialises or re-augments around that fact has moved it here. We would direct the next entrant at the resolution itself rather than at anything surrounding it.

6 Conclusion

The 2025 winning recipe survives a single 16 GB consumer GPU, but only after the inference engine is fixed. That one change is worth more than every other knob, and it is what makes per-task TTT tractable through an adapter-serving bridge. The honest result is a 12.79% no-TTT baseline and a 16.28% regularized-TTT blend on the public evaluation, 13.61% and then 18.19% on the hidden leaderboard as the per-task training deepened, with the gains bounded by adapter overfitting and by a VRAM wall on large grids that later proved to be partly a context-truncation artifact rather than a true limit. Against that single-card ceiling we set one measurement from above. A faithful reproduction of a public pipeline that one 16 GB card can hold but not serve in budget, run on the contest's own four-L4 node and repaired of two defects its community port had introduced, scores **29.90%** on the same hidden set, the mean of twelve runs of one immutable artifact. That result is not ours and not single-card, and its value here is that it prices the constraint at roughly twelve points instead of leaving it an open question.

The lessons that generalize are about measurement rather than about ARC, and the largest one is the error bar. Twelve executions of a single unchanged Kaggle kernel version span 4.17 points with a standard deviation of 1.34, which is a twenty-second of the score itself. Every build we called a regression, both public notebooks we ran as controls, and the configuration we called our best all sit inside 1.5 standard deviations of that mean. Separating two configurations at 80% power needs about four points and three runs each, so a contest allowing one submission a day permits perhaps eight properly powered comparisons in a season. We report a ledger of forty measured levers that moved nothing, and the honest reading of it is not that nothing helps but that a programme of single-submission A/B tests — which is what the field and this project both ran — was never capable of the discrimination it claimed. Beyond that: public-eval gains must be re-validated in the deployed condition, since every public-validated candidate lever we submitted regressed on the hidden set, and the one class that did transfer was repair rather than tuning. A

score displayed beside a public notebook is not a measurement of the code published there, and treating it as one cost us a week and three quantified explanations of a gap that a single control retired in one submission. A family of rules is a multiplicity, so a sweep of sixteen selectors is nearly certain to produce a winner by chance, and in our case the pool's dominance structure refuted the whole family before any of it ran. And an effect measured against a hard threshold over low-precision arithmetic needs a same-seed replicate before it means anything, since ours produced an apparent 4.4-point gain that was entirely the decoder disagreeing with itself. Each of those cost real compute to learn and none of them is specific to this benchmark.

What we could not move is the part that was never about scheduling or ranking. Across the levers we closed, the residual is that most tasks never produce the correct grid at all, which is a property of a 4B model rather than of the twelve-hour budget, and the honest next step is a stronger base rather than another pass over the same candidates. We can now price that step rather than gesture at it. The one lever available to us that changes the model rather than the pipeline around it — continued training on the compositional corpus this paper constructs — runs at 11.7 seconds per optimizer step on the 16 GB card, putting the full corpus at roughly eleven days of continuous training. Everything cheap is worth less than the noise, and the one thing that is not cheap is what the memory constraint prices out. That is the sharpest form of this paper's claim, and it is not an accuracy comparison. Those bounds belong to one small card, not to the task. The contest gives four L4 GPUs and 96 GB, where the offline frontier now leads near 66%, and the verifier, the TTT bridge, and the selection rule we built are the pieces we would carry onto that hardware, with the scaling case stated as a falsifiable hypothesis rather than a promise (Section 5.4). The 16 GB card is where the climb starts, not where it ends, and the number beneath it keeps moving. The full system is open source, the GGUF conversion recipe for the custom-vocabulary model, the AIRV pipeline, the per-task TTT bridge and sweep harness, and a development log recording every measurement and dead end behind these numbers.

A Reproducibility

Environment. A single NVIDIA RTX 5070 Ti (16 GB, Blackwell, $\approx$ 896 GB/s), native Windows 11. For inference, a prebuilt llama.cpp CUDA 12.4 `llama-server`, run on Blackwell through PTX JIT. For training, PyTorch with a LoRA via PEFT, bf16 base, gradient checkpointing. The two frameworks are never co-resident in VRAM (Section 3.4).

The two GGUF converter patches. The NVARC model has a 16-token *WordLevel* tokenizer, which trips the stock converter twice. (1) *Unknown tokenizer hash:* the loader does not recognize the tokenizer's hash and refuses to guess a pre-tokenizer. Its pre-tokenizer is in fact the `qwen2` regex, so we map the hash to `qwen2` explicitly. (2) *Empty merge list:* a WordLevel tokenizer has no byte-pair merges, but the loader requires a non-empty list, so we add two throwaway merges among the role tokens, which never fire on grid text. Both patches are in the repository's conversion script and are applied automatically by the Kaggle prep notebook.

Serving flags. `llama-server` is launched with full GPU offload (`-ngl 99`), a 16,384-token context, a single slot (`-np 1`), and the prompt cache disabled (`-no-cache-prompt`). Its own logs are redirected to

a file so they never flood the notebook. Decoding is greedy (temperature 0) with `n_predict` sized to the target grid.

Hyperparameters. AIRV uses the eight D4 augmentations and votes to two attempts. Per-task TTT (regularized): rank 16 to 32, dropout 0.05 to 0.1, 8 to 20 steps, AdamW, a fresh random augmentation per step, leave-one-out target, context capped at $\approx 1{,}600$ tokens, applied at adapter scale 1.25 at inference. Exact values per experiment are in the development log.

B Extended results

Per-scale accuracy (84 trainable tasks). Table 14 gives the full adapter-scale sweep behind Figures 7 and 8. Accuracy rises to a peak at scale 1.25 and then declines, the signature of an adapter applied too strongly.

Table 14: No-TTT vs. TTT accuracy on the 84 trainable tasks as a function of adapter scale.

adapter scale	no-TTT	0.50	0.70	0.85	1.00	**1.25**	1.40	1.50	1.75
accuracy (%)	12.8	12.0	12.8	15.2	16.8	**17.6**	16.0	16.8	16.8

Paired significance. On the 84 trainable tasks, TTT and no-TTT disagree on seven test outputs. TTT is correct on six that no-TTT misses and incorrect on one that no-TTT gets. An exact two-sided McNemar test on a $6/1$ split gives $p \approx 0.13$, so the lift is not individually significant. We report it as directional, corroborated by the configuration sweep ($+1$) and the representativeness check (no-TTT scores 12.8% on the subset versus 12.79% on the full set).

Colour permutation. Adding a random colour permutation to the augmentation set lifts the no-TTT baseline from 12.8% to 14.4% on the 84 tasks, but provides no further gain on top of the per-task adapter, which has already learned the task's colour mapping, so the two do not stack.

Deep test-time training and the training context. The configuration sweep in Table 5 conflated two variables. Its deeper cell ran 40 steps at rank 32 with no dropout and its shallow cell 20 steps at rank 16 with dropout, so more steps was only ever measured with the regularizer removed. Table 15 separates them on all 84 trainable tasks, trained at the 2000-token context the submitted system uses. Depth alone does nothing. Sixty steps at rank 16 with dropout ties the twenty-step baseline at 24 of 125. Adding capacity while keeping the regularizer on is what moves the number. Sixty steps at rank 32 reaches 27 of 125, the best trainable-task result we have measured, and it prefers the same 1.25 inference scale. The honest caveat is variance. Two seeds of the baseline configuration score 24 and 22, so a single output sits inside the noise, and the deep result rests on one seed. A second seed is the confirmation and its verdict is in the status below. The same sweep isolates the training context as a real knob. The baseline trained at 1600 tokens scores 16.0% against 19.2% at 2000, a three-point gap wider than the seed noise. This reconciles a measurement detail. The trainable-task figure in Section 4.3 and the blend it feeds were measured at the 1600-token context of the sweep harness, while every submission from v2 on trains at 2000, so those reported numbers describe a lighter configuration than the leaderboard one and are a

conservative floor for it.

Table 15: Deep-regularized TTT sweep on the 84 trainable tasks (125 test outputs). Dropout is 0.1 and the training context is 2000 tokens except where noted.

configuration	steps	rank	scale	context	correct/125 (%)
baseline (v2.1 config)	20	16	1.25	2000	24 (19.2)
baseline, seed 1	20	16	1.25	2000	22 (17.6)
deep	60	16	1.25	2000	24 (19.2)
deep	60	16	1.00	2000	23 (18.4)
deep + rank	60	32	1.25	2000	**27 (21.6)**
deep + rank	60	32	1.00	2000	25 (20.0)
deep + rank	60	64	1.25	2000	28 (22.4)
deep + rank	60	128	1.25	2000	24 (19.2)
baseline, short context	20	16	1.25	1600	20 (16.0)

Status. Confirmed. A second seed of the rank-32 deep configuration scores 26 of 125 and the first scored 27, while the two baseline seeds score 24 and 22, so the two configurations separate with no overlap and the roughly three-output lift is a real effect rather than a seed draw. This is the first configuration we have measured that beats the shipped operating point on the mechanism that works, per-task training itself, rather than on the candidate-and-voting side where five submissions regressed. It cleared it. Shipped as v6.1 with the deep-regularized adapter trained on all 232 tasks, it scored **18.19%** on the hidden set, a 4.58-point gain over v2.1's 13.61% and the first submission to beat the shipped operating point (Table 2). The transfer held because this lever improves the adapter rather than diluting the vote, so the local sweep lift predicted a real hidden gain rather than the public-to-hidden reversal that sank the candidate-side levers. An intermediate v6 scored 12.36% as the same recipe starved by a disk-exhaustion bug that left 73 tasks without an adapter, and fixing the scratch path recovered the lift. The rank dimension is now swept to its ceiling. At the same 60 steps, rank 64 scores 28 of 125 against rank 32's 27, one output and inside seed noise at double the adapter cost, and rank 128 falls to 24, an inverted U whose mechanism is the per-task adapter overfitting its tiny few-shot dataset once capacity outruns the signal. The shipped configuration sits at the peak of its own curve, so the depth-and-rank lever is closed at 18.19% and further climb on this pipeline must come from elsewhere.

That closure had a built artifact attached to it. A rank-128 submission had already been assembled from the v6 recipe on an early partial reading that higher rank helped the vote, and it was **retired without ever being submitted** once the full sweep placed rank 128 joint-last, below the rank 32 it forked from. Rank 64 leads rank 32 by a single output out of 125, which is inside seed noise at twice the adapter cost, so no rank on the curve justifies a submission. The build is archived in the repository rather than deleted, because the episode is one of the paper's arguments made prospectively rather than in hindsight. Every candidate-side lever we submitted on a partial reading regressed on the hidden set. This one failed its validation first and cost a rig afternoon instead of a submission slot.

C Engineering log of walls and dead ends

We record the failures explicitly, because on this hardware they are most of the work.

- **The decoder's batching rests on an undocumented invariant that generated data breaks.** The DFS entry point builds a single tensor from a batch of augmented prompts, so every view in a batch must tokenize to exactly the same length, and the batch membership is fixed by hard-coded index offsets over the sorted view keys. Nothing enforces the invariant and nothing checks it. It holds on all 120 ARC-AGI-2 public evaluation tasks we measured, because their prompts fit inside the context window and the sixteen views are then permutations of identical content. It fails on 4 of 24 RE-ARC tasks, whose longer demonstrations trip the length-trimmer into dropping a *different* example from each view, leaving batch members 506 tokens apart and raising before a single forward pass. Anyone reusing this decode loop on generated or otherwise longer tasks will hit it. The fix that preserves semantics is to split a ragged batch by length rather than to pad it, since the decoder passes explicit positions and padding would silently corrupt them.
- **Eager decode is overhead-bound, and the usual fix is unavailable.** Hugging Face eager decode runs at ≈ 13 tok/s, about 10% of the bandwidth roofline. `torch.compile` would close the gap, but it needs Triton, which has no native-Windows backend. The resolution was to change engines, not to optimize the eager path.
- **The server prompt cache destabilizes unique-prompt workloads.** Enabled by default, it reserves ≈ 680 MiB per unique prompt. Every ARC prompt is unique, so it never reuses and instead destabilizes sustained generation. Disabling it and running a single supervised stream fixed the stalls.
- **4-bit QLoRA is broken on Blackwell + Windows.** bitsandbytes fails to load on this platform, so the planned 4-bit continued fine-tune was infeasible. Because per-task TTT already runs in bf16 inside 16 GB, we use a bf16 LoRA throughout and drop the 4-bit path entirely.
- **PyTorch decode can turn quadratic after a training step.** Generation in the gradient framework is not only slow but intermittently degrades after an optimizer step. This is precisely why the serving bridge exists, since we never generate in PyTorch in the scored pipeline.
- **Hugging Face eager stalls offline validation, the same engine lesson again.** The rig validation scripts that drove generation through eager had no per-task timeout, so one big-grid task could hang a whole multi-hour run with no way to resume. Moving validation onto the same `llama-server` path used for scoring, with a hard per-request cap, a heartbeat file, and per-task checkpoints, removed the stalls and made the runs resumable. The engine was the bottleneck for validation just as for inference.
- **The VRAM wall on large grids was a default, twice over.** The largest-train-pair token length, not the test grid size, is what grows the training sequence. We first recorded the resulting exclusion as a hard limit, then found that forcing TTT onto those tasks had only ever been tested under a six-hundred-token truncation that admits one demonstration, and later that the framework was dispatching attention to a quadratic kernel because no flash or memory-efficient implementation exists for this GPU in this build. Forcing the cuDNN backend trains rank 256 at eight thousand tokens in 14.86 GB. The general lesson is to distrust a wall that was never probed one variable at a time.

- **Naive TTT ties the baseline.** The standard adapter configuration drives training loss to 0.000 and generalizes no better than no-TTT. Only regularization (fewer steps, lower rank, dropout) recovers a lift.

D Prompt and serialization format

The model's 16-token vocabulary is given in Table 16. A grid is serialized row by row, one token per cell (the token id equals the colour), rows separated by the newline token. Prompts are assembled at the id level as a ChatML few-shot sequence of train pairs followed by the test input, ending at the `ASSISTANT` turn for the model to complete. Because the role and special tokens have no textual spelling, the whole sequence is built from ids directly, never by tokenizing a string, which is what guarantees parity between training and inference.

Table 16: The 16-token grid vocabulary.

token ids	meaning
0 to 9	the ten ARC grid colours (id = colour)
10	newline / row separator
11	`USER` role marker
12	`ASSISTANT` role marker
13	`PAD`
14	`IM_START`
15	`IM_END`

E Data and model provenance

The base model is Qwen3-4B [5] (Apache-2.0), supervised-fine-tuned on ARC grids by NVARC [4]. Any derived weights we redistribute are released under Apache-2.0 with the required `NOTICE`. Evaluation and training tasks are the official ARC-AGI-2 corpus [2] (Apache-2.0). The fine-tune probe additionally uses RE-ARC [6] (MIT). Our own code and weights are released under MIT-0. We never train on the evaluation tasks. Full license texts and attribution are in the repository's `THIRD_PARTY_LICENSES.md` and `NOTICE`.

BibTeX.

```
@article{chollet2019measure,
  title={On the Measure of Intelligence},
  author={Chollet, Francois}, journal={arXiv:1911.01547}, year={2019}}
@techreport{arcagi2,
  title={ARC-AGI-2 Technical Report},
  institution={ARC Prize Foundation}, year={2025}}
@techreport{arcprize2025,
```

```
  title={ARC Prize 2025: Technical Report},
  institution={ARC Prize Foundation}, note={arXiv:2601.10904}, year={2026}}
@misc{qwen3, title={Qwen3 Technical Report}, author={Qwen Team}, year={2025}}
@misc{nvarc2025,
  title={NVARC: ARC Prize 2025 (ARC-AGI-2) 1st-place solution},
  author={{NVIDIA Kaggle Grandmasters}}, year={2025}}
@article{hodel2024rearc,
  title={Addressing the Abstraction and Reasoning Corpus via
         Procedural Example Generation},
  author={Hodel, Michael}, journal={arXiv:2404.07353}, year={2024}}
```

F The mathematics of the measurement

The competition's numbers are statistics over a small finite task set, and several of this work's wrong turns trace to reading them with more resolution than they possess. This appendix collects the formal content behind the rules the body applies, what a leaderboard point is, what a paired comparison can resolve, why a candidate pool cannot rank its own truth, and what the time budget actually optimizes. Everything here is elementary. The value is that each statement is attached to a measurement from the body, so the arithmetic and the project's history check each other.

F.1 The score and its quantum

Task i has n_i test outputs, the submission supplies two attempts per output, and a task contributes one credit split evenly across its outputs,

$$S \;=\; \sum_{i=1}^{N} \frac{1}{n_i} \Big| \big\{\, j \le n_i \;:\; y_{ij} \in \{a_{ij}^{(1)}, a_{ij}^{(2)}\} \big\} \Big|, \qquad \text{score} \;=\; \frac{100\,S}{N}. \tag{4}$$

On the hidden set $N = 240$, so one credit is $100/240 \approx 0.4167$ percentage points, the quantum the leaderboard moves in. The public-notebook clusters sit at 31.39, 31.81, 32.22 and 32.64, spaced 0.41 to 0.42 apart, which is that quantum observed directly, whole boards separated by single tasks. The measure also carries a quiet bias. A solved output on a two-output task earns half what it earns on a single-output task, so any method that helps preferentially on multi-output tasks is systematically underpaid by the metric.

F.2 Resolving power of a paired comparison

Two variants scored on the same tasks form a paired design, and the pairing removes the between-task difficulty variance entirely. Concordant tasks, solved by both or missed by both, carry no information about the difference. Writing b for tasks that flip miss to hit and c for the reverse, the null hypothesis of no difference makes the flips exchangeable, $b \sim \text{Binomial}(b+c, \frac{1}{2})$, and the exact two-sided McNemar test in the cleanest case, every discordant task falling the same way, gives

$$p \;=\; 2\cdot 2^{-(b+c)}, \qquad \text{so} \qquad p \le 0.05 \iff b+c \;\ge\; 1-\log_2 0.05 \;\approx\; 5.32. \tag{5}$$

Six unanimous flips are therefore the minimum resolvable effect at the five percent level, $p = 2/64 = 0.031$, while five give 0.0625 and fail. Every candidate-side lever this work measured moved one to three tasks. Such effects are not weakly supported, they are unresolvable by any single read of a board this size. The body's episodes are this bound observed. The regularized-TTT comparison of Section 4.3, six flips against one across seven discordant tasks, gives exactly $2 \cdot 8/128 = 0.125$, the $p \approx 0.13$ reported there. The selector switch shipped on three unanimous flips, $p = 0.25$ two-sided, and reversed on the full sample. And the accidental repeat run, two submissions of one build, differed by 0.41 points, one quantum, which is the empirical noise floor agreeing with the formal one. Figure 12 places the episodes on the bound.

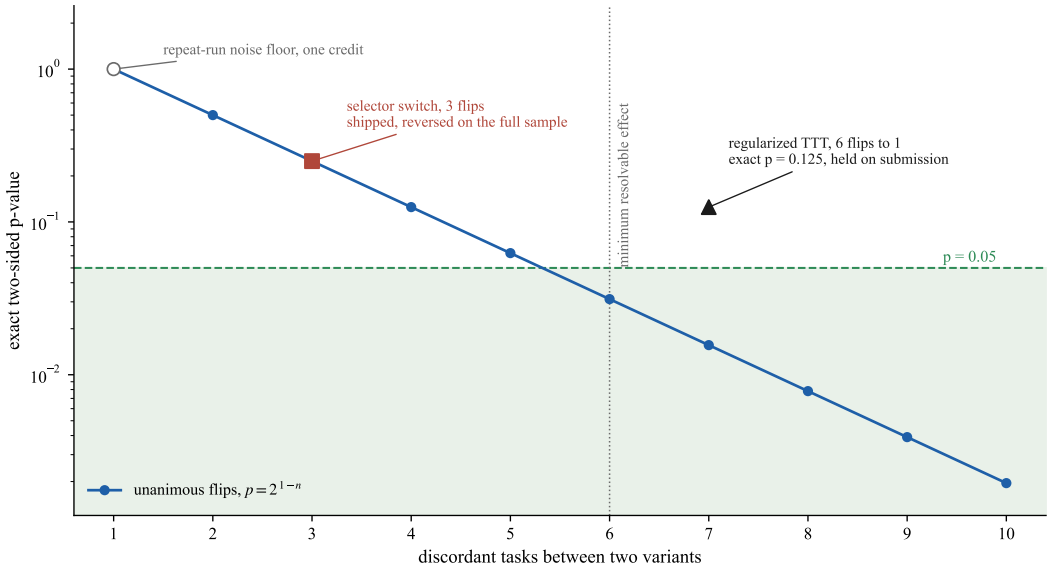


Figure 12: The resolving power of a paired comparison on this benchmark. The curve is the exact two-sided McNemar p-value when every discordant task falls one way, and six flips is the smallest unanimous effect that clears 0.05. The project's episodes sit at their exact p-values, the three-flip selector switch that shipped and reversed, the six-to-one regularized-TTT read that was held for a confirmation seed before submission, and the repeat-run noise floor of one credit.

F.3 A family of rules is a multiplicity

The selection sweep of Section 5.7 scored sixteen rules against one pool. If each rule independently had probability α of a spurious win, the chance that at least one wins is $1 - (1 - \alpha)^{16}$, which at $\alpha = 0.05$ is $1 - 0.95^{16} \approx 0.56$. The sixteen rules share the pool, so the tests are positively correlated and the true figure is smaller, but the reading survives the correction. The sweep's observed yield, exactly one rule gaining exactly one output, is what chance predicts from a search of that size over a null family, and that is why the winning rule was recorded rather than deployed.

F.4 Why the pool cannot rank its own truth

Model the pipeline as inducing, for each test output, a distribution q over grids, the composition of the adapted model, its sixteen augmented views, and the depth-first search. The pool is the sampled support of q, and every statistic the pool records for a distinct grid c is an estimate of a functional of q. The multiplicity $m(c)$ counts the augmented decodes that produced c. Each decode g carries a beam score $n_{\text{beam}}(g)$, the negative log-probability of its search path, and augmentation scores $n_{\text{aug},v}(g)$, per-view negative log-likelihoods of c under fresh augmentations. Both deployed selectors are monotone functions of these statistics,

$$\sigma_K(c) \;=\; m(c) \;-\; \frac{1}{m(c)} \sum_{g \in G_c} \overline{n}_{\text{aug}}(g), \qquad \sigma_P(c) \;=\; \sum_{g \in G_c} \big(3 - n_{\text{beam}}(g)\big) \;+\; \frac{1}{m(c)} \sum_{g \in G_c} \sum_{v} \big(3 - n_{\text{aug},v}(g)\big), \tag{6}$$

where G_c is the multiset of decodes yielding c and $\overline{n}_{\text{aug}}$ the per-decode mean. Call Σ_{mono} the class of selectors nondecreasing in m and nonincreasing in every recorded NLL. Both of Equation (6) are members, as are all sixteen swept rules.

Proposition. If a wrong grid g Pareto-dominates the true grid $y^\star$ in the recorded statistics, meaning $m(g) \geq m(y^\star)$ and every NLL statistic of g is at most the corresponding statistic of $y^\star$, with at least one comparison strict, then every selector in Σ_{mono} ranks g at or above $y^\star$. *Proof.* Immediate from monotonicity in each coordinate. □

The proposition is trivial. Its content is the premise, which is a measurable property of real pools, and it holds. On every one of the four leaked outputs of Table 8 the true grid is strictly Pareto-dominated inside its own pool, checked on multiplicity, mean augmentation NLL and best beam NLL, by one wrong candidate on two of the outputs, by two on a third, and by three on the fourth. No monotone selector can recover any of the four, so the sixteen-rule sweep was not sixteen independent failures, it was one proposition observed sixteen times. Figure 13 draws the premise.

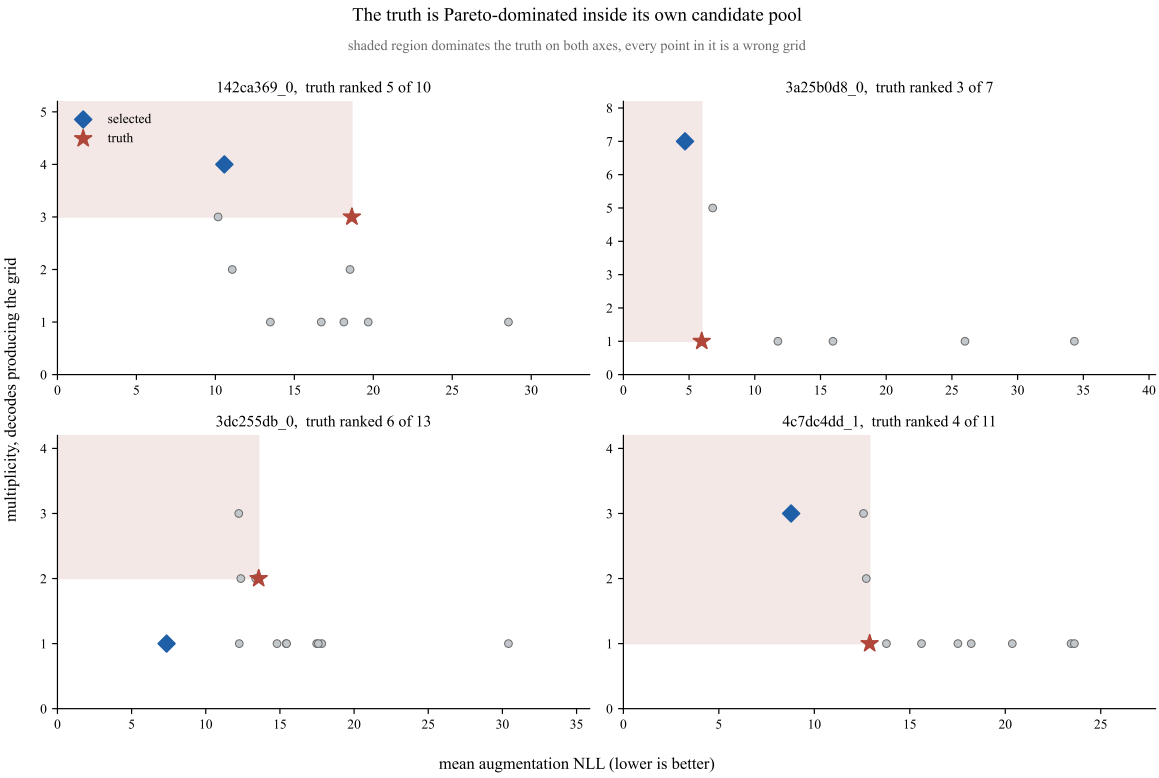


Figure 13: The four test outputs whose pool contains a truth the selector misses, every candidate plotted by mean augmentation NLL and multiplicity. The shaded rectangle dominates the truth on both plotted axes, and every point inside one is a wrong grid. The selected grid need not itself be the dominator, on `3dc255db_0` it wins through the likelihood term alone, but a dominator exists in every pool, and with the beam statistic included the domination is strict on all four, which is the premise of the proposition.

The dominance counts carry one more consequence, elementary enough to state in two sentences. A selector in Σ_{mono} that is strictly monotone in some coordinate where a dominator strictly leads places that dominator strictly above the truth, so a truth with k such dominators ranks at worst $k+1$, and two dominators already exclude it from any two-attempt selection. Both deployed selectors are strictly monotone in the coordinates that matter here, and the measured counts, one, one, three and two, hold simultaneously in the summary statistics and in the exact addends of Equation (6), checked directly on the pool. The consequence splits the leak. `3dc255db_0` and `4c7dc4dd_1`, together worth 1.5 credits, are unreachable by the entire monotone class in either attempt slot, while the two singly-dominated outputs are reachable only at rank two and only by rules constructed after seeing the answer, which the multiplicity argument of F.3 forbids. So the class ceiling on this pool lies between the attained 9.833 and 10.833 credits, against the present-in-pool 12.333, and sixty percent of the leak is closed by proof rather than by sweep.

Dominance is what the failure mode predicts rather than an accident. When the model internalizes a wrong rule, q concentrates on grids consistent with that rule, and the truth enters the pool only as a low-mass fluctuation, which is precisely low multiplicity and high NLL, that is, dominance. The same concentration defeats aggregation at cell level. Treating candidates as draws from q, if the marginal of q at a given

cell places most of its mass on a wrong colour, a per-cell majority vote converges to that wrong colour, amplifying the marginal rather than correcting it. The body measured exactly this, thirteen same-shape candidates fusing to a grid wrong in the same two cells the selected answer gets wrong.

The census of Figure 14 completes the pool's accounting and adds two measurements made after the theorem was proved. Counting Pareto dominators over the recorded summary statistics alone, only one leaked output is blocked for every monotone rule and the bound reads 18 of 60; the exact-addend analysis above is stricter, excluding a second output and giving 17. Either way two to three outputs of headroom exist inside the theorem, and the first measurement is that they are not learnable: refitting the selector weights over multiplicity, augmentation and beam terms with leave-one-task-out validation attains 14 against the deployed 15, one output *worse*, against a pre-registered bar of plus two. With four leaked outputs in sixty, weight tuning memorises them, so the headroom is real but unreachable from data of this size, and the practical ceiling is the deployed selector's. The second measurement is the shape of what the pool misses. Of the 41 truth-absent outputs, one sits a single cell from a pool candidate and six more within three cells, but the pool's errors are correlated through the model's beliefs, weighted cell-wise voting reconstructs none of the 41 under any of three weightings, so the near-misses are not recoverable by aggregation, and the six outputs where no candidate even has the right shape are beyond every selection-side idea entirely.

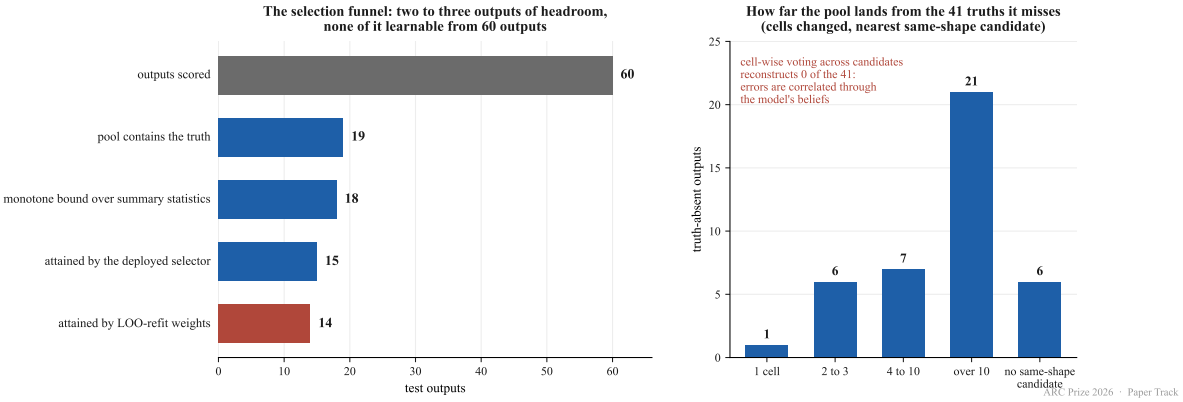


Figure 14: The pool census. Left, the selection funnel on the 60 deployed-condition outputs: the truth is present for 19, a monotone selector over the recorded summary statistics can reach at most 18 (17 under the stricter exact-addend counting above), the deployed selector reaches 15, and a leave-one-task-out refit of the selector weights reaches 14, demonstrating the gap is not learnable at this sample size. Right, the distance from each of the 41 missing truths to the nearest same-shape pool candidate. Cell-wise voting reconstructs none of them.

An escape therefore needs information not derived from q. The round-trip score of Section 5.7 has the right form, injecting a candidate as a synthetic demonstration and measuring the mean change in per-token negative log-likelihood of each held-out real demonstration,

$$\Delta(c) \;=\; \frac{1}{|D|}\sum_{i=1}^{|D|}\Big[\mathrm{NLL}\big(y_i \,\big|\, D_{-i}\cup\{(x_\star,c)\}\big) \;-\; \mathrm{NLL}\big(y_i \,\big|\, D_{-i}\big)\Big], \tag{7}$$

with D the demonstration pairs and $x_\star$ the test input. It failed in an instructive direction. The measured Δ tracks the description length of c rather than its truth, a simple low-entropy grid perturbs the conditional

barely at all, so $-\Delta$ behaves as a smoothness prior, which is once again a functional of the model's beliefs. The section's conclusion restated formally, the leak is not a ranking deficiency inside Σ_{mono}, it is q itself pointing at the wrong grid, and a repair must move q, which is training, not selection. Figure 15 shows the escape attempt's margins.

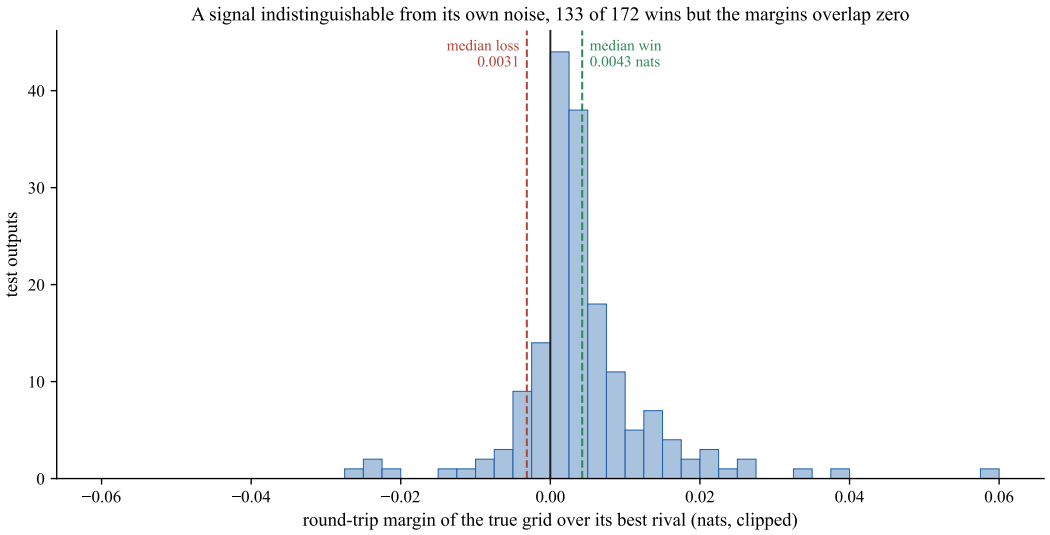


Figure 15: Margins of the round-trip signal on the 172-output gate, the true grid's advantage in Δ over its best synthetic rival. The truth wins 133 of 172 against a chance rate of one in five, but the mass hugs zero and the median winning margin of 0.0043 nats sits against a median losing margin of 0.0031, so the discrimination lives inside its own noise. Outputs beyond 0.06 nats are clipped into the edge bins.

The negative extends from the monotone class to everything the pipeline can compute about itself. Model the task as a rule θ drawn from a prior, with truth $Y^\star = g_\theta(x_\star)$, and let q stand for the trained model's entire belief state on the task, of which the sampling distribution over grids is the visible part. Let W collect everything the pipeline computes internally, the pool, its statistics, and any negative log-likelihood the model can be asked to evaluate. Two assumptions. The pool is exchangeable given q, which is how it is generated. And the internal-channel assumption, $Y^\star \perp W \mid q$, the internal data reads the task only through the model's beliefs. The second is the load-bearing idealization, and everything below is conditional on it.

Theorem (internal ceiling). Under the two assumptions, every two-attempt rule σ that is a function of W satisfies

$$\mathbb{E}\big[\mathbf{1}\{Y^\star \in \sigma(W)\}\big] \;\le\; \mathbb{E}\Big[\max_{|A|=2} \sum_{c\in A} P\big(Y^\star = c \mid q\big)\Big], \tag{8}$$

the expected top-two mass of the posterior given the model's beliefs. *Proof.* Tower over (W,q), note $\sigma(W)$ is fixed given W, replace $P(Y^\star = c \mid W,q)$ by $P(Y^\star = c \mid q)$ using the independence, and bound the resulting two-term sum by the best two-term sum pointwise. □

Equality needs the top-two set of the q-posterior to be recoverable from W, which holds approximately once the pool is large. The reading is Blackwell's, the map from q to W is a garbling, and no processing of a garbled channel creates information about a variable the channel never saw. Two corollaries follow. The round-trip score of Equation (7) is built entirely from model likelihoods, so Δ is a function of W and the

theorem bounds it, which retires that family at theorem level rather than experiment level and explains the measurement, the signal collapsed into a functional of the model's beliefs because it never had anything else to be. And the escape routes are exactly the channels that break the assumption, executing a candidate program against the demonstrations, or scoring under a second independently trained model, both of which read the task through something other than q.

We tested the second escape in the strongest form available offline and it did not pay. Every one of the 432 candidates in the dumped pool was rescored under the *un-adapted* base, which shares weights with the deployed model but not beliefs, having never seen the sixteen task-specific augmentations the adapter trained on. Against the deployed selector's 15 of 60 outputs with the truth in the top two, that likelihood scores 12 alone, 15 when added to the deployed score, and 14 when combined with multiplicity: no combination we tried recovers a single additional output, and the correct-in-pool total it is competing for is 19. The reading is not that a second opinion is worthless in general but that these particular dominators are not an artifact of the adapter. A belief state that never saw this task's augmentations ranks them above the truth as well, so the domination is a property of the candidates rather than of the training, and the cheap reason to expect a second model to help is gone.

We then ran the version the theorem actually names, rescoring the same pool under the separately trained 2B sibling of our base, and it appeared to work spectacularly. Its likelihood alone places the truth in the top two for **19** of 60 outputs against the deployed selector's 15, and 19 is exactly the number of outputs whose pool contains a correct grid at all. A perfect score against the reachable ceiling is not a result, it is a symptom, and the check took five minutes. That model assigns the correct grid a median per-token loss of **0.0002**, and it prefers the true grid over every decoded alternative on 54 of 60 outputs including **40 of the 41 where the correct grid never entered the pool at all**. It is not ranking candidates. It has memorized the public evaluation set, and a model that has memorized an answer scores it highly whether or not it could ever produce it.

A likelihood alone cannot separate memorisation from capability, since a genuinely strong model also assigns low loss, so we built the audit the claim actually requires and pre-registered the gates that could kill it. First the encoding was proved exact, the scored token ids decode back to the true grids on all 172 outputs under both tokenizers with zero mismatches, so the likelihoods are computed on the truth rather than on an artifact of a sixteen-token tokenizer that had already produced one silent zero-supervision bug. Then the likelihood was made to predict something falsifiable. A model at 0.0002 nats per token must be able to produce these answers outright, so both models greedy-decoded the public evaluation set with no test-time training and no search, and the prediction held, the 2B reproduces **80%** of the answers exactly (48 of 60) where the 4B manages 10%. The decisive arm is fresh data. RE-ARC ships the generators and verifiers for the ARC-1 concepts, so we minted 120 instances under our own seed on 1 August 2026, each instance checked by RE-ARC's own verifier, giving a column that provably postdates every published checkpoint. On that column the ordering reverts to what capability predicts, the 4B at **78.3%** against the 2B's **65.0%**. The interaction is $+83.3$ points with $z = 8.0$ (Figure 16). A 2B does not beat a 4B from the same author eight-fold on the harder published set while losing to it on easier fresh data. The narrow claim, and the only one the evidence licenses, is that the public evaluation set is not a valid measurement instrument for this checkpoint. Nothing here speaks to intent, training on the public split is standard practice in a competition scored on a private one; the finding is about what the artifact can measure, not

about the conduct of its author.

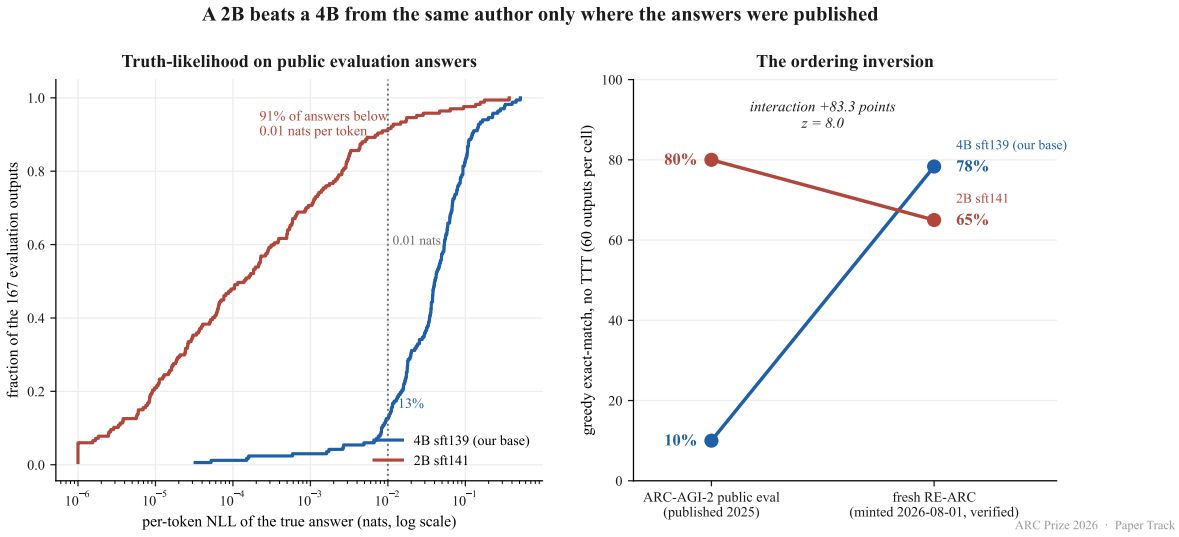


Figure 16: The contamination audit. Left, the distribution of per-token negative log-likelihood assigned to the true public evaluation answers, with 91% of the 2B's answers below 0.01 nats against 13% for the 4B. Right, greedy exact-match with no test-time training on the public evaluation set against freshly generated RE-ARC instances minted after both checkpoints were published. Capability ordering holds on fresh data and inverts on the published set, interaction $+83.3$ points, $z = 8.0$.

The audit reshapes the escape-route ledger rather than closing it. The un-adapted base fails as a second opinion by agreeing with the dominators, and the 2B's apparent 19-of-19 was recall, so no *validated* second-model gain exists. But the fresh column also shows the 2B is a genuinely capable model, 65% on data it cannot have seen, whose defect is specific to the published split.

We then ran the second route on data neither model could have memorised. Both experts were trained per task on 24 freshly generated RE-ARC instances, the 4B decoded the pool, and every unique candidate was rescored under both models over eight fixed augmentations. Of 23 outputs with a non-empty pool, 18 contained the truth, and the deployed single-expert selector already placed all 18 in its top two. **The oracle gap available to any fusion of the two experts is 0 of 18.** There is no headroom on fresh data for a second belief state to recover, because the first one is not making selection errors there; it is failing to generate. The apparent 19-of-19 on public data was recall, and the fresh replication shows the route has nothing left to give even when contamination is removed. We record this as a closure rather than an untested possibility.

The other named escape, executing task-side structure against the demonstrations, was run on the hidden set as v7.5a: the symbolic arm's object-count signal restored as a pure re-ranking prune, adding 0.5 to a beam's score per unit of disagreement with the predicted component count. It scored **27.92**, which against twelve runs of one immutable kernel version is 1.5 standard deviations below the mean and therefore *not* a demonstration of harm on its own. That figure has moved three times as the variance series grew, always toward the null, which is itself the argument for not resting a conclusion on it. The conclusion below

rests instead on the offline invariant-mining experiment, which reaches it with a within-task comparison that no hidden-set noise touches. The theorem is not touched, but its escape clause acquires a measured precondition: a task-side oracle only lifts the bound if it is more accurate than the model's own beliefs about the same property, and this one, which returns a count on every branch including its no-rule fallback, is not. A noisy oracle is strictly worse than none, because the bound it escapes is lower than the penalty it inflicts.

That precondition now has a positive control we did not have to run. Concurrent work evolves Python transformation programs for ARC-AGI-2 under a fitness that is 90% *execution* of the candidate against the task's own demonstration pairs, and reports an open-weights model rising from 12.1% to 34.0%, with a stronger model reaching 61.4% [19]. That is the same channel our theorem names, with the one property ours lacked. Execution either reproduces the demonstrations or it does not, so the oracle is exact by construction, whereas ours predicted a property from the same machinery whose beliefs it was meant to correct. Read together, the two results bracket the escape clause from both sides: exact verification nearly triples an open model's score, and approximate verification of the same kind of structure costs us three sigma. The route is real, and oracle accuracy is the whole of it. The open engineering question, which we do not answer, is whether execution-verified synthesis fits the offline track's roughly $0.20 per task when the published demonstration of it runs at $2.42 to $8.71.

The contamination finding carries one further caution for anyone building on these artifacts. Any measurement that uses that checkpoint to score, filter or validate against public ARC-AGI-2 evaluation tasks will be inflated, and the inflation is invisible unless absolute likelihoods are inspected or fresh data is minted. We report it because the published model card does not, and because a 19-of-19 result is exactly the kind of number this paper has already been fooled by once. The instrument generalises beyond this case, establish a capability ordering on data generated after publication, measure the same models on the suspect set, and read contamination off the interaction; it needs no access to training data, no private split, and no cooperation from the model's author. It has a place in a literature. Performance-based contamination testing exists in general form [10], and regenerating a benchmark from scratch to measure the drop was done for grade-school arithmetic [11]; the field's own 2025 technical report names the ARC version of the problem, "knowledge overfitting", and states explicitly that it does not quantify it [3]. The ordering inversion on generator-fresh instances is, to our knowledge, the first instrument that quantifies it for ARC checkpoints, and the generator that makes the fresh column possible ships with the benchmark's own ecosystem [6].

Identifiability closes the pool-only question from below. The pool is exchangeable given q, so its law depends on the draws only through the recorded multiset, the empirical measure is sufficient, and the best pool-only rule is a function of it. If two rules $\theta_0 \neq \theta_1$ induce the same q but different truths, the pool laws coincide and the posterior odds equal the prior odds, the pool contributes nothing toward telling the truths apart. A quantitative form needs three hypotheses rather than two, since with two confusable truths a two-attempt rule submits both and never errs. Take three rules with pairwise distinct truths whose pool laws sit within total variation η of a common anchor. Any two-attempt rule misses at least one of the three truths pointwise, so the three error probabilities sum to at least $1-2\eta$, and the worst-case error of any pool-only rule is at least $(1-2\eta)/3$, approaching one third as the profiles coincide. The three outputs whose truth was decoded exactly once are the natural place to hunt for such triples. Published separations

sharpen the margins. With Δ_{gap} the gap between the truth's mass and the best wrong mass, majority-style selection needs order $1/\Delta_{\text{gap}}^2$ samples and best-of-n order $1/\Delta_{\text{gap}}$, and a nonpositive gap, the measured case here, defeats every sample size [9], while coverage-style analyses tie the value of any inference-time selection to the policy's mass on the truth [12].

What survives for practice is an estimator design, stated as a proposition. To first order the round trip decomposes as $\Delta(c) = \alpha K(c) + \beta R(c) + \varepsilon$, a description-length term, a rule-consistency term, and noise, and the measured pathology is the first term dominating. Let π scramble the test input by a fixed cell permutation, preserving the candidate, its size and its palette while destroying the spatial link between input and candidate.

Proposition (control variate).

$$\widetilde{\Delta}(c) \;=\; \Delta(c \mid x_\star) - \Delta(c \mid \pi(x_\star)) \;=\; \beta R(c) + (\varepsilon - \varepsilon'), \tag{9}$$

the complexity term cancels to first order, and detecting a mean effect βR at z standard errors needs $F \geq (z\sigma/\beta R)^2$ held-out folds, with the measured margin scale of 0.0043 nats setting the bar. The ceiling still applies, $\widetilde{\Delta}$ is built from model likelihoods, so at best it recovers the consistency component the recorded statistics discard and cannot exceed Equation (8). Any deployment claim passes through the discipline of F.2 and F.3, stated before testing, validated on a disjoint pool.

F.5 Time as the decision variable

Let $p_i(t)$ be the probability task i is solved when given t seconds, and let T be the wall. The allocation problem is to maximize $\sum_i p_i(t_i)$ subject to $\sum_i t_i \leq T$. For concave increasing p_i the Karush-Kuhn-Tucker conditions give water-filling, $p_i'(t_i) = \lambda$ on every task receiving time and $p_i'(0) \leq \lambda$ on the rest, marginal solve probability per second equalized. The measured behaviour of this pipeline is closer to a step. A task completes its search at a natural time τ_i or fails regardless, $p_i(t) \approx s_i \mathbf{1}[t \geq \tau_i]$, which turns the allocation into a 0/1 knapsack over (s_i, τ_i). Since τ_i is unknown until the task is run, the implementable policy is a uniform cap C, and under sequential processing against the wall the outcome is

$$k(C) \;=\; \max\Big\{k \;:\; \sum_{i \leq k} \min(\tau_i, C) \;\leq\; T\Big\}, \qquad S(C) \;=\; \sum_{i \leq k(C)} p_i\big(\min(\tau_i, C)\big). \tag{10}$$

Raising C weakly raises every surviving term and weakly lowers $k(C)$, and each task beyond $k(C)$ contributes zero, so $S(C)$ is in general non-monotone and its maximizer can be interior. When the wall never binds, larger C is weakly better, which is why a 120-task public run cannot exhibit the effect while a 240-task hidden run can, and why the public and hidden reads of a cap change may legitimately disagree. The measured inputs are a τ range of 174 to 1889 seconds on the 40-task slice, a deployed cap of 1800 on every submission through the wiring fault of Section 4. The strongest public build was believed to run 1200, which is what made the cap the leading candidate for the gap. An earlier draft withdrew that candidate, on the grounds that the build forks a source hardcoding 1800 and that both sides therefore ran the same cap. That withdrawal read the wrong notebook. It inspected the build we had forked as a control, which is not the build carrying the score. Pulling `mikelou1/arc-agi2-lb33-89-minimal-perfpatch` through the Kaggle API on 1 August 2026 and reading its source shows a literal 1200-second per-task

cap written inside its own decode loop, on the effective path, inherited from nothing. The candidate was reinstated. The two builds did not run the same cap, and for two days the cap was again the leading unexplained difference between that build and ours. This is the second time in this work that a conclusion about a public notebook was reached without reading the notebook that actually holds the score, and the correction cost nothing only because the check is a single API call. The control then priced the reinstatement. The identified build, forked unmodified at its own 1200-second cap, scored 28.47 against our 30.97 at 1800 (Section 4.1.1), so the number the cap was reinstated to explain does not itself reproduce, and the hypothesis survives only as an untested one-factor question with its empirical motivation removed. The model below stands as mathematics and is reported because the cap remains a real lever on a 240-task queue, not because any measured gap is now attributed to it. Whether the wall binds on the hidden set is not observable from outside, the model predicts sensitivity to C exactly when $\sum_i \min(\tau_i, C)$ approaches T. The submission built to decide it did not. It moved the cap from 1800 to 1200 and lost 1.53 points, four credits against a one-credit noise floor, which reads as a violation of the flat-plateau prediction until one notices that the same run also executed 24 unbudgeted greedy fills through a gate that treated the string `"0"` as true. Two factors moved, so the experiment is void rather than adverse, and the prediction stands untested. We report it as void because a confounded run that happens to point the way our model does not is exactly the run one is tempted to quietly keep. One operational corollary came from the same smoke that validated the cap. Any work placed after the capped loop re-enters as unaccounted growth in τ_i, the greedy fallback measured about 80 seconds per fill past the cap, so post-loop work must carry a budget of its own. Figure 17 draws the model against the measured timings.

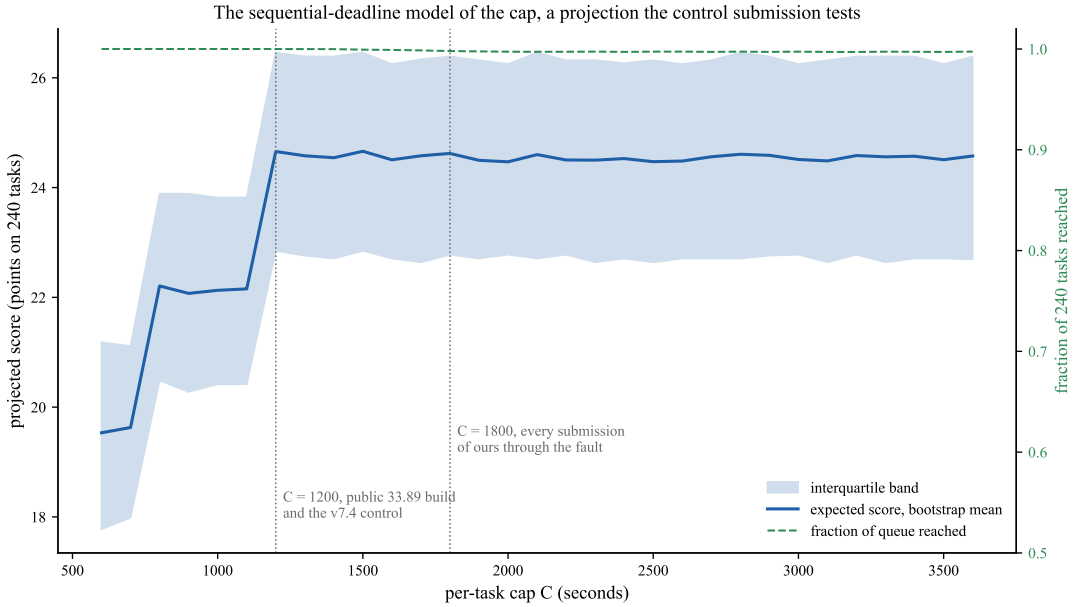


Figure 17: The sequential-deadline model of Equation (10), bootstrapped from the forty measured task timings and the credits the measurement run harvested. Under the public-eval timing distribution the wall never binds, the queue is always fully reached, and the projected score is flat from a cap of 1200 upward, so the model itself predicts the cap change is neutral unless the hidden tasks run meaningfully slower than the public ones. That is a pre-registered prediction, not a hope. Its intended test was voided by a second uncontrolled change in the same run and it remains open. The projection carries stated limits, forty samples, right-censoring at the deployed 1800, and yield only on completed searches.

The model has a closed form worth recording, because it turns the cap from a swept knob into a solved problem given one estimable distribution. Write F for the distribution of the natural time τ, $\bar{F} = 1 - F$, $h = f/\bar{F}$ its hazard, $\mu(C) = \mathbb{E}\min(\tau, C) = \int_0^C \bar{F}(t)\,dt$ the capped mean, and $\nu(C) = \mathbb{E}[s\,\mathbf{1}\{\tau \le C\}]$ the capped credit. Tasks consume independent amounts $\min(\tau_i, C)$, so by Wald's identity the queue admits

$$\mathbb{E}[k(C)] \;\approx\; \min\!\left(n,\; \frac{T_w}{\mu(C)}\right), \qquad \text{the wall binds} \iff \mu(C) > T_w/n, \tag{11}$$

with T_w the per-worker wall and n its queue share. The numbers are immediate. Net of the deadline margin and startup, $T_w \approx 42000$ seconds and $n = 60$, so the threshold is about 700 seconds, and the measured capped mean is $\hat{\mu}(1800) = 24821/40 = 620.5$ seconds, needing no censoring correction since the observations are themselves capped at the deployed limit, up to the small overshoot of the between-batch check. The wall is borderline non-binding, which is the analytic route to the verdict Figure 17 reached by bootstrap, and two independent methods agreeing is the reason to trust the flat plateau.

In the regime where the wall does bind, the objective is reached-count times credit per reached task, and the optimum has a first-order condition,

$$\mathbb{E}[S(C)] \;\approx\; \frac{T_w}{\mu(C)}\,\nu(C), \qquad \mathbb{E}[s \mid \tau = C]\,h(C) \;=\; \frac{\nu(C)}{\mu(C)} \;\text{ at } C^*, \tag{12}$$

extend the cap until the hazard-weighted marginal credit per second falls to the average credit per second. With task values constant the condition reduces to $f(C)\,\mu(C) = F(C)\,\bar{F}(C)$, equivalently $C^* =$

$\arg\min_C \mu(C)/F(C)$, and that ratio is exactly the fixed-cutoff objective of restart theory [8], whose universal schedule pays a logarithmic factor precisely for not knowing F. Here F is estimable, so the fixed cap is the right target, and the coincidence of the renewal derivation with the restart objective is a consistency check between two independent routes. Estimating F beyond the historical cap needs the censoring handled, everything above 1800 is right-censored, and the product-limit estimator with the capped-mean integral does it, after which a bootstrap resamples the joint rows to preserve the dependence between training loss, time and yield, imputes the censored tail, and simulates the four queues. The decision rule ties back to F.2, a cap change ships only when the bootstrap's lower confidence bound on the gain clears two credits, or a live paired read clears the six-flip bound.

Three classical problems sit adjacent, and each maps with a break worth naming. Pandora's box gates inspection on an index, and the deployed loss cut is that gate in disguise, run the expensive search only when the conditional credit per second given the cheap pre-search signal clears the budget's shadow price, and the measurement that no task finishing above loss 0.05 ever yielded is the index saying skip. The stochastic-knapsack adaptivity gap bounds how much any signal-adaptive cap can beat the best static one, a constant factor, so the static C^* is the primary lever and the signal is a bounded refinement, though the imported constant is heuristic here since the policy classes differ. And the prophet inequality's half-optimal fixed threshold is the floor under all of it, one fixed cap is never catastrophically worse than clairvoyance.

F.6 Composition and coverage

A heuristic closes the appendix because it frames the benchmark itself. If evaluation tasks are compositions of k primitives drawn from a library of size n, the composition space grows like n^k while any training set covers a fixed m of them, so coverage m/n^k vanishes and memorizing compositions cannot transfer. The design intent of the benchmark is exactly this regime, and the body measured its signature, a continued fine-tune on the public training distribution scored 11.63 against the stock model's 12.79 (Section 5.3). What transfers is whatever captures the primitives and the composition operator rather than the compositions, which is the formal reason the interesting levers in this work kept reducing to training-side changes, and why compression-style approaches that fit tiny models per task remain interesting despite modest absolute scores.

References

[1] F. Chollet. On the Measure of Intelligence. arXiv:1911.01547, 2019.

[2] ARC Prize Foundation. ARC-AGI-2 Technical Report. 2025. `https://arcprize.org`.

[3] ARC Prize Foundation. ARC Prize 2025: Technical Report. arXiv:2601.10904, 2026.

[4] NVIDIA Kaggle Grandmasters (NVARC). ARC Prize 2025 (ARC-AGI-2) 1st-place solution. 2025.

[5] Qwen Team. Qwen3 Technical Report. 2025.

[6] M. Hodel. Addressing the Abstraction and Reasoning Corpus via Procedural Example Generation. arXiv:2404.07353, 2024.

[7] W. Lemes de Oliveira et al. ARC-AGI-2 Technical Report (LongT5-based system with test-time training). arXiv:2603.06590, 2026.

[8] M. Luby, A. Sinclair, and D. Zuckerman. Optimal speedup of Las Vegas algorithms. *Information Processing Letters*, 47(4):173–180, 1993.

[9] B. Huang, S. Li, T. Wu, Y. Yang, A. Talwalkar, K. Ramchandran, M. I. Jordan, and J. Jiao. Sample complexity and representation ability of test-time scaling paradigms. arXiv:2506.05295, 2025.

[10] J. Dekoninck, M. Müller, and M. Vechev. ConStat: Performance-Based Contamination Detection in Large Language Models. arXiv:2405.16281, 2024.

[11] H. Zhang et al. A Careful Examination of Large Language Model Performance on Grade School Arithmetic. arXiv:2405.00332, 2024.

[12] A. Huang, A. Block, Q. Liu, N. Jiang, A. Krishnamurthy, and D. J. Foster. Is best-of-*N* the best of them? Coverage, scaling, and optimality in inference-time alignment. arXiv:2503.21878, 2025.

[13] ARC Prize Foundation. ARC Prize 2024: Technical Report. arXiv:2412.04604, 2024.

[14] D. Franzen, J. Disselhoff, and D. Hartmann. Product of Experts with LLMs: Boosting Performance on ARC Is a Matter of Perspective. arXiv:2505.07859, 2025.

[15] E. Akyürek, M. Damani, L. Qiu, H. Guo, Y. Kim, and J. Andreas. The Surprising Effectiveness of Test-Time Training for Few-Shot Learning. arXiv:2411.07279, 2024. ICML 2025.

[16] Y. Sun, X. Wang, Z. Liu, J. Miller, A. A. Efros, and M. Hardt. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. ICML 2020.

[17] Y. Gandelsman, Y. Sun, X. Chen, and A. A. Efros. Test-Time Training with Masked Autoencoders. NeurIPS 2022. arXiv:2209.07522.

[18] I. Liao and A. Gu. ARC-AGI Without Pretraining. arXiv:2512.06104, 2025.

[19] Imbue. Beating ARC-AGI-2 with Code Evolution. 2026. `https://imbue.com/research/2026-02-27-arc-agi-2-evolution/`.

[20] NVIDIA. Reimagining LLM Memory: Using Context as Training Data Unlocks Models That Learn at Test Time (TTT-E2E). NVIDIA Developer Blog, 2026. `https://developer.nvidia.com/blog/reimagining-llm-memory-using-context-as-training-data-unlocks-models-that-learn-at-test-ti`

[21] S. Willison. Qwen 3.8 27B is excellent, but it defaults to wildly overthinking things. 2026. `https://simonwillison.net/2026/Aug/16/qwen-38-27b/`.